TP Diseño y Optimización de Bases de Datos — Cadena de Hoteles

Oct 11, 2026 · @Francisco Javier Paredes

Introducción

El trabajo se implementa sobre **PostgreSQL 16.15**. Se eligió este motor porque resuelve en la propia base la regla más delicada del dominio, la no superposición de reservas, mediante una restricción `EXCLUDE` sobre rangos de fechas, y porque ofrece de forma nativa índices funcionales, parciales, GiST, BRIN y vistas materializadas, que se estudian en las Partes III a X.

Todo lo que se afirma en el informe está respaldado por mediciones reales: cada cifra se obtuvo ejecutando el código que se documenta en el punto correspondiente sobre una base de **7.535.580 reservas** y 1.000.000 de huéspedes. Todas las tablas viven en el esquema `hotel`. El procedimiento de medición se describe en el Punto 22.

Parte I — Análisis del Problema y Diseño de Datos

1. Análisis del dominio

El dominio se resuelve con 20 tablas agrupadas en cinco áreas; la entidad central es **Reserva**, que vincula un huésped con una habitación durante un intervalo de fechas.

Entidades y atributos principales

Área	Entidad	Atributos relevantes	Observación
Ubicación	País, Provincia, Localidad	código ISO, nombre, código postal	Jerarquía para no repetir nombres de provincia o país en cada dirección
Ubicación	Domicilio	calle, número, piso, depto., localidad	Estructura común a hoteles y huéspedes
Hotel	Hotel	nombre, categoría (estrellas), descripción, coordenadas, horarios de check-in/out, activo	Un domicilio por hotel

Área	Entidad	Atributos relevantes	Observación
Hotel	Contacto de hotel	tipo (teléfono, email, web, WhatsApp), valor, principal	Multivaluado: un hotel tiene varios
Hotel	Punto de interés, Tipo de punto	nombre, tipo, localidad, coordenadas	Relación N:M con hotel; la distancia es atributo de la relación
Habitación	Tipo de habitación	nombre, capacidad máxima	Catálogo global (Single, Doble, Suite…)
Habitación	Habitación	hotel, número, piso, tipo, activa	El número es único dentro de cada hotel
Habitación	Comodidad	nombre	N:M con habitación
Habitación	Tarifa (de lista)	hotel, tipo, vigencia desde/hasta, precio por noche	Precio de referencia vigente en un período
Huésped	Huésped	tipo y nro. de documento, nombre, apellido, nacimiento, nacionalidad, domicilio	Identificado por su documento
Huésped	Teléfono, Email	número/dirección, tipo, principal	Multivaluados: puede tener varios
Reserva	Estado de reserva	código, descripción, si ocupa la habitación	P, C, F, X, N
Reserva	Reserva	habitación, huésped, fecha inicio, fecha fin, cantidad de huéspedes, estado, tarifa por noche, fechas de creación y confirmación	Tabla de mayor volumen del sistema

Decisiones de interpretación

- **Comodidades por habitación, no por tipo.** El enunciado dice que los tipos de habitación tienen "determinadas comodidades", pero dos habitaciones dobles del mismo hotel pueden diferir (una con balcón, otra sin). Asignarlas a la habitación es más flexible; las comodidades típicas de un tipo se obtienen agregando.
- **Dos conceptos de tarifa.** La *tarifa de lista* es el precio vigente por hotel y tipo de habitación. La *tarifa pactada* (`reserva.tarifa_noche`) es la que se fija al confirmar. No es redundancia: la lista cambia con el tiempo y la reserva debe conservar el precio

acordado.

- **Intervalo semiabierto [inicio, fin).** `fecha_fin` es el día de salida y no se cobra. Así, una reserva que termina el 15 y otra que empieza el 15 no se superponen, y la cantidad de noches es `fecha_fin - fecha_inicio`.
- **Un titular por reserva.** Se registra el huésped titular y la cantidad de personas; el enunciado no pide registrar a cada acompañante.

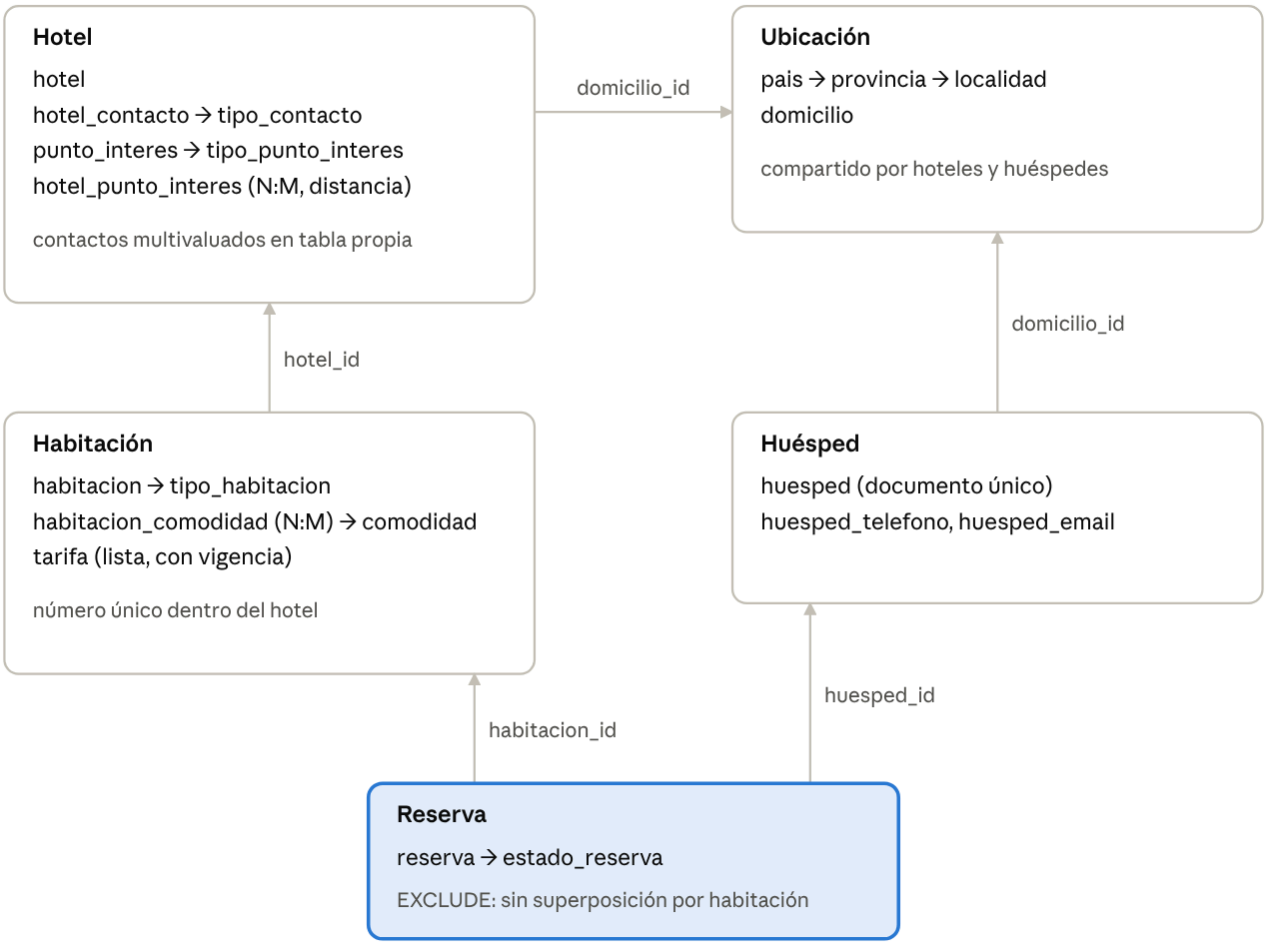
Reglas de negocio identificadas

1. **RN1** — Una habitación no puede tener dos reservas activas (pendiente, confirmada o finalizada) con períodos superpuestos.
2. **RN2** — La fecha de salida es posterior a la de inicio; una estadía dura como máximo 90 noches.
3. **RN3** — La tarifa se establece únicamente al confirmar: una reserva pendiente no tiene tarifa; una confirmada o finalizada sí, junto con su fecha de confirmación.
4. **RN4** — La cantidad de huéspedes no supera la capacidad del tipo de habitación.
5. **RN5** — Un huésped se identifica por tipo y número de documento; no puede repetirse.
6. **RN6** — Un huésped o un hotel puede tener varios medios de contacto, pero como máximo uno *principal* por tipo.
7. **RN7** — El número de habitación es único dentro del hotel.
8. **RN8** — Las tarifas de lista de un mismo hotel y tipo de habitación no pueden tener vigencias solapadas.
9. **RN9** — No se puede eliminar un hotel, habitación o huésped que tenga reservas asociadas (se preserva el historial).
10. **RN10** — La categoría del hotel va de 1 a 5 estrellas; las coordenadas están dentro de rangos geográficos válidos.
11. **RN11** — Una reserva cancelada o *no-show* libera la habitación: no participa de la regla RN1.

2. Modelo relacional

El modelo tiene 20 relaciones. Notación: **PK** clave primaria, **FK** clave foránea (→ tabla referenciada), **UQ** clave candidata única.

Reserva une un huésped con una habitación; el resto cuelga de ellas



modelo relacional · 5 áreas, 20 tablas

Las flechas siguen la dirección de las claves foráneas (de la tabla hija a la padre). Toda consulta de negocio parte de `reserva` y sube hacia la habitación, el hotel o el huésped.

Ubicación

- **pais** (pais_id PK, codigo_iso UQ, nombre UQ)
- **provincia** (provincia_id PK, pais_id FK→pais, nombre) — UQ(pais_id, nombre)
- **localidad** (localidad_id PK, provincia_id FK→provincia, nombre, codigo_postal) — UQ(provincia_id, nombre)
- **domicilio** (domicilio_id PK, localidad_id FK→localidad, calle, numero, piso, departamento, codigo_postal)

Hotel

- **hotel** (hotel_id PK, nombre UQ, categoria, descripcion, domicilio_id FK→domicilio UQ, latitud, longitud, hora_checkin, hora_checkout, activo, fecha_alta)

- **tipo_contacto** (tipo_contacto_id PK, nombre UQ)
- **hotel_contacto** (hotel_contacto_id PK, hotel_id FK→hotel, tipo_contacto_id FK→tipo_contacto, valor, principal) — UQ(hotel_id, tipo_contacto_id, valor)
- **tipo_punto_interes** (tipo_punto_interes_id PK, nombre UQ)
- **punto_interes** (punto_interes_id PK, tipo_punto_interes_id FK, localidad_id FK→localidad, nombre, descripcion, latitud, longitud) — UQ(localidad_id, nombre)
- **hotel_punto_interes** (hotel_id FK→hotel, punto_interes_id FK→punto_interes, distancia_m) — PK(hotel_id, punto_interes_id)

Habitación

- **tipo_habitacion** (tipo_habitacion_id PK, nombre UQ, capacidad_maxima, descripcion)
- **comodidad** (comodidad_id PK, nombre UQ)
- **habitacion** (habitacion_id PK, hotel_id FK→hotel, tipo_habitacion_id FK→tipo_habitacion, numero, piso, activa) — UQ(hotel_id, numero)
- **habitacion_comodidad** (habitacion_id FK→habitacion, comodidad_id FK→comodidad) — PK(habitacion_id, comodidad_id)
- **tarifa** (tarifa_id PK, hotel_id FK→hotel, tipo_habitacion_id FK→tipo_habitacion, vigencia_desde, vigencia_hasta, precio_noche)

Huésped

- **huesped** (huesped_id PK, tipo_documento, numero_documento, nombre, apellido, fecha_nacimiento, nacionalidad_id FK→pais, domicilio_id FK→domicilio UQ, fecha_registro) — UQ(tipo_documento, numero_documento)
- **huesped_telefono** (huesped_telefono_id PK, huesped_id FK→huesped, tipo, numero, principal) — UQ(huesped_id, numero)
- **huesped_email** (huesped_email_id PK, huesped_id FK→huesped, email, principal) — UQ(huesped_id, email)

Reserva

- **estado_reserva** (estado PK, descripcion UQ, ocupa)
- **reserva** (reserva_id PK, habitacion_id FK→habitacion, huesped_id FK→huesped, fecha_inicio, fecha_fin, cantidad_huespedes, estado FK→estado_reserva, tarifa_noche, fecha_creacion, fecha_confirmacion, observaciones)

Transformaciones aplicadas al pasar del modelo conceptual al relacional

Situación conceptual	Transformación	Ejemplo
Relación 1:N	FK en la tabla del lado N	`habitacion.hotel_id`, `reserva.huesped_id`
Relación N:M	Tabla intermedia con PK compuesta por las dos FK	`habitacion_comodidad`, `hotel_punto_interes`
N:M con atributo propio	El atributo va en la tabla intermedia	`distancia_m` en `hotel_punto_interes`
Atributo multivaluado	Tabla propia con FK al dueño	teléfonos y emails del huésped, contactos del hotel
Atributo compuesto (domicilio)	Tabla aparte, referenciada con FK UNIQUE (1:1)	`hotel.domicilio_id`, `huesped.domicilio_id`
Dominio cerrado que puede crecer	Tabla catálogo	`tipo_contacto`, `tipo_habitacion`, `estado_reserva`
Dominio cerrado y estable	CHECK sobre la columna	`tipo_documento`, `huesped_telefono.tipo`
Identificador natural largo o mutable	Clave subrogada + UQ sobre la natural	`huesped`: PK `huesped_id`, UQ(tipo_documento, numero_documento)

Se usan **claves subrogadas** generadas por identidad en todas las entidades porque las claves naturales (documento, nombre del hotel) son largas, compuestas o pueden corregirse. Propagar una clave de 8 bytes en `reserva` es más barato que propagar un documento de hasta 30 bytes en millones de filas, y una corrección del documento no obliga a actualizar las reservas. La clave natural se conserva como **UNIQUE** para no perder la regla de unicidad del negocio.

En las tablas intermedias N:M la clave es **natural compuesta** (las dos FK), ya que ese par es justamente lo que no debe repetirse.

3. Normalización

Todas las relaciones del modelo están en **Forma Normal de Boyce-Codd (FNBC)**: en cada una, el lado izquierdo de toda dependencia funcional no trivial es una superclave. El camino se muestra partiendo de una tabla única de reservas, como la que surgiría de una planilla.

Punto de partida (sin normalizar)

`RESERVA_PLANA (nro_reserva, hotel, direccion_hotel, provincia_hotel, telefonos_hotel, nro_habitacion, piso, tipo_hab, capacidad, comodidades, dni, nombre, apellido, telefonos_huesped, emails_huesped, domicilio_huesped, fecha_inicio, fecha_fin, estado, tarifa)`

1FN — atributos atómicos

`telefonos_hotel`, `comodidades`, `telefonos_huesped` y `emails_huesped` son multivaluados: violan 1FN. Se separan en `hotel_contacto`, `habitacion_comodidad`, `huesped_telefono` y `huesped_email`, cada una con una fila por valor. `domicilio` es compuesto y se descompone en calle, número, piso, depto. y localidad.

2FN — sin dependencias parciales

En `habitacion_comodidad` (PK compuesta habitacion_id + comodidad_id) no se guarda el nombre de la comodidad: dependería sólo de `comodidad_id`. Lo mismo en `hotel_punto_interes`: el nombre del punto depende sólo de `punto_interes_id`, mientras que `distancia_m` sí depende del par completo y por eso queda en la tabla intermedia. Las tablas con PK simple están en 2FN trivialmente.

3FN — sin dependencias transitivas

Dependencias funcionales identificadas en la tabla plana y su resolución:

Dependencia transitiva	Problema si se mantiene	Resolución
reserva → habitación → hotel	El hotel se repite en cada reserva	`reserva` guarda sólo `habitacion_id`
habitación → tipo_hab → capacidad	La capacidad se repite por habitación	`tipo_habitacion(capacidad_maxima)`
reserva → dni → nombre, apellido	Datos del huésped repetidos en cada reserva	`huesped`, referenciado por `huesped_id`
domicilio → localidad → provincia → país	El nombre de la provincia se repite en cada dirección	Jerarquía `localidad → provincia → pais`

Dependencia transitiva	Problema si se mantiene	Resolución
reserva → estado → descripción	Texto del estado repetido 6 millones de veces	Catálogo `estado_reserva`

Dependencias funcionales que quedan en cada relación final (ejemplos):

- `reserva`: reserva_id → {habitacion_id, huesped_id, fecha_inicio, fecha_fin, cantidad_huespedes, estado, tarifa_noche, …}. No hay otra dependencia: `tarifa_noche` **no** depende de la habitación ni del período, porque es el precio pactado en esa reserva.
- `habitacion`: habitacion_id → todo; (hotel_id, numero) → todo. Ambas son claves candidatas: cumple FNBC.
- `huesped`: huesped_id → todo; (tipo_documento, numero_documento) → todo. Ambas son claves.
- `tarifa`: tarifa_id → todo. (hotel_id, tipo_habitacion_id, fecha) determina una única tarifa, pero esa regla no es una dependencia funcional entre columnas sino una restricción temporal; se garantiza con `EXCLUDE` (RN8).

FNBC

El caso típico que viola FNBC sin violar 3FN es una relación con dos claves candidatas superpuestas. En `habitacion` las dos claves (`habitacion_id` y `hotel_id, numero`) no generan anomalías porque ambas determinan la fila completa. No existen dependencias cuyo determinante no sea superclave.

Anomalías que el modelo evita

Anomalía	En la tabla plana	En el modelo normalizado
Inserción	No se puede dar de alta un hotel ni una habitación sin una reserva	Hotel y habitación existen por sí mismos
Modificación	Cambiar el teléfono de un huésped exige actualizar todas sus reservas; si falla una, queda inconsistente	Se modifica una sola fila en `huesped_telefono`
Eliminación	Borrar la única reserva de un huésped borra sus datos personales	La reserva se elimina sin afectar al huésped

Excepciones conscientes

- `domicilio.codigo_postal` y `localidad.codigo_postal` conviven: una ciudad grande tiene varios códigos postales, por lo que el código depende de la dirección y no sólo de la localidad. No es redundancia.
- `reserva.tarifa_noche` frente a `tarifa.precio_noche`: es una **instantánea histórica**, no una dependencia derivable (ver Punto 1).

4. Integridad y reglas de negocio

Nueve de las once reglas de negocio se garantizan desde la base; las 14 pruebas del script `02_pruebas_integridad.sql` intentan violarlas y la base rechazó todas las que debía rechazar.

Dónde se garantiza cada regla

Regla	Mecanismo	Dónde
RN1 sin superposición	`EXCLUDE USING gist (habitacion_id WITH =, daterange(fecha_inicio, fecha_fin, '[)') WITH &&) WHERE (estado IN ('P','C','F'))`	Base
RN2 fechas coherentes	`CHECK (fecha_fin > fecha_inicio)` y `CHECK (fecha_fin - fecha_inicio <= 90)`	Base
RN3 tarifa al confirmar	`CHECK` que liga `estado`, `tarifa_noche` y `fecha_confirmacion`	Base
RN4 capacidad	Trigger `BEFORE INSERT/UPDATE` (cruza dos tablas, un CHECK no puede)	Base
RN5 documento único	`UNIQUE (tipo_documento, numero_documento)`	Base
RN6 un contacto principal	Índice `UNIQUE` parcial `WHERE principal`	Base
RN7 número de habitación	`UNIQUE (hotel_id, numero)`	Base

Regla	Mecanismo	Dónde
RN8 vigencias de tarifa	`EXCLUDE USING gist` sobre `daterange`	Base
RN9 no borrar con historial	FK con `ON DELETE` restrictivo (por defecto)	Base
RN10 rangos válidos	`CHECK` en categoría y coordenadas	Base
Formato de email y teléfono	`CHECK` con expresión regular (validación sintáctica mínima)	Base + aplicación
Que el email exista y sea del huésped	Envío de un código de verificación	Aplicación
Que la tarifa pactada respete la de lista	Política comercial (descuentos, promociones)	Aplicación
Transiciones válidas de estado (P→C→F, nunca F→P)	Máquina de estados	Aplicación (o trigger, si se quisiera blindar)

Criterio usado: va a la base toda regla cuya violación deja datos inconsistentes sin importar qué programa escriba (una app web, un script de carga, un `psql` manual). Va a la aplicación lo que depende de procesos externos o de políticas que cambian sin cambiar el esquema.

Por qué RN1 no puede quedar sólo en la aplicación

Si la aplicación hace "consulto si está libre y, si lo está, inserto", dos usuarios que reservan la misma habitación al mismo tiempo ven ambos la habitación libre y ambos insertan. Es una condición de carrera clásica. La restricción `EXCLUDE` la resuelve porque su índice GiST se verifica dentro de la misma operación de inserción: la segunda transacción espera a la primera y, al confirmarse ésta, falla con `conflicting key value violates exclusion constraint`. Se demuestra con dos conexiones en la Parte VIII.

Resultado de las pruebas

#	Prueba	Esperado	Resultado obtenido
1	Reserva superpuesta (12 al 18 sobre una del 10 al 15)	Rechazar	Rechazada por `ex_reserva_superpuesta`
2	Reserva contigua (entra el 15, día que sale la otra)	Aceptar	Aceptada

#	Prueba	Esperado	Resultado obtenido
3	Fecha de salida anterior al inicio	Rechazar	Rechazada por `ck_reserva_fechas`
4	Reserva pendiente con tarifa cargada	Rechazar	Rechazada por `ck_reserva_tarifa`
5	Confirmar sin tarifa	Rechazar	Rechazada por `ck_reserva_tarifa`
6	Confirmar con tarifa	Aceptar	Aceptada
7	3 huéspedes en una doble (capacidad 2)	Rechazar	Rechazada por el trigger de capacidad
8	Cancelada superpuesta a otra activa	Aceptar	Aceptada (las canceladas no bloquean)
9	Documento duplicado	Rechazar	Rechazada por `uq_huesped_documento`
10	Dos emails principales para un huésped	Rechazar	Rechazada por `uq_huesped_email_principal`
11	Email sin formato válido	Rechazar	Rechazada por `CHECK` de formato
12	Tarifas de lista con vigencias solapadas	Rechazar	Rechazada por `ex_tarifa_solapada`
13	Borrar huésped con reservas	Rechazar	Rechazada por la FK `reserva_huesped_id_fkey`
14	Hotel de 6 estrellas	Rechazar	Rechazada por `hotel_categoria_check`

Las pruebas 2 y 8 son tan importantes como las rechazadas: confirman que la regla no es más restrictiva de lo que el negocio pide.

5. Normalización versus desnormalización

Se propone una desnormalización controlada: copiar `hotel_id` en `reserva`, garantizando con una clave foránea compuesta que la copia nunca contradiga al original. Su efecto se mide con la base cargada (6 millones de reservas) en la Parte VII.

El problema

Las consultas más frecuentes del negocio son por hotel: ocupación del hotel en un mes, ranking de hoteles por reservas, facturación por hotel. En el modelo normalizado `reserva` no conoce el hotel, así que todas obligan a un `JOIN` con `habitacion`:

```
SELECT h.hotel_id, count(*)
  FROM reserva r
  JOIN habitacion h ON h.habitacion_id = r.habitacion_id
 WHERE r.fecha_inicio >= '2026-01-01' AND r.fecha_inicio < '2026-02-01'
 GROUP BY h.hotel_id;
```

Con millones de reservas, ese `JOIN` se ejecuta sobre todas las filas del período, y no se puede crear un índice `(hotel_id, fecha_inicio)` sobre `reserva` porque la columna no está en la tabla.

La propuesta y cómo se protege la integridad

```
ALTER TABLE habitacion ADD CONSTRAINT uq_habitacion_hotel UNIQUE
(habitacion_id, hotel_id);
ALTER TABLE reserva ADD COLUMN hotel_id INTEGER;
UPDATE reserva r SET hotel_id = h.hotel_id FROM habitacion h WHERE
h.habitacion_id = r.habitacion_id;
ALTER TABLE reserva ALTER COLUMN hotel_id SET NOT NULL,
  ADD CONSTRAINT fk_reserva_hab_hotel FOREIGN KEY (habitacion_id, hotel_id)
      REFERENCES habitacion (habitacion_id, hotel_id) ON UPDATE CASCADE;
CREATE INDEX ix_reserva_hotel_fecha ON reserva (hotel_id, fecha_inicio);
```

La clave foránea compuesta es lo que la hace "controlada": la base rechaza cualquier reserva cuyo `hotel_id` no coincida con el hotel de su habitación, y si una habitación cambiara de hotel el `ON UPDATE CASCADE` propaga el cambio. La anomalía de modificación que la normalización evita queda cubierta por el motor, no por la disciplina del programador.

Costos y beneficios esperados

Aspecto	Efecto
Consultas por hotel y período	Sin `JOIN`; pueden usar el índice `(hotel_id, fecha_inicio)`
Espacio	+4 bytes por reserva (`INTEGER`) más el índice nuevo

Aspecto	Efecto
`INSERT` en reserva	Una verificación de FK más y un índice más para mantener
Riesgo de inconsistencia	Nulo: lo impide la FK compuesta
Particionamiento (Parte X)	Habilita particionar `reserva` por hotel si se quisiera

Segundo escenario: resumen de ocupación

Los tableros de gestión consultan ocupación e ingresos por hotel y mes, sobre datos históricos que ya no cambian. Una **vista materializada** con esos totales es otra forma de desnormalización: lee una tabla de cientos de filas en lugar de millones, a cambio de refrescarla periódicamente. No compromete la integridad porque la fuente de verdad sigue siendo `reserva`. Se implementa y se mide en el Punto 52.

Parte II — Diseño Físico y Almacenamiento

6. Implementación física

El script completo de creación se transcribe a continuación y se ejecuta sin errores sobre una base vacía de PostgreSQL 16. Crea 20 tablas, 1 extensión, 1 función con su trigger, 13 índices explícitos y carga los catálogos fijos. Es re-ejecutable: empieza con `DROP SCHEMA IF EXISTS hotel CASCADE`.

Estructura del script:

1. Esquema `hotel` y extensión `btree_gist` (necesaria para combinar `=` y `&&` en un `EXCLUDE`).
2. Ubicación: `pais`, `provincia`, `localidad`, `domicilio`.
3. Hotel: `hotel`, `tipo_contacto`, `hotel_contacto`, `tipo_punto_interes`, `punto_interes`, `hotel_punto_interes`.
4. Habitación: `tipo_habitacion`, `comodidad`, `habitacion`, `habitacion_comodidad`, `tarifa`.
5. Huésped: `huesped`, `huesped_telefono`, `huesped_email`.
6. Reserva: `estado_reserva`, `reserva`.
7. Función `fn_valida_capacidad()` y trigger `trg_reserva_capacidad`.
8. Índices sobre claves foráneas.
9. Carga de catálogos (`estado_reserva`, `tipo_contacto`).

Script completo de creación:

```
DROP SCHEMA IF EXISTS hotel CASCADE;
CREATE SCHEMA hotel;
SET search_path TO hotel, public;

-- btree_gist permite combinar "=" (habitación) y "&&" (rangos de fechas)
-- en una misma restricción EXCLUDE: impide reservas superpuestas.
CREATE EXTENSION IF NOT EXISTS btree_gist;

-- ------------------------------------------------------------------
-- 1. Ubicación geográfica (país > provincia > localidad)
-- ------------------------------------------------------------------
CREATE TABLE pais (
    pais_id     SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    codigo_iso  CHAR(2)      NOT NULL UNIQUE CHECK (codigo_iso ~ '^[A-Z]
{2}$'),
    nombre      VARCHAR(80)  NOT NULL UNIQUE
);

CREATE TABLE provincia (
    provincia_id SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    pais_id      SMALLINT     NOT NULL REFERENCES pais(pais_id),
    nombre       VARCHAR(80)  NOT NULL,
    CONSTRAINT uq_provincia UNIQUE (pais_id, nombre)
);

CREATE TABLE localidad (
    localidad_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    provincia_id  SMALLINT     NOT NULL REFERENCES provincia(provincia_id),
    nombre        VARCHAR(100) NOT NULL,
    codigo_postal VARCHAR(10),
    CONSTRAINT uq_localidad UNIQUE (provincia_id, nombre)
);

-- Domicilio: lo usan hoteles y huéspedes (misma estructura)
CREATE TABLE domicilio (
    domicilio_id  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    localidad_id  INTEGER      NOT NULL REFERENCES localidad(localidad_id),
    calle         VARCHAR(120) NOT NULL,
    numero        VARCHAR(10)  NOT NULL,          -- admite "S/N", "1234 bis"
    piso          VARCHAR(5),
    departamento  VARCHAR(5),
    codigo_postal VARCHAR(10)
);
```

```
-- ----------------------------------------------------------------------
-- 2. Hoteles, contactos y puntos de interés
-- ----------------------------------------------------------------------
CREATE TABLE hotel (
    hotel_id      INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre        VARCHAR(120)  NOT NULL UNIQUE,
    categoria     SMALLINT      NOT NULL CHECK (categoria BETWEEN 1 AND 5),
    descripcion   TEXT,
    domicilio_id  BIGINT        NOT NULL UNIQUE REFERENCES
domicilio(domicilio_id),
    latitud       NUMERIC(9,6)  CHECK (latitud  BETWEEN -90  AND 90),
    longitud      NUMERIC(9,6)  CHECK (longitud BETWEEN -180 AND 180),
    hora_checkin  TIME          NOT NULL DEFAULT '14:00',
    hora_checkout TIME          NOT NULL DEFAULT '10:00',
    activo        BOOLEAN       NOT NULL DEFAULT TRUE,
    fecha_alta    DATE          NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE tipo_contacto (
    tipo_contacto_id SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre           VARCHAR(30) NOT NULL UNIQUE   -- TELEFONO, EMAIL, WEB,
WHATSAPP
);

CREATE TABLE hotel_contacto (
    hotel_contacto_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    hotel_id          INTEGER      NOT NULL REFERENCES hotel(hotel_id) ON
DELETE CASCADE,
    tipo_contacto_id  SMALLINT     NOT NULL REFERENCES
tipo_contacto(tipo_contacto_id),
    valor             VARCHAR(150) NOT NULL,
    principal         BOOLEAN      NOT NULL DEFAULT FALSE,
    CONSTRAINT uq_hotel_contacto UNIQUE (hotel_id, tipo_contacto_id, valor)
);
-- Un solo contacto principal por hotel y por tipo
CREATE UNIQUE INDEX uq_hotel_contacto_principal
    ON hotel_contacto (hotel_id, tipo_contacto_id) WHERE principal;

CREATE TABLE tipo_punto_interes (
    tipo_punto_interes_id SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre                VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE punto_interes (
    punto_interes_id      INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    tipo_punto_interes_id SMALLINT     NOT NULL REFERENCES
```

```
tipo_punto_interes(tipo_punto_interes_id),
    localidad_id          INTEGER      NOT NULL REFERENCES
localidad(localidad_id),
    nombre                VARCHAR(120) NOT NULL,
    descripcion           TEXT,
    latitud               NUMERIC(9,6) CHECK (latitud  BETWEEN -90  AND 90),
    longitud              NUMERIC(9,6) CHECK (longitud BETWEEN -180 AND 180),
    CONSTRAINT uq_punto_interes UNIQUE (localidad_id, nombre)
);

-- N:M hotel - punto de interés; la distancia es atributo de la relación
CREATE TABLE hotel_punto_interes (
    hotel_id         INTEGER NOT NULL REFERENCES hotel(hotel_id) ON DELETE
CASCADE,
    punto_interes_id INTEGER NOT NULL REFERENCES
punto_interes(punto_interes_id),
    distancia_m      INTEGER NOT NULL CHECK (distancia_m >= 0),
    PRIMARY KEY (hotel_id, punto_interes_id)
);

-- ---------------------------------------------------------------------
-- 3. Habitaciones, tipos, comodidades y tarifas de lista
-- ---------------------------------------------------------------------
CREATE TABLE tipo_habitacion (
    tipo_habitacion_id SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre             VARCHAR(50) NOT NULL UNIQUE,
    capacidad_maxima   SMALLINT    NOT NULL CHECK (capacidad_maxima BETWEEN 1
AND 12),
    descripcion        TEXT
);

CREATE TABLE comodidad (
    comodidad_id SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre       VARCHAR(60) NOT NULL UNIQUE
);

CREATE TABLE habitacion (
    habitacion_id      INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    hotel_id           INTEGER     NOT NULL REFERENCES hotel(hotel_id),
    tipo_habitacion_id SMALLINT    NOT NULL REFERENCES
tipo_habitacion(tipo_habitacion_id),
    numero             VARCHAR(10) NOT NULL,
    piso               SMALLINT    NOT NULL CHECK (piso BETWEEN -5 AND 200),
    activa             BOOLEAN     NOT NULL DEFAULT TRUE,
    CONSTRAINT uq_habitacion_numero UNIQUE (hotel_id, numero)
);
```

```
CREATE TABLE habitacion_comodidad (
    habitacion_id INTEGER  NOT NULL REFERENCES habitacion(habitacion_id) ON
DELETE CASCADE,
    comodidad_id  SMALLINT NOT NULL REFERENCES comodidad(comodidad_id),
    PRIMARY KEY (habitacion_id, comodidad_id)
);

-- Tarifa de lista por hotel y tipo, con vigencia (la pactada queda en la
reserva)
CREATE TABLE tarifa (
    tarifa_id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    hotel_id           INTEGER       NOT NULL REFERENCES hotel(hotel_id),
    tipo_habitacion_id SMALLINT      NOT NULL REFERENCES
tipo_habitacion(tipo_habitacion_id),
    vigencia_desde     DATE          NOT NULL,
    vigencia_hasta     DATE,
    precio_noche       NUMERIC(12,2) NOT NULL CHECK (precio_noche > 0),
    CONSTRAINT ck_tarifa_vigencia CHECK (vigencia_hasta IS NULL OR
vigencia_hasta > vigencia_desde),
    CONSTRAINT ex_tarifa_solapada EXCLUDE USING gist (
        hotel_id WITH =,
        tipo_habitacion_id WITH =,
        daterange(vigencia_desde, vigencia_hasta, '[)') WITH &&
    )
);

-- ------------------------------------------------------------------
-- 4. Huéspedes y sus medios de contacto
-- ------------------------------------------------------------------
CREATE TABLE huesped (
    huesped_id       BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    tipo_documento   VARCHAR(10)  NOT NULL CHECK (tipo_documento IN
('DNI','PASAPORTE','CI','LE','LC','OTRO')),
    numero_documento VARCHAR(20)  NOT NULL,
    nombre           VARCHAR(80)  NOT NULL,
    apellido         VARCHAR(80)  NOT NULL,
    fecha_nacimiento DATE         CHECK (fecha_nacimiento > DATE '1900-01-
01'),
    nacionalidad_id  SMALLINT     REFERENCES pais(pais_id),
    domicilio_id     BIGINT       UNIQUE REFERENCES domicilio(domicilio_id),
    fecha_registro   TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CONSTRAINT uq_huesped_documento UNIQUE (tipo_documento, numero_documento)
);

CREATE TABLE huesped_telefono (
```

```
    huesped_telefono_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    huesped_id BIGINT      NOT NULL REFERENCES huesped(huesped_id) ON DELETE
CASCADE,
    tipo       VARCHAR(10) NOT NULL DEFAULT 'MOVIL' CHECK (tipo IN
('MOVIL','FIJO','TRABAJO')),
    numero     VARCHAR(25) NOT NULL CHECK (numero ~ '^\+?[0-9 ()-]{6,24}$'),
    principal  BOOLEAN     NOT NULL DEFAULT FALSE,
    CONSTRAINT uq_huesped_telefono UNIQUE (huesped_id, numero)
);
CREATE UNIQUE INDEX uq_huesped_telefono_principal
    ON huesped_telefono (huesped_id) WHERE principal;

CREATE TABLE huesped_email (
    huesped_email_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    huesped_id BIGINT       NOT NULL REFERENCES huesped(huesped_id) ON DELETE
CASCADE,
    email      VARCHAR(254) NOT NULL CHECK (email ~* '^[^@\s]+@[^@\s]+\.
[^@\s]+$'),
    principal  BOOLEAN      NOT NULL DEFAULT FALSE,
    CONSTRAINT uq_huesped_email UNIQUE (huesped_id, email)
);
CREATE UNIQUE INDEX uq_huesped_email_principal
    ON huesped_email (huesped_id) WHERE principal;

-- ---------------------------------------------------------------------
-- 5. Reservas
-- ---------------------------------------------------------------------
-- Estados: P Pendiente, C Confirmada, F Finalizada, X Cancelada, N No-show
CREATE TABLE estado_reserva (
    estado      CHAR(1)     PRIMARY KEY CHECK (estado IN
('P','C','F','X','N')),
    descripcion VARCHAR(30) NOT NULL UNIQUE,
    ocupa       BOOLEAN     NOT NULL      -- ¿bloquea la habitación?
);

CREATE TABLE reserva (
    reserva_id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    habitacion_id      INTEGER       NOT NULL REFERENCES
habitacion(habitacion_id),
    huesped_id         BIGINT        NOT NULL REFERENCES huesped(huesped_id),
    fecha_inicio       DATE          NOT NULL,   -- noche de llegada
    fecha_fin          DATE          NOT NULL,   -- día de salida (no se
cobra)
    cantidad_huespedes SMALLINT      NOT NULL DEFAULT 1 CHECK
(cantidad_huespedes BETWEEN 1 AND 12),
    estado             CHAR(1)       NOT NULL DEFAULT 'P' REFERENCES
```

```
estado_reserva(estado),
    tarifa_noche       NUMERIC(12,2) CHECK (tarifa_noche > 0),
    fecha_creacion     TIMESTAMPTZ   NOT NULL DEFAULT now(),
    fecha_confirmacion TIMESTAMPTZ,
    observaciones      TEXT,
    CONSTRAINT ck_reserva_fechas   CHECK (fecha_fin > fecha_inicio),
    CONSTRAINT ck_reserva_duracion CHECK (fecha_fin - fecha_inicio <= 90),
    -- La tarifa existe sólo si la reserva fue confirmada (C o F)
    CONSTRAINT ck_reserva_tarifa CHECK (
           (estado = 'P'        AND tarifa_noche IS NULL     AND
fecha_confirmacion IS NULL)
        OR (estado IN ('C','F') AND tarifa_noche IS NOT NULL AND
fecha_confirmacion IS NOT NULL)
        OR (estado IN ('X','N'))
    ),
    -- Una habitación no puede tener dos reservas activas superpuestas.
    -- Intervalo [inicio, fin): la salida de una puede ser la llegada de
otra.
    CONSTRAINT ex_reserva_superpuesta EXCLUDE USING gist (
        habitacion_id WITH =,
        daterange(fecha_inicio, fecha_fin, '[)') WITH &&
    ) WHERE (estado IN ('P','C','F'))
);

-- ---------------------------------------------------------------------
-- 6. Regla entre tablas: cantidad de huéspedes <= capacidad del tipo
-- ---------------------------------------------------------------------
CREATE FUNCTION fn_valida_capacidad() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
    v_cap SMALLINT;
BEGIN
    SELECT th.capacidad_maxima INTO v_cap
      FROM habitacion h
      JOIN tipo_habitacion th USING (tipo_habitacion_id)
     WHERE h.habitacion_id = NEW.habitacion_id;

    IF NEW.cantidad_huespedes > v_cap THEN
        RAISE EXCEPTION 'La habitación % admite como máximo % huéspedes (se
pidieron %)',
              NEW.habitacion_id, v_cap, NEW.cantidad_huespedes
              USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
END $$;
```

```
CREATE TRIGGER trg_reserva_capacidad
    BEFORE INSERT OR UPDATE OF habitacion_id, cantidad_huespedes ON reserva
    FOR EACH ROW EXECUTE FUNCTION fn_valida_capacidad();

-- ---------------------------------------------------------------------
-- 7. Índices sobre claves foráneas (PostgreSQL no los crea solo)
-- ---------------------------------------------------------------------
CREATE INDEX ix_provincia_pais          ON provincia (pais_id);
CREATE INDEX ix_domicilio_localidad     ON domicilio (localidad_id);
CREATE INDEX ix_hotel_contacto_tipo     ON hotel_contacto (tipo_contacto_id);
CREATE INDEX ix_punto_interes_localidad ON punto_interes (localidad_id);
CREATE INDEX ix_hpi_punto               ON hotel_punto_interes
(punto_interes_id);
CREATE INDEX ix_habitacion_tipo         ON habitacion (tipo_habitacion_id);
CREATE INDEX ix_hab_comodidad_comodidad ON habitacion_comodidad
(comodidad_id);
CREATE INDEX ix_huesped_telefono_h      ON huesped_telefono (huesped_id);
CREATE INDEX ix_huesped_email_h         ON huesped_email (huesped_id);
CREATE INDEX ix_reserva_huesped         ON reserva (huesped_id);
-- reserva.habitacion_id queda cubierta por el GiST de la restricción
EXCLUDE;
-- habitacion.hotel_id por uq_habitacion_numero (prefijo izquierdo).

-- ---------------------------------------------------------------------
-- 8. Catálogos fijos
-- ---------------------------------------------------------------------
INSERT INTO estado_reserva VALUES
    ('P','Pendiente',  TRUE),
    ('C','Confirmada', TRUE),
    ('F','Finalizada', TRUE),
    ('X','Cancelada',  FALSE),
    ('N','No-show',    FALSE);

INSERT INTO tipo_contacto (nombre) VALUES
    ('TELEFONO'),('EMAIL'),('WEB'),('WHATSAPP');
```

Las reglas que garantiza este script se prueban en el Punto 4 y su costo de escritura se mide en el Punto 8. En la Parte VII se agrega la columna `hotel_id` a `reserva` (desnormalización controlada del Punto 5) y en el Punto 11 los índices de la estrategia general.

7. Selección de tipos de datos

Cada tipo se eligió por el rango que necesita el dato, no "por las dudas": en una tabla de 6 millones de filas, cada byte de más son 6 MB en la tabla y otro tanto en cada índice que incluya la columna. Las mediciones son del script `03_tipos_datos.sql`.

Mediciones

Tipo	Bytes por valor	Observación medida
`SMALLINT`	2	Hasta 32.767
`INTEGER`	4	Hasta 2.147.483.647; `2147483647 + 1` da *integer out of range*
`BIGINT`	8	La misma suma da 2147483648 sin error
`BOOLEAN`	1	
`DATE`	4	`'2027-03-15' - '2027-03-10'` = 5 (noches, entero)
`TIMESTAMP` / `TIMESTAMPTZ`	8	La misma resta con horas da *4 days 20:00:00*
`NUMERIC(12,2)` con 85000.50	12	Sumar 0,10 un millón de veces da 100000.00 exacto
`DOUBLE PRECISION`	8	La misma suma da 100000.00000133288
`REAL`	4	La misma suma da 100958.34 (error de casi 1 %)
`CHAR(10)` con 'DNI'	14	Se rellena con 7 espacios
`VARCHAR(10)` / `TEXT` con 'DNI'	7	1 byte de cabecera + 3 de contenido + alineación

Justificación por familia

INT / BIGINT / SMALLINT. `reserva_id` y `huesped_id` son `BIGINT`: la tabla de reservas es la que crece sin límite (6 millones en el experimento; una cadena real acumula décadas) y un `INTEGER` agotado obliga a una migración bloqueante. `habitacion_id`, `hotel_id` y `localidad_id` son `INTEGER`: ninguna cadena tiene 2 mil millones de habitaciones. Los catálogos (`pais_id`, `tipo_habitacion_id`, `comodidad_id`) y valores chicos (`categoria`,

`piso`, `cantidad_huespedes`) son `SMALLINT`. Como `habitacion_id` está en cada reserva y en sus índices, usar `INTEGER` en vez de `BIGINT` ahorra unos 24 MB de datos sólo en esa columna.

CHAR / VARCHAR / TEXT. En PostgreSQL los tres usan el mismo almacenamiento de longitud variable; `CHAR(n)` sólo agrega relleno con espacios, que ocupa espacio y se descarta al compararlo o concatenarlo. Por eso se usa `CHAR` únicamente donde la longitud es fija por definición: `codigo_iso CHAR(2)` y `estado CHAR(1)`. `VARCHAR(n)` se usa cuando el límite es una regla de negocio (email hasta 254 caracteres según el estándar, documento hasta 20). `TEXT` se usa para texto libre sin límite natural (`descripcion`, `observaciones`).

DATE / TIMESTAMP / TIMESTAMPTZ. `fecha_inicio` y `fecha_fin` son `DATE` porque una reserva se pacta por noches, no por horas: ocupa 4 bytes en lugar de 8, la resta da directamente la cantidad de noches y los rangos `daterange` funcionan sin problemas de horario. La hora de ingreso es una política del hotel (`hora_checkin TIME`), no de cada reserva. `fecha_creacion` y `fecha_confirmacion` registran un **instante** y son `TIMESTAMPTZ`: el mismo valor se mostró como 14:00 en Jujuy y 18:00 en Madrid, porque se guarda en UTC. Un `TIMESTAMP` sin zona sería ambiguo en una cadena con hoteles en distintos países.

DECIMAL / FLOAT / DOUBLE. Todo importe es `NUMERIC(12,2)`. La medición muestra que `REAL` y `DOUBLE PRECISION` acumulan error de redondeo al sumar; en facturación eso es inaceptable. 12 dígitos con 2 decimales admiten hasta 9.999.999.999,99, suficiente para pesos argentinos con inflación. Las coordenadas usan `NUMERIC(9,6)`: 6 decimales dan precisión de unos 11 cm y no se hacen cálculos acumulativos sobre ellas.

BOOLEAN. `activo`, `activa`, `principal`, `ocupa`: 1 byte, sólo admite verdadero o falso y permite índices parciales legibles (`WHERE principal`). Se evitó `estado = 'S'/'N'` en texto, que admitiría valores inválidos.

Ancho de fila de `reserva`

Una reserva confirmada con los tipos elegidos ocupa **96 bytes** (medido con `pg_column_size`, incluye los 24 bytes de cabecera de tupla). La misma fila con `BIGINT` en todas las claves y en `cantidad_huespedes`, `TIMESTAMP` en las fechas de estadía y `CHAR(10)` en el estado ocupa **120 bytes**: un 25 % más. En 6 millones de filas son unos 140 MB adicionales sólo en la tabla, más el crecimiento de cada índice y de la memoria necesaria para mantenerla en caché.

8. Claves y restricciones físicas

Las restricciones tienen un costo de escritura medible: insertar 500.000 reservas tardó 1,4 s sin restricciones y 25,7 s con todas, y casi todo ese aumento lo explica la restricción `EXCLUDE`. Se aceptó el costo porque esa regla no puede garantizarse de otro modo sin condiciones de carrera (Punto 4).

Qué se implementó

Mecanismo	Cantidad	Ejemplos
`PRIMARY KEY`	20	Subrogadas con `GENERATED ALWAYS AS IDENTITY`; compuestas en tablas N:M
`FOREIGN KEY`	23	Todas con `ON DELETE` restrictivo salvo contactos, comodidades y puntos de interés de un hotel (`CASCADE`: no tienen sentido sin su dueño)
`UNIQUE`	18	Claves naturales, `(hotel_id, numero)`, 1:1 de domicilio
`UNIQUE` parcial	3	Un contacto principal por tipo
`CHECK`	22	Fechas, tarifa según estado, rangos, formatos
`EXCLUDE`	2	Reservas superpuestas, vigencias de tarifas
Trigger	1	Capacidad de la habitación
`NOT NULL`	En toda columna obligatoria	`tarifa_noche` y `fecha_confirmacion` son nulables a propósito

`GENERATED ALWAYS AS IDENTITY` se prefirió a `SERIAL` porque es SQL estándar e impide insertar un id a mano por error (para hacerlo hay que escribir `OVERRIDING SYSTEM VALUE`), lo que evita desincronizar la secuencia.

Impacto medido en la escritura

Script `04_impacto_restricciones.sql`: las mismas 500.000 reservas (1.000 habitaciones × 500 estadías de 3 noches, sin superposiciones) se insertan en un solo `INSERT … SELECT` sobre cuatro variantes de la tabla.

Variante	Tiempo	Índices creados	Tamaño de índices
A) Sin restricciones	1,4 s	ninguno	0 MB
B) PK + FK + CHECK	5,3 s	PK (B-Tree)	11 MB
D) B + `EXCLUDE` (sin trigger)	22,8 s	PK + GiST + FK huésped	46 MB
C) Completa: D + trigger de capacidad	25,7 s	PK + GiST + FK huésped	46 MB

La tabla ocupa 40 MB en las cuatro variantes; lo que cambia es el costo de mantener índices y verificar reglas.

Lectura de los resultados:

- **PK y FK** cuadruplican el tiempo: cada fila actualiza el B-Tree de la PK y busca su habitación y su huésped en los índices de las tablas padre.
- **`EXCLUDE`** es el más caro (~17 s de los 25,7): por cada fila el motor busca en el índice GiST si algún rango de la misma habitación se superpone y luego inserta la entrada. GiST es más lento que B-Tree para insertar porque sus nodos admiten solapamiento y a veces hay que recorrer varias ramas.
- **El trigger** agrega ~3 s: una consulta `habitacion JOIN tipo_habitacion` por fila.

Consecuencias de diseño

- Para la **carga masiva** de 6 millones (Parte IV) conviene cargar con las restricciones y luego crear los índices secundarios, o cargar habitación por habitación de forma ordenada. Lo que **no** se hará es desactivar el `EXCLUDE` para ganar velocidad: los datos de prueba podrían quedar con superposiciones y todas las mediciones posteriores serían sobre datos inválidos.
- En operación normal (una reserva por vez) el costo es de milisegundos por fila y no es un problema.
- El índice GiST de la restricción también sirve para **consultar** disponibilidad (Punto 26): no es un costo sin retorno.

9. Motor y estructuras de almacenamiento

PostgreSQL tiene un único motor de almacenamiento: guarda cada tabla como un **heap** (montículo de filas sin orden) dividido en páginas de 8 KB, y cada índice como una estructura aparte que apunta a las filas por su posición física. Es la diferencia más

importante con InnoDB, donde la tabla *es* el índice de la clave primaria.

Organización física

- **Archivos.** Cada tabla e índice es un archivo en el directorio de datos (`reserva` está en `base/16384/19406`), partido en segmentos de 1 GB. Junto al archivo principal hay dos auxiliares: el *free space map* (32 kB, qué páginas tienen lugar libre) y el *visibility map* (8 kB, qué páginas tienen sólo filas visibles para todos; lo usan `VACUUM` y los *index-only scans*).
- **Páginas.** Bloques de 8.192 bytes (`block_size = 8192`): una cabecera, un arreglo de punteros a las filas y las filas mismas. Las 500.000 reservas de prueba ocupan 5.155 páginas, unas 97 filas por página.
- **Tuplas e identificador físico.** Cada fila se ubica por su `ctid` = (página, posición). Los índices guardan ese `ctid` , no la clave primaria.
- **TOAST.** Valores de más de unos 2 kB (por ejemplo una `observaciones` muy larga) se comprimen o se mueven a una tabla auxiliar, para que la fila base siga entrando en la página.

MVCC: un UPDATE escribe una fila nueva

PostgreSQL implementa concurrencia multiversión guardando las versiones dentro de la misma tabla. Cada tupla tiene `xmin` (transacción que la creó) y `xmax` (la que la borró o reemplazó). Medido sobre la reserva 1000:

Momento	`ctid`	`xmin`
Antes del `UPDATE`	(10,30)	994
Después del `UPDATE`	(5154,65)	1010

La fila "se movió" a la última página: el `UPDATE` escribió una versión nueva y la anterior quedó como **tupla muerta** hasta que `VACUUM` recupere su espacio. Consecuencias: los `UPDATE` masivos hacen crecer la tabla, todos los índices deben apuntar a la nueva versión (salvo optimización HOT, cuando no cambia una columna indexada y hay lugar en la misma página) y el mantenimiento con `VACUUM` es obligatorio (Punto 48). La ventaja: los lectores nunca bloquean a los escritores ni al revés.

Transacciones y durabilidad

Toda modificación se escribe primero en el **WAL** (*write-ahead log*) y recién después, de forma diferida, en las páginas de datos. Al hacer `COMMIT` sólo es necesario que el WAL esté en disco (`fsync = on` en el entorno); si el servidor se cae, al arrancar reproduce el

WAL. Las páginas modificadas viven en `shared_buffers` y el proceso *checkpointer* las baja a disco periódicamente.

Comparación con InnoDB (MySQL / MariaDB)

Aspecto	PostgreSQL	InnoDB
Organización de la tabla	Heap sin orden	Índice agrupado (B+Tree) por la PK; las filas están en las hojas
Índices secundarios apuntan a	`ctid` (posición física)	El valor de la PK; se busca dos veces
Tamaño de página	8 KB	16 KB
Versiones anteriores (MVCC)	En la misma tabla (tuplas muertas)	En el *undo log*, aparte
Limpieza	`VACUUM` / autovacuum	*Purge thread*
Registro de recuperación	WAL	*Redo log* + *doublewrite buffer*
Bloqueos	A nivel fila, guardados en la propia tupla	A nivel fila sobre registros del índice, con *gap locks* y *next-key locks*
Caché	`shared_buffers` + caché del sistema operativo	*Buffer pool* (normalmente 50–75 % de la RAM, sin depender del SO)
Índices disponibles	B-Tree, Hash, GiST, SP-GiST, GIN, BRIN	B+Tree, full-text, espacial (R-Tree)

Implicancias para este TP: en InnoDB la elección de la PK define el orden físico de las reservas, y una PK aleatoria fragmenta la tabla; en PostgreSQL el orden físico es el de inserción y se puede reordenar una vez con `CLUSTER`. En InnoDB un índice secundario "lleva" la PK adentro, en PostgreSQL no. Y la restricción de no superposición, que aquí es un `EXCLUDE`, en InnoDB requeriría un trigger con `SELECT … FOR UPDATE` y apoyarse en los *gap locks*.

10. Estrategias de almacenamiento

Con 6 millones de reservas, la tabla y sus índices (~1 GB proyectado) superan los 128 MB de `shared_buffers` del entorno; por eso el rendimiento dependerá de cuántas páginas haya que leer y de si están en la caché del sistema operativo o en disco.

Entorno de experimentación

Componente	Valor
Máquina	Virtual (KVM), Linux 6.18, Ubuntu 24.04.5 LTS
CPU	Intel Xeon @ 2,10 GHz, 2 núcleos, 1 hilo por núcleo
RAM	7,8 GB, sin swap
Disco	Virtual (`vda` , 256 GB), sistema de archivos ext4
Escritura secuencial (1 GB, sin caché)	929 MB/s
Lectura secuencial (1 GB, sin caché)	1,8 GB/s
Lectura en bloques de 8 KB, sin caché	195 MB/s (~24.000 lecturas/s)
Sincronizaciones a disco (`pg_test_fsync`)	~3.500 por segundo (283 µs cada una)
Motor	PostgreSQL 16.15, configuración por defecto
`shared_buffers`	128 MB
`work_mem`	4 MB
`maintenance_work_mem`	64 MB
`effective_cache_size`	4 GB
`random_page_cost`	4 (valor pensado para discos mecánicos)
`max_connections`	100

> Los integrantes del grupo deben completar una columna con los mismos datos de su equipo al repetir los scripts (`lscpu` , `free -h` , `SHOW shared_buffers;` , etc.). Las conclusiones se comparan por **proporciones** entre variantes, no por tiempos absolutos.

Tamaño y crecimiento medidos

Con 500.000 reservas (script 04): tabla 40 MB, índices 46 MB (PK 11 MB, GiST de la restricción 33 MB, FK huésped 3 MB). Los índices ya pesan más que la tabla. Proyección lineal a 6 millones:

Objeto	500 mil	6 millones (proyectado)
Tabla `reserva`	40 MB	~480 MB
Índices actuales	46 MB	~550 MB
Total	87 MB	~1 GB

Medido después de la carga real (Punto 20): con 7,5 millones de reservas la tabla ocupó 719 MB y sus índices 663 MB, en línea con la proyección.

Cómo influye el almacenamiento en el rendimiento

- **El costo de una consulta es, ante todo, páginas leídas.** Un recorrido completo de `reserva` con 6 millones de filas lee unas 60.000 páginas; una búsqueda por índice B-Tree lee 3 o 4. El optimizador estima ese costo con `seq_page_cost` (1) y `random_page_cost` (4).
- **Lectura secuencial frente a aleatoria.** El disco del entorno es unas 9 veces más rápido leyendo en secuencia (1,8 GB/s) que en bloques sueltos (195 MB/s). Por eso, cuando una consulta devuelve muchas filas, un recorrido completo puede ganarle a un índice: el índice obliga a saltar de página en página (Punto 32).
- **Dos niveles de caché.** PostgreSQL lee a través de su propio `shared_buffers` y, debajo, de la caché de páginas del sistema operativo. Con 7,8 GB de RAM, la tabla de 6 millones puede quedar entera en la caché del SO aunque no entre en `shared_buffers`: la primera ejecución de una consulta (*cold cache*) puede ser mucho más lenta que las siguientes (Punto 23).
- **Ancho de fila.** A filas más angostas, más filas por página y menos I/O. La elección de tipos (Punto 7) reduce un 20 % el tamaño frente a la alternativa "todo BIGINT/TIMESTAMP".
- **Crecimiento por MVCC.** Cada `UPDATE` deja una tupla muerta (Punto 9). Una carga que confirma millones de reservas en un `UPDATE` duplica temporalmente la tabla hasta que `VACUUM` la limpia. Conviene cargar las reservas ya con su estado final.
- **Escritura.** Cada `COMMIT` espera una sincronización del WAL (~283 µs en este disco). Insertar fila por fila en transacciones separadas limita a unas 3.500 filas por segundo; agrupar muchas filas por transacción o usar `COPY` elimina ese techo. Es la base de la estrategia de carga masiva (Punto 20).

Estrategias adoptadas

1. Tipos de datos ajustados al rango real (Punto 7).

2. Carga masiva con `COPY` o `INSERT … SELECT` en transacciones grandes, con el estado final de cada reserva para no generar tuplas muertas.
3. `ANALYZE` y `VACUUM` después de cada carga para actualizar estadísticas y el *visibility map*.
4. Medición de tamaño con `pg_relation_size` / `pg_indexes_size` en cada experimento, para relacionar tiempos con páginas leídas.
5. Evaluación de particionamiento por fecha cuando la tabla crezca (Parte X) y de índices BRIN para fechas (Punto 13), que ocupan una fracción de un B-Tree.

Parte III — Índices y Estructuras de Acceso

11. Estrategia general de índices

La estrategia parte de los índices que exigen las restricciones (PK, UNIQUE, EXCLUDE), agrega uno por cada clave foránea consultada y sólo cuatro secundarios más, cada uno atado a una consulta concreta. Sobre `reserva`, la tabla de 7,5 millones, hay apenas cuatro índices; los demás que se prueban en esta parte se crean y se eliminan dentro de cada experimento.

Índices de la base (`01_schema.sql` + `21_estrategia_indices.sql`)

Tabla	Índice	Tipo	Columnas	Consultas objetivo	Tamaño
reserva	`reserva_pkey`	PK, B-Tree	reserva_id	Acceso por número de reserva, paginación por clave	161 MB
reserva	`ex_reserva_superpuesta`	EXCLUDE, GiST	habitacion_id, daterange(inicio, fin)	Garantiza RN1; consultas de disponibilidad (Punto 26) y la FK `habitacion_id`	429 MB
reserva	`ix_reserva_huesped`	FK, B-Tree	huesped_id	Historial del huésped; verificar la FK al borrar un huésped	72 MB
reserva	`ix_reserva_fecha_inicio`	Simple, B-Tree	fecha_inicio	Reservas por día, mes, año; ocupación; reportes	52 MB
huesped	`uq_huesped_documento`	UNIQUE	tipo_documento, numero_documento	Búsqueda por documento en el check-in (RN5)	52 MB
huesped	`ix_huesped_apellido_nombre`	Compuesto	apellido, nombre	Búsqueda por nombre en recepción, listados ordenados	7 MB

Tabla	Índice	Tipo	Columnas	Consultas objetivo	Tamaño
huesped	`ix_huesped_fecha_registro`	Simple	fecha_registro	Altas recientes (consulta 24.3)	21 MB
huesped_email	`ix_huesped_email_lower`	Funcional	lower(email)	Búsqueda por email sin distinguir mayúsculas	50 MB
huesped_email / _telefono	`uq_..._principal`	UNIQUE parcial	huesped_id WHERE principal	Garantiza RN6	20 MB c/u
habitacion	`uq_habitacion_numero`	UNIQUE compuesto	hotel_id, numero	RN7; habitaciones de un hotel (también sirve a la FK `hotel_id` por prefijo izquierdo)	184 kB
tarifa	`ex_tarifa_solapada`	EXCLUDE, GiST	hotel_id, tipo, daterange(vigencia)	RN8; buscar la tarifa vigente	536 kB
Resto de las FK	`ix_<tabla>_<columna>`	Simple	columna FK	`JOIN` con la tabla padre y verificación al borrar el padre	< 7 MB

Criterios aplicados

- **Toda PK y todo UNIQUE generan un índice** automáticamente: no se duplican.
- **PostgreSQL no indexa las FK automáticamente.** Sin índice, borrar un huésped obliga a recorrer los 7,5 millones de reservas para verificar que no tenga ninguna. Por eso cada FK consultada tiene el suyo, salvo cuando otro índice ya la empieza (prefijo izquierdo).
- **Columna más selectiva o más consultada primero** en los compuestos (Punto 14).
- **No se indexan columnas de baja selectividad solas** (`estado`, `cantidad_huespedes`): Punto 17.
- **Los índices de optimización** (cobertura, funcional sobre apellido, `(huesped_id, fecha_inicio)`, `(hotel_id, fecha_inicio)`) se agregan en la Parte VII sólo si la medición lo justifica.

Redundancias detectadas

Al listar los tamaños aparecieron dos índices redundantes del esquema original: `ix_huesped_email_h (huesped_id)` y `ix_huesped_telefono_h (huesped_id)`. Ya existen `uq_huesped_email (huesped_id, email)` y `uq_huesped_telefono (huesped_id, numero)`, que empiezan por `huesped_id` y resuelven las mismas búsquedas. Eliminarlos ahorraría 81 MB y una actualización de índice por cada alta de contacto. Se dejan en el script para que la comparación sea visible, y se recomienda eliminarlos.

Otro hallazgo: `ix_huesped_apellido_nombre` ocupa sólo 7 MB para un millón de filas, contra 21 MB de `ix_huesped_fecha_registro`. Desde PostgreSQL 13 el B-Tree **deduplica** claves repetidas: con 50 apellidos × 40 nombres hay 2.000 combinaciones distintas, y cada una se guarda una vez con su lista de filas.

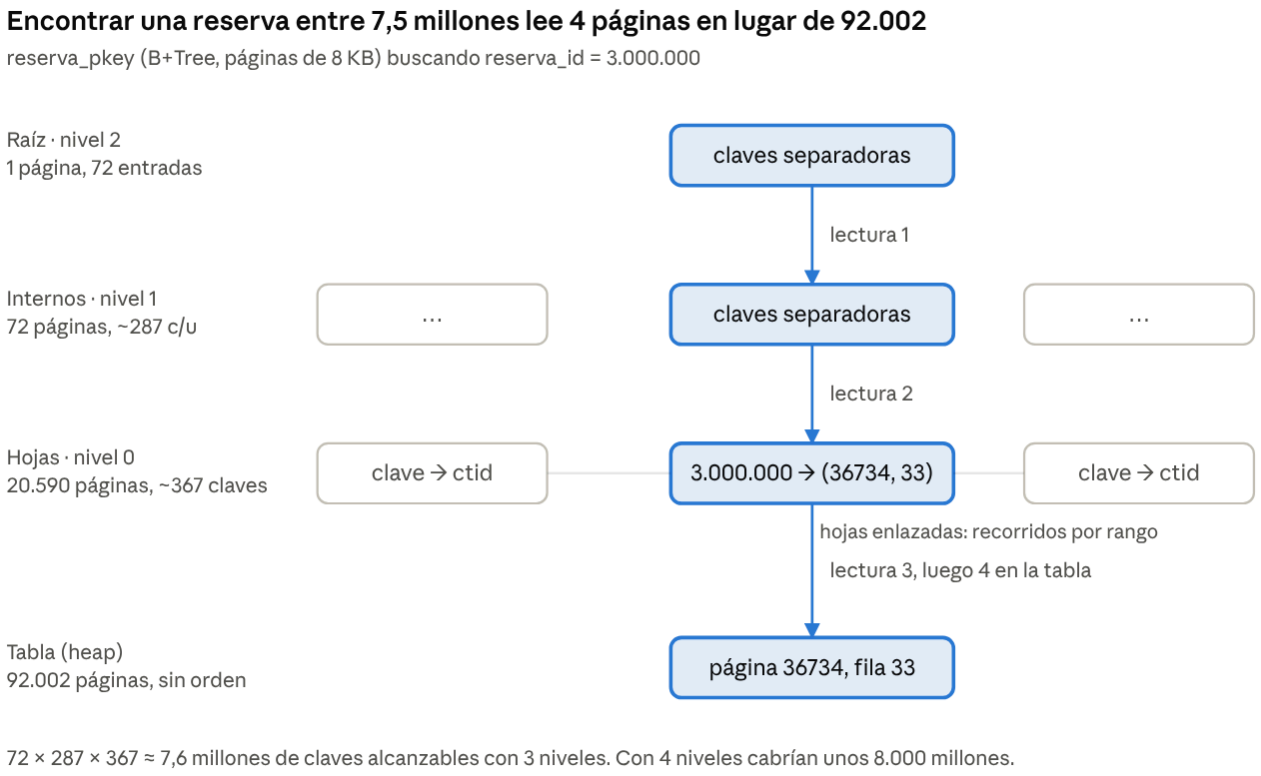
12. Árboles B-Tree y B+Tree

El índice de la PK de `reserva` tiene **3 niveles** para 7,5 millones de claves: encontrar una reserva lee 4 páginas (3 del índice y 1 de la tabla) y tarda 0,26 ms. Sin el índice, el motor lee las 92.002 páginas de la tabla y tarda 397 ms: 1.500 veces más.

Funcionamiento

Un **B-Tree** es un árbol balanceado cuyos nodos tienen muchas claves ordenadas y muchos hijos, de modo que todas las hojas quedan a la misma profundidad. Un **B+Tree** es la variante que usan los motores relacionales: las claves completas y los punteros a las filas están sólo en las hojas, los nodos internos guardan claves separadoras para elegir el camino, y las hojas están enlazadas en orden para recorrer rangos sin volver a subir. El B-Tree de PostgreSQL es en realidad un B+Tree (algoritmo de Lehman y Yao, con enlaces también entre nodos del mismo nivel para permitir concurrencia).

- **Nodo = página.** Cada nodo ocupa una página de 8 KB, la unidad que el motor lee del disco. Lo que importa no es cuántas comparaciones hace la CPU, sino cuántas páginas lee.
- **Factor de ramificación.** Una hoja de `reserva_pkey` guarda 367 entradas de 16 bytes (clave + `ctid`); la raíz, 72. Como cada nodo tiene cientos de hijos, la altura crece con el logaritmo en base ~300 de la cantidad de filas.
- **Altura medida.** `bt_metap` informa nivel de raíz 2, es decir 3 niveles. El índice ocupa 20.663 páginas (161 MB); la tabla, 92.002 (719 MB).
- **Inserción y división.** Cuando una hoja se llena se divide en dos y el padre recibe una nueva clave separadora; si el padre se llena también se divide, y si se divide la raíz el árbol crece un nivel. Por eso el árbol queda siempre balanceado. Una PK creciente (identidad) siempre inserta en la hoja más a la derecha, y PostgreSQL optimiza ese caso dejando las hojas casi llenas (en la hoja medida quedaron 808 bytes libres de 8.192).



20_indices.py · bt_metap y bt_page_stats sobre reserva_pkey

Búsqueda secuencial frente a búsqueda por índice

Estrategia	Páginas leídas	Tiempo (mediana)	Costo teórico
Index Scan sobre `reserva_pkey`	4	0,26 ms	$O(\log_n N) + 1$ = altura + 1 lectura de la tabla
Seq Scan (índice desactivado)	92.002	397 ms	O(N/filas por página) = todas las páginas

El recorrido secuencial tiene a su favor que lee páginas contiguas (el disco lo hace 9 veces más rápido que leerlas salteadas, Punto 10). Por eso, cuando una consulta necesita una fracción grande de la tabla, recorrerla entera puede ser más barato que ir y volver entre índice y tabla: el Punto 32 mide dónde está ese punto de equilibrio.

Costo de búsqueda en función del volumen

Con un factor de ramificación de ~300, la altura crece muy lento: 3 niveles cubren hasta unos 27 millones de claves y 4 niveles más de 8.000 millones. Cuadruplicar las reservas no agregaría ni una lectura a la búsqueda por PK; en cambio el recorrido secuencial crece en proporción directa al tamaño de la tabla.

13. Otros tipos de índices

PostgreSQL ofrece seis métodos de acceso además del B-Tree en sus variantes. El hallazgo más útil para este dominio es BRIN: un índice de **40 kB** respondió una consulta por rango de fechas de creación en 2,8 ms, igual que un B-Tree de **138 MB**, porque las reservas se guardan en el orden en que se crean.

Resultados medidos

Índice	Columna y consulta	Tamaño	Sin índice	Con índice
BRIN	`fecha_creacion`, una semana	40 kB	506 ms (Seq Scan)	2,8 ms
B-Tree (comparación)	`fecha_creacion`, una semana	138 MB	506 ms	3,0 ms
BRIN sin correlación	`habitacion_id = 500`	—	404 ms	el optimizador lo descarta y recorre la tabla
Hash	`email = '…'`	32 MB (B-Tree: 50 MB)	—	0,09 ms
Hash con rango	`email > 'z'`	—	—	no aplicable: Seq Scan
GIN + pg_trgm	`apellido LIKE '%rrey%'`	13 MB	98 ms	15 ms
Parcial B-Tree	`(fecha_inicio) WHERE estado = 'P'`	1 MB (completo: 51 MB)	—	15,6 ms para 22.975 filas
GiST (de la restricción)	superposición de rangos para una habitación	429 MB	—	0,26 ms

Características, ventajas y limitaciones

Tipo	Cómo funciona	Ventajas	Limitaciones	Uso en este sistema
BRIN (*Block Range Index*)	Guarda el mínimo y el máximo de cada grupo de 128 páginas	Diminuto; casi no cuesta mantenerlo	Sólo sirve si el orden físico sigue a la columna (`correlation` cercana a 1). Devuelve bloques candidatos que hay que releer	`fecha_creacion` y `reserva_id` (correlación 1,000 medida); históricos de solo agregado
Hash	Tabla de hash de la clave; desde PG 10 registrado en WAL	Más chico que un B-Tree para claves largas; búsqueda O(1)	Sólo igualdad: no rangos, no orden, no `LIKE`; no sirve para `UNIQUE`	Búsquedas exactas por email o token. En la práctica el B-Tree casi siempre alcanza
GIN (*Generalized Inverted Index*)	Índice invertido: cada elemento (palabra, trigrama, clave JSON) apunta a la lista de filas que lo contienen	`LIKE '%x%'`, búsqueda de texto, arreglos, `jsonb`	Inserciones lentas (se mitiga con *fast update*), tamaño mayor	Buscador de huéspedes por fragmento de apellido
GiST (*Generalized Search Tree*)	Árbol balanceado de "cajas" que pueden solaparse	Rangos, geometría, cercanía (`<->`), restricciones `EXCLUDE`	Más lento que B-Tree para igualdad simple; inserciones más caras (Punto 8)	Reservas superpuestas, tarifas, y distancia hotel–punto de interés con PostGIS
SP-GiST	Árbol particionado sin solapamiento (quadtree, k-d tree, trie)	Datos con particiones naturales: puntos, prefijos de texto, IPs	Pocos operadores	Coordenadas de hoteles si se buscaran por zona

Tipo	Cómo funciona	Ventajas	Limitaciones	Uso en este sistema
Parcial	Cualquier tipo con `WHERE`: indexa sólo un subconjunto	Mucho más chico (1 MB vs 51 MB) y más rápido de mantener	La consulta debe incluir la condición para que se use	Reservas pendientes, contacto principal (RN6)
Funcional	Indexa el resultado de una expresión	Hace SARGables consultas con funciones	La consulta debe usar la misma expresión (Punto 16)	`lower(email)`, `lower(apellido)`
De cobertura (`INCLUDE`)	B-Tree con columnas extra sólo en las hojas	Index Only Scan sin visitar la tabla	Más grande (Punto 15)	Reportes de facturación

Lectura de los resultados

- **BRIN depende de la correlación física.** `pg_stats.correlation` vale 1,000 para `fecha_creacion` y 0,000 para `habitacion_id`. Con `habitacion_id` cada rango de 128 páginas contiene casi todas las habitaciones, el índice no descarta nada y el optimizador prefiere el recorrido completo.
- **Hash no reemplaza al B-Tree**: ahorra 36 % de espacio en emails, pero no sirve para la consulta de rango que el B-Tree resuelve con el mismo índice.
- **GIN con trigramas** es la única forma indexada de `LIKE '%texto%'`: la búsqueda de 98 ms pasó a 15 ms.
- **El GiST de la restricción no es sólo un costo**: también resuelve en 0,26 ms la pregunta "¿se superpone algo con este período en esta habitación?".

Otros motores: MySQL/InnoDB ofrece B+Tree, índices `FULLTEXT` (invertidos, equivalentes a GIN para texto), `SPATIAL` (R-Tree) e índices hash adaptativos internos; SQL Server y Oracle agregan índices *columnstore* y de mapa de bits, útiles para análisis de grandes volúmenes.

14. Índices compuestos y regla del prefijo izquierdo

Un índice compuesto sólo sirve para **navegar** cuando la consulta filtra por su primera columna. Con el índice `(apellido, nombre, fecha_nacimiento)` sobre un millón de huéspedes, toda consulta que incluye `apellido` tarda menos de 16 ms; toda consulta que lo omite recorre la tabla entera (55 a 97 ms).

Experimento 1: índice de tres columnas en `huesped`

`CREATE INDEX ix_h_ape_nom_nac ON huesped (apellido, nombre, fecha_nacimiento)`

Caso	Filtro	¿Usa el índice?	Plan	Páginas	Tiempo
A	apellido	Sí, prefijo de 1 columna	Bitmap Index Scan	7.855	15,2 ms
B	apellido + nombre	Sí, prefijo de 2	Bitmap Index Scan	289	0,61 ms
C	apellido + nombre + nacimiento	Sí, las 3	Index Scan	3	0,16 ms
D	apellido + nacimiento (salta nombre)	Sí, navega por apellido y filtra nacimiento dentro del índice	Index Scan	68	0,65 ms
E	nombre	**No**	Seq Scan	11.479	96,8 ms
F	nombre + nacimiento	**No**	Seq Scan	11.479	64,4 ms
G	nacimiento	**No**	Seq Scan	11.479	55,3 ms
H	`ORDER BY apellido, nombre LIMIT 20`	Sí, para evitar el ordenamiento	Index Scan	23	0,51 ms
I	`ORDER BY nombre LIMIT 20`	**No**	Seq Scan + Sort	11.479	216 ms

Experimento 2: el orden de las columnas en `reserva`

Se crearon por separado `(estado, fecha_inicio)` y `(fecha_inicio, estado)` (51–52 MB cada uno) y se probaron tres consultas `count(*)`:

Consulta	`(estado, fecha_inicio)`	`(fecha_inicio, estado)`
`estado = 'N'`	717 páginas, 14,1 ms	7.068 páginas, 21,6 ms (recorre todo el índice)
`fecha_inicio` en una semana	6.554 páginas, 14,9 ms (recorre todo el índice)	13 páginas, 0,58 ms
`estado = 'X'` + un mes de fecha	8 páginas, 0,57 ms	40 páginas, 0,50 ms

Cuando el filtro no incluye la primera columna, PostgreSQL todavía puede **recorrer el índice completo** (Index Only Scan) porque es más chico que la tabla y contiene las dos columnas; pero no puede saltar a la zona buscada, y lee 500 veces más páginas que con el

orden correcto. Cuando la consulta filtra por las dos columnas, ambos órdenes funcionan.

La regla del prefijo izquierdo

Las entradas del índice están ordenadas primero por la columna 1, dentro de ella por la columna 2, y así sucesivamente, como una guía telefónica ordenada por apellido y luego nombre. Se puede buscar a todos los "Tolaba" o a "Tolaba, Rocío", pero no a todas las "Rocío": están desparramadas por toda la guía. Por eso:

1. El índice `(a, b, c)` sirve para filtros sobre `a`, `a+b` y `a+b+c`, y para `ORDER BY a`, `a, b`, `a, b, c`.
2. Para `a + c` navega por `a` y descarta en el índice las que no cumplen `c` (caso D: 68 páginas contra 7.855 del caso A).
3. Sin `a` no hay navegación: a lo sumo un recorrido completo del índice (Experimento 2) o de la tabla.
4. En el orden conviene poner primero la columna que **siempre** aparece en las consultas, y entre las que aparecen siempre, la de igualdad antes que la de rango: en `(estado, fecha_inicio)` con `estado = 'X' AND fecha entre…` el motor salta a 'X' y recorre un rango continuo (8 páginas).

PostgreSQL 16 no implementa *skip scan* (saltear la primera columna recorriendo cada uno de sus valores); se incorporó en la versión 18. MySQL lo tiene desde la 8.0.13 para casos de primera columna con pocos valores distintos.

15. Índices de cobertura

El índice de cobertura hizo que la consulta de facturación diaria de un trimestre leyera **904 páginas en lugar de 112.756** y bajara de 145 ms a 48 ms, porque ya no visita la tabla. El costo es un índice 5,6 veces más grande que el simple (291 MB contra 52 MB).

Consulta objetivo

Facturación por día del primer trimestre de 2025, sólo estadías finalizadas:

```
SELECT fecha_inicio, sum(tarifa_noche * (fecha_fin - fecha_inicio))
  FROM reserva
 WHERE fecha_inicio >= '2025-01-01' AND fecha_inicio < '2025-04-01' AND
estado = 'F'
 GROUP BY fecha_inicio ORDER BY fecha_inicio;
```

La consulta usa cuatro columnas: filtra por `fecha_inicio` y `estado`, y lee `fecha_fin` y `tarifa_noche`.

Comparación

Variante	Plan	Visitas a la tabla	Páginas	Tiempo
Índice simple `(fecha_inicio)`	Index Scan + GroupAggregate	Una por cada una de las 156.783 reservas del trimestre	112.756	144,6 ms
Índice de cobertura `(fecha_inicio) INCLUDE (estado, fecha_fin, tarifa_noche)`	**Index Only Scan** + GroupAggregate	Ninguna (`Heap Fetches: 0`)	904	48,4 ms

Por qué funciona

- Con el índice simple, el motor encuentra las reservas del trimestre en el índice, pero para conocer `estado`, `fecha_fin` y `tarifa_noche` debe ir a la tabla por cada una. Como las reservas de un mismo día están repartidas en muchas páginas, son 112.756 lecturas.
- Con `INCLUDE`, esas columnas viajan en las hojas del índice. El plan cambia a **Index Only Scan** y el motor no toca la tabla: lee 904 páginas consecutivas del índice.
- La agregación (`GroupAggregate`) aprovecha además que el índice ya entrega las filas ordenadas por fecha: no hace falta ordenar.

Condiciones y costos

- **Visibility map al día.** Index Only Scan sólo evita la tabla en las páginas marcadas como "todas visibles". Por eso se ejecutó `VACUUM` antes de medir; en una tabla con muchas modificaciones recientes, `Heap Fetches` crece y la ventaja se reduce.
- **`INCLUDE` frente a columnas clave.** Las columnas incluidas no ordenan ni sirven para filtrar al navegar; sólo se leen. Se usó `INCLUDE` en lugar de `(fecha_inicio, estado, fecha_fin, tarifa_noche)` porque hace los nodos internos más chicos y deja claro que la clave es la fecha.
- **Tamaño y escritura.** 291 MB extra que ocupan memoria caché y se actualizan con cada `INSERT` y con cada `UPDATE` de esas columnas. El efecto sobre la escritura se mide en el Punto 37.

16. Índices funcionales o basados en expresiones

PostgreSQL soporta índices sobre expresiones. La búsqueda de huéspedes sin distinguir mayúsculas pasó de **216 ms a 0,87 ms** al indexar `lower(apellido), lower(nombre)`: el índice común sobre las mismas columnas no servía porque la consulta aplica una función sobre ellas.

Experimento

```
SELECT huesped_id, nombre, apellido FROM huesped
 WHERE lower(apellido) = 'mamaní' AND lower(nombre) = 'rocío';
```

Índice disponible	Plan	Páginas	Tiempo
`(apellido, nombre)`	Seq Scan: el índice no se puede usar	11.479	216,3 ms
`(lower(apellido), lower(nombre))`	Bitmap Index Scan sobre el funcional	334	0,87 ms
Consulta **sin** `lower()` con ambos índices	Usa `(apellido, nombre)`	334	0,66 ms

Implicancias

- **La expresión debe coincidir exactamente.** El índice sobre `lower(apellido)` sólo se usa para `lower(apellido) = …`; no para `upper(apellido)`, ni para `apellido ILIKE …`.
- **La función debe ser `IMMUTABLE`**: mismo resultado para la misma entrada, siempre. `lower()` lo es. `date_trunc('month', fecha)` sobre `date` no puede indexarse directamente, porque se convierte a `timestamptz` y depende de la zona horaria de la sesión; para fechas conviene usar rangos (Punto 30).
- **Costo.** Cada `INSERT` y cada `UPDATE` de apellido o nombre calcula la expresión y actualiza el índice.
- **Estadísticas propias.** `ANALYZE` calcula estadísticas sobre la expresión indexada, con lo que el optimizador estima mejor cuántas filas devuelve `lower(apellido) = …`.

Alternativas equivalentes en otros motores

- **MySQL 8.0.13+**: índices funcionales con doble paréntesis, `CREATE INDEX ix ON huesped ((lower(apellido)))`. En versiones anteriores, una **columna generada** (`apellido_lower VARCHAR(80) AS (lower(apellido)) STORED`) con un índice común.

- **MariaDB**: no tiene índices funcionales; se usa la columna generada indexada.
- **Collation sin distinción de mayúsculas**: en MySQL las collations `_ci` comparan sin distinguir mayúsculas y un índice común alcanza. En PostgreSQL existen collations no deterministas ICU o la extensión `citext`, a cambio de perder optimizaciones como `LIKE` por prefijo.

17. Cardinalidad y selectividad

Un índice es útil cuando la condición es **selectiva** (devuelve pocas filas) y esas filas están en **pocas páginas**. El índice sobre `estado` lo mostró con claridad: `estado = 'P'` (2 % de las filas) leyó 6.407 páginas en 62 ms, mientras que `estado = 'N'` (3,8 %) leyó 84.639 páginas, casi la tabla entera, en 673 ms.

Definiciones

- **Cardinalidad** de una columna: cantidad de valores distintos (`n_distinct`). Alta en `reserva_id`, baja en `estado`.
- **Selectividad** de una condición: fracción de filas que la cumplen. Una condición es *muy selectiva* cuando esa fracción es chica. Para una igualdad sobre valores uniformes, selectividad ≈ 1 / cardinalidad.

Columnas de `reserva` (estadísticas de `pg_stats` y conteos reales)

Columna	Valores distintos	Cardinalidad	Selectividad de `col = valor`	¿Índice útil sola?
`reserva_id`	7.535.580 (única)	Máxima	1 fila	Sí (PK)
`huesped_id`	~990 mil reales (el muestreo estima 315.649)	Alta	Muy variable por el sesgo: 1 a 14.054 filas	Sí
`tarifa_noche`	32.622 estimados	Alta	Rara vez se busca por igualdad	No: se busca por rango o se agrega

Columna	Valores distintos	Cardinalidad	Selectividad de `col = valor`	¿Índice útil sola?
`habitacion_id`	7.663	Media	~980 filas por habitación	Sí, como primera columna de un compuesto (GiST de la restricción)
`fecha_inicio`	4.562	Media	~1.650 filas por día; un mes ~50.000	Sí, para rangos acotados
`cantidad_huespedes`	6	Baja	2 personas: 43,9 %	No
`estado`	5	Muy baja	F 81,2 %, X 10 %, N 3,8 %, C 3,1 %, P 2 %	Sólo para los valores raros, y mejor como índice parcial

Experimento: índice sobre `estado`

Condición	Selectividad	Plan elegido	Páginas leídas	Tiempo*
`estado = 'P'`	1,97 %	Bitmap Index Scan	6.407	62 ms
`estado = 'N'`	3,77 %	Bitmap Index Scan	84.639 de 92.002	673 ms
`estado = 'F'`	81,16 %	Seq Scan (ignora el índice)	92.002	2.360 ms

* Incluye traer todas las filas al cliente: en `F` son 6,1 millones.

Tres conclusiones:

1. **El optimizador decide por valor, no por columna.** Con el mismo índice eligió usarlo para `P` y `N` y descartarlo para `F`, porque `pg_stats.most_common_freqs` le dice que `F` es el 81 %: leer el índice y luego casi todas las páginas sería peor que recorrer la tabla una vez.
2. **La selectividad no alcanza: importa dónde están las filas.** `N` es apenas el doble de `P`, pero las reservas no-show están repartidas a lo largo de 12 años de historia (una o dos por página), mientras que las pendientes son todas futuras y se crearon recientemente: están juntas al final de la tabla. Es la *correlación física* que también decidía el éxito de BRIN.

3. **Índices de baja cardinalidad son malos candidatos.** El índice de `estado` ocupa 50 MB y sólo ayuda en el 5 % de los casos. Un índice parcial `WHERE estado = 'P'` cubre el caso útil con 1 MB (Punto 13).

Relación con la utilidad de un índice

	Selectividad alta (pocas filas)	Selectividad baja (muchas filas)
Filas juntas (correlación alta)	Índice ideal	Índice o BRIN pueden servir: leen un bloque continuo
Filas dispersas (correlación baja)	Índice útil (pocas lecturas aleatorias)	Recorrido completo: el índice causaría una lectura aleatoria por fila

18. ¿Por qué no indexar todos los campos?

Con 13 índices en lugar de 3, insertar 100.000 reservas tardó **4,6 veces más** y actualizarlas **3,6 veces más**, y los índices pasaron a ocupar 1,7 veces el tamaño de la tabla. A cambio, sólo una de las tres lecturas de prueba mejoró de forma importante, porque las otras ya estaban cubiertas por la estrategia limitada.

Escenarios (copias de 1.000.000 de reservas)

	`r_pocos` (estrategia limitada)	`r_muchos` (un índice por campo)
Índices	3: PK, `huesped_id`, `fecha_inicio`	13: PK, uno por cada una de las 10 columnas restantes y 2 compuestos
Tabla	96 MB	96 MB
Índices	45 MB	165 MB

Escritura

Operación (100.000 filas)	`r_pocos`	`r_muchos`	Relación
`INSERT`	712 ms	3.290 ms	4,6×
`UPDATE` de `tarifa_noche` (indexada en `r_muchos`)	2.293 ms	8.320 ms	3,6×

Operación (100.000 filas)	`r_pocos`	`r_muchos`	Relación
`UPDATE` de `cantidad_huespedes` (columna no relevante)	1.736 ms	5.510 ms	3,2×
`DELETE`	123 ms	72 ms	sin diferencia apreciable

Lectura

Consulta	`r_pocos`	`r_muchos`
Reservas de un huésped	0,31 ms	0,35 ms
Reservas de una habitación en 2015	87,5 ms	0,65 ms
Canceladas de un día	1,45 ms	0,44 ms

Interpretación

- **Cada índice es una estructura más que mantener.** Un `INSERT` en `r_muchos` escribe la fila y 13 entradas de índice, cada una en el lugar ordenado que le corresponde (con posibles divisiones de páginas y escrituras en el WAL).
- **El `UPDATE` de una columna no indexada también fue 3,2 veces más lento.** Por MVCC, un `UPDATE` crea una versión nueva de la fila en otra posición física, y todos los índices deben apuntar a ella. PostgreSQL lo evita con las actualizaciones **HOT** (*Heap-Only Tuple*) cuando la nueva versión entra en la misma página y no cambia ninguna columna indexada; aquí la tabla recién creada no tenía espacio libre en sus páginas, así que cada `UPDATE` actualizó los 13 índices. Con un índice por columna, además, *cualquier* `UPDATE` toca una columna indexada y HOT nunca se aplica.
- **El `DELETE` no muestra diferencia**: en PostgreSQL borrar sólo marca la fila como eliminada (`xmax`); las entradas de índice se limpian después, en el `VACUUM` . El costo aparece más tarde, en el mantenimiento.
- **Las lecturas sólo mejoran donde había una consulta que lo pedía.** El compuesto `(habitacion_id, fecha_inicio)` aceleró 130 veces la búsqueda por habitación; esa consulta justifica *ese* índice, no los otros 9.

Cuándo un índice es perjudicial

1. **En tablas con muchas escrituras** (la tabla de reservas recibe altas y confirmaciones todo el día): cada índice multiplica el costo de cada operación y el volumen de WAL.

2. **Sobre columnas de baja selectividad** (`estado` , `cantidad_huespedes`): ocupan espacio y el optimizador no los usa para los valores frecuentes (Punto 17).
3. **Índices redundantes**, que repiten el prefijo de otro (como `ix_huesped_email_h` , Punto 11).
4. **Cuando compiten por la memoria**: 165 MB de índices poco usados desplazan de `shared_buffers` páginas que sí se necesitan.
5. **Cuando confunden al optimizador**: más alternativas implican más tiempo de planificación y más chances de elegir un plan subóptimo si las estadísticas están desactualizadas.

El criterio adoptado: un índice se crea cuando hay una consulta frecuente que lo necesita, se mide su efecto en lectura **y** en escritura, y se revisa periódicamente `pg_stat_user_indexes.idx_scan` para eliminar los que nunca se usan.

Parte IV — Datos, Carga Masiva y Metodología Experimental

19. Generación de datos de prueba

Los datos se generan con dos scripts reproducibles: `10_generar_base.sql` (datos maestros, con `setseed(0.42)`) y `11_generar_reservas.py` (reservas, con semilla 42 y una fecha "hoy" fija: 11/10/2026). Ejecutados dos veces producen exactamente los mismos datos.

Volúmenes generados

Tabla	Filas
hotel	123
habitacion	7.663
habitacion_comodidad	68.776
tarifa (lista)	9.594
punto_interes / hotel_punto_interes	126 / 496
huesped	1.000.000
huesped_telefono	1.349.961
huesped_email	1.069.925

Tabla	Filas
reserva	**7.535.580**

Criterios por entidad

- **Hoteles.** 123 hoteles en 21 localidades turísticas reales de 15 provincias, repartidos por peso: Buenos Aires 25, Bariloche 12, Mendoza 10, Córdoba, Mar del Plata y Salta 8 cada una, hasta Humahuaca con 1. Categoría con predominio de 3 y 4 estrellas.
- **Habitaciones.** Cantidad según categoría: 15–30 en 2 estrellas, 30–60 en 3, 50–100 en 4 y 80–150 en 5. Tipos: 50 % dobles, 10 % singles, 15 % triples, 12 % cuádruples, 6 % suites, 7 % familiares. Comodidades más probables a mayor categoría y en suites.
- **Huéspedes.** 1 millón, 85 % argentinos con DNI y el resto con pasaporte de 9 países. Nombres de 40 nombres y 50 apellidos frecuentes en Argentina, con distribución sesgada de apellidos (algunos muy comunes). Un domicilio cada uno, 1 a 2 teléfonos y 0 a 2 emails.
- **Fechas.** Estadías del 01/01/2015 al 30/06/2027. Cada habitación recorre el calendario alternando estadías y días libres, por lo que **nunca** genera superposiciones (la carga pasa por la restricción `EXCLUDE` como prueba).
- **Estadía.** 1 a 14 noches, moda en 2 noches (25 %); 1 noche 20 %, 3 noches 20 %.
- **Temporada.** Días libres entre estadías: promedio 0,7 en temporada alta (enero, febrero, julio), 2 en media (marzo, octubre, diciembre) y 3,2 en baja. Cada hotel tiene además un factor de popularidad aleatorio entre 0,55 y 1,6.
- **Huésped de cada reserva.** Elección sesgada (`id = N × u^2,2`): pocos huéspedes frecuentes concentran muchas reservas, como en un programa de fidelidad.
- **Estados.** Reservas anteriores a hoy: 86 % finalizadas, 10 % canceladas, 4 % no-show. Futuras: 55 % confirmadas, 35 % pendientes, 10 % canceladas.
- **Tarifa.** USD por noche = base por categoría (35 a 220) × tipo de habitación (0,8 a 2,2) × temporada (1 a 1,3) × año (+3 % anual) × ruido ±10 %. Se usan dólares para evitar la distorsión inflacionaria. Sólo tienen tarifa las reservas confirmadas, finalizadas y la mitad de las canceladas o no-show (las que llegaron a confirmarse).
- **Creación y confirmación.** La reserva se crea entre 0 y 120 días antes de la llegada, con más reservas de último momento (`anticipación = 120 × u²`); se confirma entre 5 minutos y 48 horas después.
- **Orden físico.** El CSV se ordena por `fecha_creacion` antes de cargar, para que el orden físico de la tabla sea el de una operación real (las reservas se insertan a medida que se crean).

20. Carga masiva

Se cargaron **7.535.580 reservas** (supera el mínimo de 6 millones) con `COPY` en 7 min 57 s, unas 15.800 filas por segundo, sin desactivar ninguna restricción. La tabla quedó en 719 MB y sus índices en 663 MB.

Procedimiento (`12_carga_masiva.sh`)

1. `11_generar_reservas.py` escribe un CSV de 681 MB (1 min 32 s).
2. `sort` del sistema operativo lo ordena por fecha de creación (17 s, ordenamiento externo con 1 GB de memoria).
3. Se elimina el índice secundario `ix_reserva_huesped` para no mantenerlo fila por fila.
4. `\copy reserva (...) FROM 'reservas.csv' WITH (FORMAT csv, NULL '')`. La PK, las FK, los CHECK, el `EXCLUDE` y el trigger de capacidad se verifican durante la carga.
5. Se recrea `ix_reserva_huesped` (3,4 s) y se ejecuta `VACUUM (ANALYZE)` (0,9 s).

Comparación de métodos de carga (50.000 filas, misma tabla y restricciones)

Método	Filas por segundo	Relación con COPY
`INSERT` fila por fila, cada uno en su transacción	2.595	19 veces más lento
`INSERT` fila por fila en una sola transacción	8.066	6 veces más lento
`executemany` (lote) en una transacción	19.850	2,5 veces más lento
`COPY`	49.358	—

El salto entre las dos primeras filas es el costo del `COMMIT`: cada uno espera la sincronización del WAL a disco (~283 µs medidos en el Punto 10). El resto de la diferencia es el viaje de ida y vuelta cliente-servidor y el análisis de cada sentencia, que `COPY` hace una sola vez.

Resultado

Objeto	Tamaño
Tabla `reserva`	719 MB
`ex_reserva_superpuesta` (GiST)	429 MB
`reserva_pkey` (B-Tree)	161 MB
`ix_reserva_huesped` (B-Tree)	72 MB

Objeto	Tamaño
Total `reserva`	1.382 MB
Base completa	2.199 MB

Dificultades encontradas

- **La carga a 6 millones fue 3 veces más lenta por fila que la prueba de 50.000** (15.800 contra 49.358 filas/s). La causa es el índice GiST de la restricción `EXCLUDE`: con 50.000 filas cabe en memoria; con millones crece a 429 MB, supera `shared_buffers` (128 MB) y cada verificación lee páginas del disco.
- **Checkpoints frecuentes.** Con `max_wal_size = 1 GB` el servidor forzó un checkpoint cada 1 a 3 minutos durante la carga. En una carga repetida conviene subirlo temporalmente.
- **Datos maestros con tipos.** `round(double precision, int)` no existe en PostgreSQL: hubo que convertir `random()` a `numeric` para redondear coordenadas.
- **Reinicio del entorno.** La primera generación se interrumpió por un reinicio de la máquina virtual; como los scripts son re-ejecutables (empiezan con `DROP SCHEMA ... CASCADE`), se volvió a correr sin pasos manuales.
- **Decisión descartada:** desactivar el `EXCLUDE` para cargar más rápido. Se mantuvo porque garantiza que los 7,5 millones de reservas son válidas: todas las mediciones posteriores se hacen sobre datos que respetan las reglas de negocio.

21. Distribución de los datos

Los datos están deliberadamente sesgados en tres dimensiones (estado, huésped y hotel) y uniformes en otra (año). Ese sesgo es lo que hace que un mismo SQL tenga costos muy distintos según el valor buscado, y que el optimizador necesite estadísticas para decidir.

Temporada (sesgo moderado)



22_distribucion.sql · 7,5 M reservas, inicios 2016–2025

Por año la distribución es casi uniforme: entre 597 mil y 605 mil reservas por año completo (2015–2026), y 297.671 en el primer semestre de 2027.

Estado (sesgo fuerte, baja cardinalidad)

Estado	Reservas	%
F Finalizada	6.116.023	81,16
X Cancelada	753.914	10,00
N No-show	284.356	3,77
C Confirmada	232.557	3,09
P Pendiente	148.730	1,97

El optimizador conoce esta distribución: `pg_stats.most_common_vals = {F,X,N,C,P}` con frecuencias `{0,814; 0,101; 0,039; 0,029; 0,018}`, estimadas sobre una muestra de 30.000 filas y casi idénticas a las reales.

Huésped (sesgo extremo)

989.826 huéspedes tienen al menos una reserva; la mediana es 5 reservas, el promedio 7,6 y el máximo **14.054** (el huésped 1). El 10 % de los huéspedes con más reservas concentra el **36,1 %** de todas. Un huésped con 14 mil reservas no es realista para una persona; en la

práctica representa una cuenta corporativa o agencia de viajes, y sirve justamente como caso extremo.

Hotel y apellido

- El hotel más reservado (Bariloche 9, 5 estrellas) tiene 153.438 reservas; el menos reservado (Buenos Aires 2, 2 estrellas) 13.679: 11 veces menos, por cantidad de habitaciones y popularidad.
- "González" aparece en 86.802 huéspedes; "Ledesma" en 12.375.

Consecuencias sobre cardinalidad, selectividad y optimizador

Consulta	Filas	Páginas leídas	Tiempo
`huesped_id = 1`	14.054	13.039	61,5 ms
`huesped_id = 900000`	2	5	0,08 ms

El plan es el mismo (Bitmap Index Scan sobre `ix_reserva_huesped`), pero el costo es 800 veces mayor: el huésped frecuente tiene sus reservas desparramadas en 13.022 páginas distintas, porque se fueron insertando a lo largo de 12 años. Consecuencias:

1. **La selectividad depende del valor, no de la columna.** `estado = 'P'` (2 %) puede justificar un índice; `estado = 'F'` (81 %) nunca (Punto 17).
2. **Una estimación promedio se equivoca en los extremos.** Para valores fuera de la lista de más comunes el optimizador usa un promedio; con un sesgo así puede subestimar mucho (Punto 31).
3. **Planes genéricos peligrosos.** Una sentencia preparada con plan genérico usa el mismo plan para el huésped 1 y el 900000.
4. **Las mediciones deben elegir valores representativos.** Los benchmarks de la Parte VII indican siempre qué valor se usó.

22. Metodología de benchmarking

Cada medición del TP se toma con el mismo procedimiento, automatizado en `tp.py`: una ejecución de calentamiento, luego 3 a 5 repeticiones, y se informa la **mediana** junto con el plan (`EXPLAIN (ANALYZE, BUFFERS)`) de una ejecución adicional. Así cada tiempo se puede explicar con las páginas que leyó.

Condiciones fijas

Aspecto	Valor
Hardware	VM KVM, 2 núcleos Intel Xeon 2,1 GHz, 7,8 GB RAM, disco virtual ext4 (detalle en el Punto 10)
Sistema operativo	Ubuntu 24.04.5 LTS, Linux 6.18
Motor	PostgreSQL 16.15
Configuración	Por defecto (`shared_buffers` 128 MB, `work_mem` 4 MB) salvo en los experimentos del Punto 44 en adelante, que la modifican y lo indican
Volumen	7.535.580 reservas, 1.000.000 de huéspedes, 7.663 habitaciones; base de 2,2 GB
Conexión	Cliente Python (psycopg 3.3) por socket Unix, en la misma máquina: sin latencia de red salvo donde se mide
Conexiones simultáneas	1, salvo en concurrencia (2–3) y pool (1 a 32 con `pgbench`)
Paralelismo	`max_parallel_workers_per_gather = 0` : con 2 núcleos los planes paralelos agregan variación sin cambiar las conclusiones
Sentencias preparadas	Desactivadas en el cliente (`prepare_threshold=None`) para que cada repetición se planifique de nuevo
Estadísticas	`ANALYZE` después de cada carga y de cada índice creado

Procedimiento por medición

1. Ejecutar una vez y descartar (calentamiento de caché: ver Punto 23).
2. Ejecutar N veces (N = 3 en consultas largas, 5 en cortas) trayendo **todas** las filas al cliente, y tomar la mediana del tiempo de pared.
3. Ejecutar `EXPLAIN (ANALYZE, BUFFERS)` y registrar nodos, filas estimadas y reales, páginas leídas de caché (`shared hit`) y de disco (`shared read`).
4. Cuando se comparan dos variantes, cambiar **una sola cosa** (un índice, un `SET` , una reescritura) y repetir los pasos 1 a 3.

5. Para forzar un plan alternativo se usan `SET enable_seqscan / enable_indexscan / enable_hashjoin = off`, sólo dentro de la sesión del experimento.

Por qué la mediana y por qué las páginas

La mediana descarta ejecuciones atípicas (un checkpoint, el autovacuum, otro proceso de la VM) sin el sesgo del promedio. El tiempo depende de la máquina; la cantidad de páginas no, por eso las conclusiones se apoyan en ambas y otro integrante del grupo puede comparar sus resultados por proporciones.

Reproducibilidad

Todo el TP se reproduce en este orden:

```
createdb hoteles
psql -d hoteles -f 01_schema.sql
psql -d hoteles -f 10_generar_base.sql
python3 11_generar_reservas.py reservas_raw.csv
LC_ALL=C sort -t, -k1,1 reservas_raw.csv > reservas.csv
./12_carga_masiva.sh reservas.csv
psql -d hoteles -f 22_distribucion.sql
python3 20_indices.py
psql -d hoteles -f 21_estrategia_indices.sql
python3 31_correr_consultas.py
python3 40_optimizador.py
python3 50_optimizacion.py
python3 60_concurrencia.py
python3 80_particiones.py
python3 70_servidor.py      # reinicia el servidor y vacía la caché del SO
```

Las salidas completas de cada experimento (planes de ejecución, tiempos y páginas leídas) se resumen en las tablas de cada punto.

23. Cold cache y warm cache

La primera ejecución de una consulta después de un reinicio puede ser mucho más lenta que las siguientes: la búsqueda de las reservas de 200 huéspedes tardó **36,3 ms en frío y 1,5 ms en caliente** (24 veces menos). Comparar una ejecución fría de una variante con una caliente de otra lleva a conclusiones falsas; por eso todas las mediciones del TP descartan la primera ejecución y comparan medianas en caliente.

Procedimiento (`70_servidor.py`)

- **Caché fría completa**: reiniciar PostgreSQL (vacía `shared_buffers`) y además vaciar la caché de páginas del sistema operativo (`sync; echo 3 > /proc/sys/vm/drop_caches`). Así la primera ejecución lee realmente del disco.
- **Caché caliente**: repetir la misma consulta 3 veces inmediatamente después.
- Con `track_io_timing = on`, `EXPLAIN (ANALYZE, BUFFERS)` informa además cuánto tiempo pasó esperando al disco.

Resultados

Consulta	Ejecución	Tiempo	Páginas en caché / leídas	Tiempo de I/O
Reservas de 200 huéspedes (índice)	Fría	36,3 ms	807 / **458**	32,9 ms
	Caliente (mediana de 3)	1,6 ms	1.262 / 0	—
Agregación por mes de 2024 (índice de fecha)	Fría	215 ms	551 / **535**	10,3 ms
	Caliente	170 ms	1.086 / 0	—
`count(*)` de toda la tabla	Fría	1.804 ms	443.139 / **15.641**	400 ms
	Caliente	1.247 ms	458.779 / 0	—
`count(*)`, reiniciando sólo PostgreSQL (caché del SO intacta)	Primera	1.227 ms	443.138 / 15.641	127 ms

Interpretación

- **La diferencia depende de qué fracción del tiempo es I/O.** En la búsqueda por índice, 33 de los 36 ms fueron espera de disco: en caliente casi todo desaparece (×24). En la agregación por mes y el `count(*)` la mayor parte es CPU (agrupar, contar), y la caché sólo ahorra el tiempo de lectura (×1,3 a ×1,4).
- **Hay dos cachés.** Reiniciando sólo PostgreSQL, las 15.641 páginas "leídas" vinieron de la caché del sistema operativo: el I/O bajó de 400 ms a 127 ms. `shared read` en `EXPLAIN` no garantiza una lectura de disco físico.

- **Las páginas leídas no cambian, el tiempo sí.** Fría o caliente, la consulta toca las mismas páginas; lo que cambia es dónde las encuentra (`hit` vs `read`). Por eso los informes de este TP siempre muestran ambas cifras.

Cómo evitar conclusiones incorrectas

1. Descartar la primera ejecución (calentamiento) y usar la mediana de varias repeticiones.
2. Comparar variantes en el **mismo** estado de caché: todas en caliente, o todas en frío con el mismo procedimiento de vaciado.
3. Mirar `Buffers` además del tiempo: si una variante leyó del disco y la otra no, la comparación de tiempos no es válida.
4. Si interesa el rendimiento "del primer usuario del día", medir explícitamente en frío, como aquí.
5. Tener en cuenta que en producción la caché caliente es lo habitual para los datos recientes (las reservas de este mes) y la fría para los históricos que se consultan rara vez.

Parte V — SQL y Consultas del Sistema

24. Consultas SQL de dificultad progresiva

Las diez consultas van de un filtro de fechas sobre una tabla a funciones de ventana sobre seis tablas, y cubren cada construcción pedida. Todas están en `30_consultas.sql`; `31_correr_consultas.py` las mide con la metodología del Punto 22 sobre la base completa. "Hoy" es la fecha fija 11/10/2026. Cada encabezado indica filas devueltas, mediana y los nodos principales del plan.

24.1 — Fechas: reservas que comienzan hoy. 1.507 filas · 2,3 ms · Index Scan sobre `ix_reserva_fecha_inicio`.

```
SELECT reserva_id, habitacion_id, huesped_id, fecha_fin
  FROM reserva
 WHERE fecha_inicio = DATE '2026-10-11' AND estado IN ('C','P');
```

24.2 — INNER JOIN: huéspedes que ingresan hoy con su habitación. 1.507 filas · 10,0 ms · Index Scan + Nested Loop + Hash Join.

```
SELECT r.reserva_id, hu.apellido, hu.nombre, hb.numero AS habitacion,
r.fecha_fin
  FROM reserva r
  JOIN huesped hu    ON hu.huesped_id = r.huesped_id
  JOIN habitacion hb ON hb.habitacion_id = r.habitacion_id
 WHERE r.fecha_inicio = DATE '2026-10-11' AND r.estado IN ('C','P')
 ORDER BY hu.apellido, hu.nombre;
```

24.3 — LEFT JOIN: huéspedes registrados en 2026 que todavía no reservaron. 50 filas · 1.420 ms · Seq Scan sobre reserva (anti-join).

```
SELECT hu.huesped_id, hu.apellido, hu.nombre, hu.fecha_registro
  FROM huesped hu
  LEFT JOIN reserva r ON r.huesped_id = hu.huesped_id
 WHERE hu.fecha_registro >= TIMESTAMPTZ '2026-01-01 00:00-03'
   AND r.reserva_id IS NULL
 ORDER BY hu.fecha_registro DESC
 LIMIT 50;
```

24.4 — EXISTS: hoteles con alguna suite reservada para el próximo mes. 117 filas · 27,9 ms · Index Scan por fecha + semi-join por hash.

```
SELECT h.hotel_id, h.nombre
  FROM hotel h
 WHERE EXISTS (SELECT 1
                 FROM habitacion hb JOIN reserva r ON r.habitacion_id =
hb.habitacion_id
                WHERE hb.hotel_id = h.hotel_id AND hb.tipo_habitacion_id = 5
                  AND r.fecha_inicio >= '2026-11-01' AND r.fecha_inicio <
'2026-12-01'
                  AND r.estado IN ('C','P'));
```

24.5 — CASE WHEN: clasificación de reservas de 2025 por duración. 4 filas · 325 ms · Index Scan por fecha + HashAggregate.

```
SELECT CASE WHEN fecha_fin - fecha_inicio = 1 THEN '1 noche'
            WHEN fecha_fin - fecha_inicio <= 3 THEN '2 a 3 noches'
            WHEN fecha_fin - fecha_inicio <= 7 THEN '4 a 7 noches'
            ELSE 'más de 7 noches' END AS duracion,
       count(*) AS reservas
  FROM reserva
 WHERE fecha_inicio >= '2025-01-01' AND fecha_inicio < '2026-01-01'
 GROUP BY 1 ORDER BY min(fecha_fin - fecha_inicio);
```

24.6 — GROUP BY y agregaciones: estadísticas por estado. 5 filas · 2.184 ms · Seq Scan sobre las 7,5 M de reservas + HashAggregate.

```
SELECT e.descripcion, count(*) AS reservas,
       round(avg(r.fecha_fin - r.fecha_inicio), 2) AS noches_promedio,
       round(avg(r.tarifa_noche), 2) AS tarifa_promedio
  FROM reserva r JOIN estado_reserva e USING (estado)
 GROUP BY e.descripcion ORDER BY reservas DESC;
```

24.7 — ORDER BY y LIMIT: las 10 estadías más largas finalizadas. 10 filas · 1.088 ms · Seq Scan + top-N heapsort.

```
SELECT reserva_id, habitacion_id, fecha_inicio, fecha_fin, fecha_fin -
fecha_inicio AS noches
  FROM reserva WHERE estado = 'F'
 ORDER BY noches DESC, fecha_inicio DESC LIMIT 10;
```

24.8 — Subconsulta escalar: reservas 2025 con tarifa mayor al promedio del año. 1 fila · 652 ms · dos Index Scan por fecha (la subconsulta se calcula una sola vez, como *InitPlan*).

```
SELECT count(*)
  FROM reserva r
 WHERE r.fecha_inicio >= '2025-01-01' AND r.fecha_inicio < '2026-01-01'
   AND r.tarifa_noche > (SELECT avg(tarifa_noche) FROM reserva
                         WHERE fecha_inicio >= '2025-01-01' AND fecha_inicio
< '2026-01-01');
```

24.9 — Subconsulta en FROM y HAVING: huéspedes con más de 25 reservas. 20 filas · 853 ms · GroupAggregate sobre el índice de huésped + Hash Join.

```
SELECT hu.huesped_id, hu.apellido, hu.nombre, t.reservas
  FROM (SELECT huesped_id, count(*) AS reservas FROM reserva GROUP BY
huesped_id HAVING count(*) > 25) t
  JOIN huesped hu USING (huesped_id)
 ORDER BY t.reservas DESC LIMIT 20;
```

24.10 — Funciones de ventana: ranking de hoteles por reservas en 2025 dentro de su provincia. 123 filas · 533 ms · cinco Hash Join + WindowAgg.

```
SELECT provincia, hotel, reservas, rank() OVER (PARTITION BY provincia ORDER
BY reservas DESC) AS puesto
  FROM (SELECT p.nombre AS provincia, h.nombre AS hotel, count(*) AS reservas
          FROM reserva r
          JOIN habitacion hb ON hb.habitacion_id = r.habitacion_id
          JOIN hotel h       ON h.hotel_id = hb.hotel_id
          JOIN domicilio d   ON d.domicilio_id = h.domicilio_id
          JOIN localidad l   ON l.localidad_id = d.localidad_id
          JOIN provincia p   ON p.provincia_id = l.provincia_id
         WHERE r.fecha_inicio >= '2025-01-01' AND r.fecha_inicio < '2026-01-
01' AND r.estado = 'F'
         GROUP BY p.nombre, h.nombre) t
 ORDER BY provincia, puesto;
```

Las consultas que tocan un día o un mes resuelven en milisegundos por el índice de fecha; las que agregan toda la tabla (24.6, 24.7) recorren las 92.002 páginas y tardan 1 a 2 segundos. La 24.3 es candidata a optimizar: el anti-join recorre toda la tabla de reservas para devolver 50 huéspedes.

25. Consultas complejas

Se desarrollaron 15 consultas que responden preguntas reales del negocio; **9 involucran cuatro o más tablas** (el mínimo pedido era 5). Las que trabajan sobre un hotel, un huésped o un período corto responden en menos de 15 ms; los reportes que agregan un año entero o más tardan entre 0,5 y 2,2 segundos y son los candidatos a optimizar en la Parte VII.

#	Situación del sistema	Tablas	Filas	Tiempo
25.1	Disponibilidad: habitaciones dobles libres en Bariloche del 10 al 17/01/2027	6	4	14,1 ms
25.2	Reservas confirmadas de diciembre 2026 con huésped y hotel	4	100	6,8 ms
25.3	Top 10 hoteles con más reservas finalizadas en 2025	3	10	500 ms
25.4	Ocupación mensual 2025 por hotel (noches vendidas / noches disponibles)	3	1.476	1.605 ms
25.5	Facturación anual por provincia y categoría	6	126	2.198 ms
25.6	Historial de un huésped con hotel, localidad y comodidades	8	152	4,2 ms

#	Situación del sistema	Tablas	Filas	Tiempo
25.7	Huéspedes frecuentes: más de 5 provincias visitadas	6	20	1.335 ms
25.8	Puntos de interés a menos de 2 km de hoteles 5 estrellas	4	3	0,9 ms
25.9	Tasa de cancelación y no-show por hotel en 2025	3	10	485 ms
25.10	Anticipación promedio de reserva por temporada	1	3	1.088 ms
25.11	Evolución interanual de reservas por mes (LAG)	1	48	751 ms
25.12	Tarifa pactada frente a tarifa de lista por tipo de habitación	5	24	1.498 ms
25.13	Huéspedes extranjeros por país de origen y destino preferido	7	189	847 ms
25.14	Habitaciones sin reservas en los últimos 90 días	3	0	80,9 ms
25.15	Huéspedes en casa hoy con check-out mañana y su teléfono	5	1.298	799 ms

Observaciones

- **25.1 y 25.14** usan `NOT EXISTS` con la misma condición de superposición que la restricción `EXCLUDE`; el plan recorre las habitaciones candidatas y por cada una consulta el índice GiST (Nested Loop).
- **25.4** expande cada estadía en sus noches con `generate_series` para calcular la ocupación real de cada mes, incluidas las estadías que cruzan de un mes a otro.
- **25.12** une cada reserva con la tarifa de lista vigente en su fecha (JOIN por rango de vigencia).
- **25.14** devolvió 0 filas: en los últimos 90 días todas las habitaciones tuvieron al menos una reserva.
- **25.5, 25.7 y 25.12** son las más lentas porque unen cientos de miles de reservas con 5 o 6 tablas antes de agrupar; el Punto 33 analiza cómo reducir el trabajo de esos JOIN.

Código

25.1 — Disponibilidad: habitaciones dobles libres en Bariloche del 10 al 17/01/2027.

```
SELECT h.nombre AS hotel, hb.numero, hb.piso, th.nombre AS tipo
  FROM habitacion hb
  JOIN tipo_habitacion th ON th.tipo_habitacion_id = hb.tipo_habitacion_id
  JOIN hotel h            ON h.hotel_id = hb.hotel_id
  JOIN domicilio d        ON d.domicilio_id = h.domicilio_id
  JOIN localidad l        ON l.localidad_id = d.localidad_id
 WHERE l.nombre = 'San Carlos de Bariloche' AND th.nombre = 'Doble' AND
hb.activa
   AND NOT EXISTS (SELECT 1 FROM reserva r
                   WHERE r.habitacion_id = hb.habitacion_id AND r.estado IN
('P','C','F')
                     AND daterange(r.fecha_inicio, r.fecha_fin, '[)') &&
daterange('2027-01-10', '2027-01-17', '[)'))
 ORDER BY h.nombre, hb.numero;
```

25.2 — Reservas confirmadas de diciembre 2026 con huésped y hotel.

```
SELECT r.reserva_id, h.nombre AS hotel, hb.numero, hu.apellido || ', ' ||
hu.nombre AS huesped,
       r.fecha_inicio, r.fecha_fin, r.tarifa_noche
  FROM reserva r
  JOIN habitacion hb ON hb.habitacion_id = r.habitacion_id
  JOIN hotel h       ON h.hotel_id = hb.hotel_id
  JOIN huesped hu    ON hu.huesped_id = r.huesped_id
 WHERE r.fecha_inicio >= '2026-12-01' AND r.fecha_inicio < '2027-01-01' AND
r.estado = 'C'
 ORDER BY r.fecha_inicio, h.nombre
 LIMIT 100;
```

25.3 — Top 10 hoteles con más reservas finalizadas en 2025.

```
SELECT h.nombre, h.categoria, count(*) AS reservas
  FROM reserva r
  JOIN habitacion hb ON hb.habitacion_id = r.habitacion_id
  JOIN hotel h       ON h.hotel_id = hb.hotel_id
 WHERE r.fecha_inicio >= '2025-01-01' AND r.fecha_inicio < '2026-01-01' AND
r.estado = 'F'
 GROUP BY h.hotel_id ORDER BY reservas DESC LIMIT 10;
```

25.4 — Ocupación mensual 2025 por hotel.

```
WITH cap AS (SELECT hotel_id, count(*) AS habitaciones FROM habitacion GROUP
BY hotel_id),
     noches AS (
       SELECT hb.hotel_id, date_trunc('month', g)::date AS mes, count(*) AS
noches_ocupadas
         FROM reserva r
         JOIN habitacion hb ON hb.habitacion_id = r.habitacion_id
         CROSS JOIN LATERAL generate_series(r.fecha_inicio, r.fecha_fin - 1,
INTERVAL '1 day') g
        WHERE r.estado = 'F' AND r.fecha_inicio < '2026-01-01' AND
r.fecha_fin > '2025-01-01'
          AND g >= '2025-01-01' AND g < '2026-01-01'
        GROUP BY 1, 2)
SELECT h.nombre, n.mes,
       round(100.0 * n.noches_ocupadas / (c.habitaciones * extract(day FROM
(n.mes + INTERVAL '1 month' - INTERVAL '1 day'))), 1) AS ocupacion_pct
  FROM noches n JOIN cap c USING (hotel_id) JOIN hotel h USING (hotel_id)
 ORDER BY h.nombre, n.mes;
```

25.5 — Facturación anual por provincia y categoría.

```
SELECT p.nombre AS provincia, h.categoria, extract(year FROM
r.fecha_inicio)::int AS anio,
       sum(r.tarifa_noche * (r.fecha_fin - r.fecha_inicio)) AS facturado_usd
  FROM reserva r
  JOIN habitacion hb ON hb.habitacion_id = r.habitacion_id
  JOIN hotel h       ON h.hotel_id = hb.hotel_id
  JOIN domicilio d   ON d.domicilio_id = h.domicilio_id
  JOIN localidad l   ON l.localidad_id = d.localidad_id
  JOIN provincia p   ON p.provincia_id = l.provincia_id
 WHERE r.estado = 'F' AND r.fecha_inicio >= '2023-01-01' AND r.fecha_inicio <
'2026-01-01'
 GROUP BY 1, 2, 3 ORDER BY 1, 2, 3;
```

25.6 — Historial de un huésped con hotel, localidad y comodidades.

```
SELECT r.fecha_inicio, r.fecha_fin, e.descripcion AS estado, h.nombre AS
hotel, l.nombre AS localidad,
       string_agg(co.nombre, ', ' ORDER BY co.nombre) AS comodidades
  FROM reserva r
  JOIN estado_reserva e ON e.estado = r.estado
  JOIN habitacion hb    ON hb.habitacion_id = r.habitacion_id
  JOIN hotel h          ON h.hotel_id = hb.hotel_id
  JOIN domicilio d      ON d.domicilio_id = h.domicilio_id
  JOIN localidad l      ON l.localidad_id = d.localidad_id
  LEFT JOIN habitacion_comodidad hc ON hc.habitacion_id = hb.habitacion_id
  LEFT JOIN comodidad co ON co.comodidad_id = hc.comodidad_id
 WHERE r.huesped_id = 1234
 GROUP BY r.reserva_id, e.descripcion, h.nombre, l.nombre
 ORDER BY r.fecha_inicio DESC;
```

25.7 — Huéspedes frecuentes: más de 5 provincias visitadas.

```
SELECT hu.huesped_id, hu.apellido, hu.nombre, count(DISTINCT l.provincia_id)
AS provincias, count(*) AS estadias
  FROM reserva r
  JOIN huesped hu    ON hu.huesped_id = r.huesped_id
  JOIN habitacion hb ON hb.habitacion_id = r.habitacion_id
  JOIN hotel h       ON h.hotel_id = hb.hotel_id
  JOIN domicilio d   ON d.domicilio_id = h.domicilio_id
  JOIN localidad l   ON l.localidad_id = d.localidad_id
 WHERE r.estado = 'F' AND r.huesped_id <= 2000
 GROUP BY hu.huesped_id
HAVING count(DISTINCT l.provincia_id) > 5
 ORDER BY provincias DESC, estadias DESC LIMIT 20;
```

25.8 — Puntos de interés a menos de 2 km de hoteles 5 estrellas.

```
SELECT h.nombre AS hotel, t.nombre AS tipo, p.nombre AS punto, hp.distancia_m
  FROM hotel h
  JOIN hotel_punto_interes hp ON hp.hotel_id = h.hotel_id
  JOIN punto_interes p        ON p.punto_interes_id = hp.punto_interes_id
  JOIN tipo_punto_interes t   ON t.tipo_punto_interes_id =
p.tipo_punto_interes_id
 WHERE h.categoria = 5 AND hp.distancia_m < 2000
 ORDER BY h.nombre, hp.distancia_m;
```

25.9 — Tasa de cancelación y no-show por hotel en 2025.

```
SELECT h.nombre,
       count(*) AS reservas,
       round(100.0 * count(*) FILTER (WHERE r.estado = 'X') / count(*), 2) AS
cancelacion_pct,
       round(100.0 * count(*) FILTER (WHERE r.estado = 'N') / count(*), 2) AS
no_show_pct
  FROM reserva r JOIN habitacion hb USING (habitacion_id) JOIN hotel h USING
(hotel_id)
 WHERE r.fecha_inicio >= '2025-01-01' AND r.fecha_inicio < '2026-01-01'
 GROUP BY h.hotel_id ORDER BY cancelacion_pct DESC LIMIT 10;
```

25.10 — Anticipación promedio de reserva por temporada.

```
SELECT CASE WHEN extract(month FROM fecha_inicio) IN (1,2,7) THEN 'alta'
            WHEN extract(month FROM fecha_inicio) IN (3,10,12) THEN 'media'
ELSE 'baja' END AS temporada,
       round(avg(fecha_inicio - fecha_creacion::date), 1) AS
dias_anticipacion,
       count(*) AS reservas
  FROM reserva
 WHERE fecha_inicio >= '2024-01-01' AND fecha_inicio < '2026-01-01'
 GROUP BY 1 ORDER BY 2 DESC;
```

25.11 — Evolución interanual de reservas por mes (LAG).

```
SELECT anio, mes, reservas,
       round(100.0 * (reservas - lag(reservas) OVER (PARTITION BY mes ORDER
BY anio))
             / lag(reservas) OVER (PARTITION BY mes ORDER BY anio), 1) AS
variacion_pct
  FROM (SELECT extract(year FROM fecha_inicio)::int AS anio, extract(month
FROM fecha_inicio)::int AS mes, count(*) AS reservas
          FROM reserva WHERE fecha_inicio >= '2022-01-01' AND fecha_inicio <
'2026-01-01'
         GROUP BY 1, 2) t
 ORDER BY mes, anio;
```

25.12 — Tarifa pactada frente a tarifa de lista por tipo de habitación.

```
SELECT th.nombre AS tipo, h.categoria,
       round(avg(r.tarifa_noche), 2) AS tarifa_pactada,
       round(avg(t.precio_noche), 2)  AS tarifa_lista,
       round(100.0 * (avg(r.tarifa_noche) - avg(t.precio_noche)) /
avg(t.precio_noche), 1) AS desvio_pct
  FROM reserva r
  JOIN habitacion hb      ON hb.habitacion_id = r.habitacion_id
  JOIN hotel h            ON h.hotel_id = hb.hotel_id
  JOIN tipo_habitacion th ON th.tipo_habitacion_id = hb.tipo_habitacion_id
  JOIN tarifa t           ON t.hotel_id = h.hotel_id AND t.tipo_habitacion_id
= hb.tipo_habitacion_id
                         AND r.fecha_inicio >= t.vigencia_desde AND
r.fecha_inicio < t.vigencia_hasta
 WHERE r.estado = 'F' AND r.fecha_inicio >= '2025-01-01' AND r.fecha_inicio <
'2026-01-01'
 GROUP BY th.nombre, h.categoria ORDER BY h.categoria, th.nombre;
```

25.13 — Huéspedes extranjeros por país de origen y destino preferido.

```
SELECT pa.nombre AS pais_origen, l.nombre AS destino, count(*) AS estadias,
       rank() OVER (PARTITION BY pa.nombre ORDER BY count(*) DESC) AS puesto
  FROM reserva r
  JOIN huesped hu    ON hu.huesped_id = r.huesped_id
  JOIN pais pa       ON pa.pais_id = hu.nacionalidad_id
  JOIN habitacion hb ON hb.habitacion_id = r.habitacion_id
  JOIN hotel h       ON h.hotel_id = hb.hotel_id
  JOIN domicilio d   ON d.domicilio_id = h.domicilio_id
  JOIN localidad l   ON l.localidad_id = d.localidad_id
 WHERE pa.codigo_iso <> 'AR' AND r.estado = 'F' AND r.fecha_inicio >= '2025-
01-01' AND r.fecha_inicio < '2026-01-01'
 GROUP BY pa.nombre, l.nombre
 ORDER BY pa.nombre, puesto;
```

25.14 — Habitaciones sin reservas en los últimos 90 días.

```
SELECT h.nombre, hb.numero
  FROM habitacion hb JOIN hotel h USING (hotel_id)
 WHERE NOT EXISTS (SELECT 1 FROM reserva r
                   WHERE r.habitacion_id = hb.habitacion_id
                     AND r.fecha_inicio >= DATE '2026-10-11' - 90 AND
r.fecha_inicio < DATE '2026-10-11')
 ORDER BY 1, 2;
```

25.15 — Huéspedes en casa hoy con check-out mañana y su teléfono principal.

```
SELECT h.nombre AS hotel, hb.numero, hu.apellido, hu.nombre, tel.numero AS
telefono
  FROM reserva r
  JOIN habitacion hb ON hb.habitacion_id = r.habitacion_id
  JOIN hotel h       ON h.hotel_id = hb.hotel_id
  JOIN huesped hu    ON hu.huesped_id = r.huesped_id
  LEFT JOIN huesped_telefono tel ON tel.huesped_id = hu.huesped_id AND
tel.principal
 WHERE r.fecha_fin = DATE '2026-10-12' AND r.fecha_inicio <= DATE '2026-10-
11' AND r.estado IN ('C','F')
 ORDER BY h.nombre, hb.numero;
```

26. Consultas sobre disponibilidad

Una habitación está disponible para el período `[llegada, salida)` si **no existe** ninguna reserva activa (pendiente, confirmada o finalizada) de esa habitación que se superponga con él. Dos intervalos semiabiertos `[a, b)` y `[c, d)` se superponen si y sólo si `a < d` **y** `c < b`; es la misma condición que usa la restricción `EXCLUDE` con el operador `&&`.

Tres formas de preguntar si la habitación 1234 está libre del 2 al 6/07/2026

Esa habitación tiene una reserva del 3 al 13/07. La llegada pedida (día 2) cae en un hueco, pero la estadía choca con la reserva siguiente.

26.1 — Comparación explícita de extremos. Resultado: **no disponible** (correcto) · 0,3 ms · usa el índice GiST.

```
SELECT NOT EXISTS (
  SELECT 1 FROM reserva
   WHERE habitacion_id = 1234 AND estado IN ('P','C','F')
     AND fecha_inicio < DATE '2026-07-06'   -- empieza antes de que yo me
vaya
     AND fecha_fin    > DATE '2026-07-02'   -- termina después de que yo
llegue
) AS disponible;
```

26.2 — Operador de rangos `&&`. Resultado: **no disponible** (correcto) · 0,1 ms · 6 páginas.

```
SELECT NOT EXISTS (
  SELECT 1 FROM reserva
   WHERE habitacion_id = 1234 AND estado IN ('P','C','F')
     AND daterange(fecha_inicio, fecha_fin, '[)') && daterange('2026-07-02',
'2026-07-06', '[)')
) AS disponible;
```

26.3 — Versión errónea frecuente. Resultado: **disponible** (incorrecto).

```
SELECT NOT EXISTS (
  SELECT 1 FROM reserva
   WHERE habitacion_id = 1234 AND estado IN ('P','C','F')
     AND DATE '2026-07-02' BETWEEN fecha_inicio AND fecha_fin
) AS disponible_mal;
```

Sólo verifica si el día de llegada cae dentro de otra reserva. No detecta una reserva que empieza *durante* la estadía pedida, y además, por usar `BETWEEN` (intervalo cerrado), marcaría como ocupado el día en que el huésped anterior se va, aunque la habitación se libera ese día.

Versión	Resultado	Páginas	Tiempo	Comentario
26.1 comparación explícita	No disponible ✔	207	0,3 ms	El GiST indexa el rango, no los extremos: navega por la habitación y filtra
26.2 `&&` sobre `daterange`	No disponible ✔	6	0,1 ms	Coincide exactamente con la expresión del índice de la restricción
26.3 `BETWEEN` sobre la llegada	Disponible ✘	967	0,8 ms	Lógica incorrecta

Habitaciones libres de un hotel por tipo

26.4 · 2 filas · 1,4 ms. Para cada habitación del hotel 13 aplica la condición de 26.2.

```
SELECT th.nombre, count(*) AS libres
  FROM habitacion hb JOIN tipo_habitacion th USING (tipo_habitacion_id)
 WHERE hb.hotel_id = 13
   AND NOT EXISTS (SELECT 1 FROM reserva r
                   WHERE r.habitacion_id = hb.habitacion_id AND r.estado IN
('P','C','F')
                     AND daterange(r.fecha_inicio, r.fecha_fin, '[)') &&
daterange('2027-02-01','2027-02-08','[)'))
 GROUP BY th.nombre ORDER BY th.nombre;
```

En temporada alta (febrero) sólo quedan una suite y una triple libres en ese hotel. La consulta 25.1 aplica la misma lógica a todas las dobles de una ciudad.

Reglas de negocio contempladas

- Sólo bloquean las reservas en estado P, C o F; las canceladas y no-show liberan la habitación (RN11), igual que en la restricción.
- Intervalo semiabierto: una reserva que termina el 2 no impide llegar el 2.
- Habitación activa (`hb.activa`) en 25.1.
- La consulta informa disponibilidad; la garantía real la da el `EXCLUDE` al insertar. Entre la consulta y el `INSERT` otro usuario puede reservar la misma habitación: la base rechazará la segunda reserva (Punto 39).

27. Consultas históricas y temporales

La forma de escribir un filtro de fechas cambia el tiempo hasta 380 veces con el mismo resultado: contar las reservas de julio de 2024 tardó **1.077 ms con `date_trunc` sobre la columna y 2,8 ms con un rango** de fechas. La regla es no aplicar funciones a la columna filtrada y expresar los períodos como intervalos semiabiertos `[desde, hasta)`.

Consultas del sistema

#	Pregunta	Resultado	Tiempo
27.1	Reservas futuras (desde hoy) por estado	C 232.557 · P 148.730 · X 42.192	208 ms
27.2	Históricas por año	~603 mil por año desde 2015	1.319 ms
27.3	Por mes de 2025	enero 67.202 …	145 ms
27.4	Estadías **en curso** en julio 2025 (se solapan con el mes, no sólo empiezan en él)	58.632	537 ms

```
-- 27.1 futuras
SELECT estado, count(*) FROM reserva WHERE fecha_inicio >= DATE '2026-10-11'
GROUP BY estado;

-- 27.2 históricas por año
SELECT extract(year FROM fecha_inicio)::int AS anio, count(*) FROM reserva
 WHERE fecha_inicio < DATE '2026-10-11' GROUP BY 1 ORDER BY 1;

-- 27.3 por mes (date_trunc en el SELECT/GROUP BY, rango en el WHERE)
SELECT date_trunc('month', fecha_inicio)::date AS mes, count(*) FROM reserva
 WHERE fecha_inicio >= '2025-01-01' AND fecha_inicio < '2026-01-01' GROUP BY
1 ORDER BY 1;

-- 27.4 período de ocupación: toda estadía que se superpone con julio
SELECT count(*) FROM reserva
 WHERE fecha_inicio < '2025-08-01' AND fecha_fin > '2025-07-01' AND estado =
'F';
```

La 27.4 muestra la diferencia entre "reservas *del* mes" (las que empiezan en él) y "ocupación *en* el mes" (las que se superponen, incluidas las que empezaron en junio). La segunda no puede resolverse sólo con el índice de `fecha_inicio`, porque la condición `fecha_inicio < '2025-08-01'` abarca toda la historia: el optimizador recorrió la tabla completa.

Comparación de formas de expresar el mismo filtro

Filtro	Expresión	Páginas	Tiempo	¿SARGable?
Año 2024	`extract(year FROM fecha_inicio) = 2024`	7.075	809 ms	No
Año 2024	`to_char(fecha_inicio, 'YYYY') = '2024'`	7.075	1.416 ms	No
Año 2024	`fecha_inicio BETWEEN '2024-01-01' AND '2024-12-31'`	572	25,0 ms	Sí
Año 2024	`fecha_inicio >= '2024-01-01' AND fecha_inicio < '2025-01-01'`	572	25,3 ms	Sí
Julio 2024	`date_trunc('month', fecha_inicio) = '2024-07-01'`	7.075	1.077 ms	No
Julio 2024	`fecha_inicio >= '2024-07-01' AND fecha_inicio < '2024-08-01'`	60	2,8 ms	Sí

Todas devuelven lo mismo (604.593 reservas en 2024, 64.985 en julio). Con funciones sobre la columna el motor recorre **todo** el índice (7.075 páginas, Index Only Scan completo) y evalúa la función 7,5 millones de veces; con rangos navega directo a la porción del año o del mes. `to_char` es la peor: además de no usar el índice, convierte cada fecha a texto.

`BETWEEN` frente a rango semiabierto. Rinden igual sobre una columna `DATE`, pero `BETWEEN` es cerrado en ambos extremos: si la columna fuera `TIMESTAMP`, `BETWEEN '2024-01-01' AND '2024-12-31'` dejaría afuera todo lo ocurrido el 31/12 después de las 00:00. El rango `>= inicio AND < inicio del período siguiente` es correcto para cualquier tipo y no requiere calcular el último día del mes (28, 29, 30 o 31).

28. Transporte de datos

Traer al cliente un año de reservas con `SELECT *` y JOIN a cuatro tablas tardó **11,6 segundos**, aunque el servidor terminó su trabajo en 1,5 s: el resto fue mover 174 MB. Pedir sólo las dos columnas necesarias bajó a 0,5 s. La cantidad de filas y de columnas que viajan al cliente es tan importante como el plan de ejecución.

Pocas y muchas filas, pocas y muchas columnas (`32_transporte.py`)

Pedido	Filas	Tiempo en el servidor	Tiempo hasta que el cliente tiene todo	Volumen aprox.
1 fila, 2 columnas (por PK)	1	0,1 ms	0,4 ms	—
1 fila, `SELECT *` con JOIN a 4 tablas	1	0,6 ms	0,9 ms	—
Agregado de un año calculado en el servidor	1	147 ms	71 ms	—
Un año, 2 columnas	604.593	198 ms	483 ms	7,6 MB
Un año, `SELECT *` (11 columnas)	604.593	126 ms	1.433 ms	57,3 MB
Un año, `SELECT *` con JOIN a 4 tablas (~40 columnas)	604.593	1.465 ms	**11.601 ms**	174,3 MB

Con pocas filas el ancho no importa (0,4 vs 0,9 ms). Con muchas filas el tiempo crece casi en proporción a los megabytes: pasar de 2 a 11 columnas triplicó el tiempo, y a 40 columnas lo multiplicó por 24. Si el dato que se necesita es un total, calcularlo en el

servidor (agregado: 71 ms) evita mover 600 mil filas.

`SELECT *` frente a columnas necesarias

Además del volumen, `SELECT *` impide usar índices de cobertura. Con un índice `(huesped_id, fecha_inicio) INCLUDE (estado)`, las reservas de los huéspedes 1 a 50 (83.260 filas):

Consulta	Plan	Páginas	Tiempo
`SELECT fecha_inicio, estado …`	Index Only Scan	3.023	34,7 ms
`SELECT * …`	Bitmap Heap Scan (tiene que ir a la tabla)	56.565	328 ms

`SELECT *` también hace frágil a la aplicación: si se agrega una columna (como `hotel_id` en la Parte VII), todas las consultas empiezan a traerla sin necesitarla.

LIMIT

Consulta	Plan	Páginas	Tiempo
`ORDER BY fecha_inicio DESC LIMIT 20`	Index Scan Backward: lee las primeras 20 entradas y se detiene	13	0,41 ms
`ORDER BY tarifa_noche DESC LIMIT 20` (sin índice)	Seq Scan + *top-N heapsort* en 27 kB	99.112	770 ms
Traer el año ordenado y cortar en el cliente	Ordena 604.593 filas con archivos temporales	11.153 + 12.639 temporales	1.719 ms

`LIMIT` rinde al máximo cuando un índice entrega las filas en el orden pedido: el motor se detiene al llegar a 20. Sin índice igual conviene: el *top-N heapsort* sólo guarda las 20 mejores en memoria en lugar de ordenar todo. Cortar en el cliente es lo peor: ordena y transfiere todo para descartar casi todo.

Estrategias de paginación (páginas de 50 reservas)

Página	`LIMIT 50 OFFSET n`	Páginas leídas	*Keyset* (`WHERE reserva_id > último`)	Páginas leídas
1	0,46 ms	4	0,20 ms	4

Página	`LIMIT 50 OFFSET n`	Páginas leídas	*Keyset* (`WHERE reserva_id > último`)	Páginas leídas
100	0,92 ms	95	0,24 ms	5
10.000	51,7 ms	9.329	0,37 ms	6
100.000	484 ms	93.515	0,37 ms	5
150.000 (última)	704 ms	139.745	0,35 ms	5

Con `OFFSET` el motor tiene que recorrer y descartar todas las filas anteriores: el costo crece en proporción al número de página. Con *keyset* (también llamada paginación por cursor o *seek*), la aplicación recuerda la clave de la última fila mostrada y pide las siguientes: el índice navega directo y todas las páginas cuestan lo mismo.

Para un orden por fecha se usa la clave compuesta, que admite el índice: `WHERE (fecha_inicio, reserva_id) > ('2024-07-01', 4000000) ORDER BY fecha_inicio, reserva_id LIMIT 50` respondió en 1,4 ms. Limitaciones de *keyset*: no permite saltar a "la página 3.471" y requiere un orden estable y único (por eso se agrega `reserva_id` como desempate).

Parte VI — Optimizador, SARGabilidad y Planes de Ejecución

29. EXPLAIN y EXPLAIN ANALYZE

`EXPLAIN` muestra el plan que el optimizador **eligió y estimó** sin ejecutar la consulta; `EXPLAIN ANALYZE` la **ejecuta** y agrega lo que realmente pasó. La diferencia entre ambas es la herramienta principal de diagnóstico: en el ejemplo, el optimizador estimó 495.733 reservas y fueron 518.250 (4,5 % de error), por lo que el plan elegido es coherente con la realidad.

Consulta analizada: top 10 hoteles por reservas finalizadas en 2025

```
Limit  (cost=28935.07..28935.10 rows=10) (actual time=689.820..689.827
rows=10 loops=1)
  Buffers: shared hit=381191 read=5319
  ->  Sort  (rows=123) (actual rows=10)
        Sort Key: (count(*)) DESC
        Sort Method: top-N heapsort  Memory: 26kB
        ->  HashAggregate  (rows=123) (actual rows=123)
```

```
               Group Key: h.hotel_id
               Batches: 1  Memory Usage: 48kB
               ->  Hash Join  (rows=495733) (actual time=2.476..621.505
rows=518250)
                     Hash Cond: (hb.hotel_id = h.hotel_id)
                     ->  Hash Join  (rows=495733) (actual rows=518250)
                           Hash Cond: (r.habitacion_id = hb.habitacion_id)
                           ->  Index Scan using ix_reserva_fecha_inicio on
reserva r
                                 (cost=0.43..23577.18 rows=495733) (actual
time=0.046..422.258 rows=518250)
                                 Index Cond: ((fecha_inicio >= '2025-01-01')
AND (fecha_inicio < '2026-01-01'))
                                 Filter: (estado = 'F')
                                 Rows Removed by Filter: 85669
                                 Buffers: shared hit=381186 read=5269
                           ->  Hash  (rows=7663)  Buckets: 8192  Batches: 1
Memory Usage: 364kB
                                 ->  Seq Scan on habitacion hb  (rows=7663)
(actual rows=7663)
                     ->  Hash  (rows=123)
                           ->  Seq Scan on hotel h  (rows=123) (actual
rows=123)
Planning Time: 0.355 ms
Execution Time: 689.909 ms
```

(Salida resumida: se omitieron las líneas de costo repetidas de los nodos internos.)

Cómo leerlo

Indicador	Dónde aparece	Qué dice en este plan
Orden de ejecución	De adentro hacia afuera, de abajo hacia arriba	1) se recorren `habitacion` y `hotel` y se arman tablas hash en memoria; 2) se leen las reservas de 2025 por el índice; 3) cada reserva busca su habitación y su hotel en las tablas hash; 4) se agrupa por hotel; 5) se ordena y se cortan 10
Índices posibles y usado	`Index Scan using …`, `Index Cond`	Usa `ix_reserva_fecha_inicio` para el rango de fechas. `estado` no está en el índice: se aplica como `Filter` y descarta 85.669 filas ya leídas

Indicador	Dónde aparece	Qué dice en este plan
Tipo de acceso	Nombre del nodo	Index Scan sobre reserva; Seq Scan sobre las tablas chicas (leer 49 páginas enteras es más barato que usar un índice)
Filas estimadas vs reales	`rows=` en `cost` vs en `actual`	495.733 estimadas, 518.250 reales: estimación buena
Costo	`cost=inicio..total`	Unidades arbitrarias del optimizador (1 = leer una página secuencial). Sirve para comparar planes, no es tiempo
Tiempo real	`actual time=inicio..fin` (ms, por iteración)	El Index Scan consume 422 de los 690 ms: es el nodo a optimizar
Iteraciones	`loops=`	1 en todos los nodos: no hay Nested Loop que repita un nodo
Páginas	`Buffers: shared hit / read`	381.186 accesos encontrados en caché y 5.269 leídos del disco o de la caché del SO
Ordenamientos	`Sort Method`	`top-N heapsort` en 26 kB: como sólo se quieren 10, no ordena las 123 filas completas
Memoria de hash y agregación	`Batches`, `Memory Usage`	`Batches: 1`: todo entró en `work_mem` sin usar disco
Recorridos completos	`Seq Scan`	Sólo sobre `habitacion` (7.663 filas) y `hotel` (123)

Un plan con tabla temporal en disco

Ordenar todas las reservas de 2024 por tarifa (`ORDER BY tarifa_noche DESC LIMIT 100000`) muestra la otra cara:

```
Sort  (rows=587587) (actual time=989.781..1001.697 rows=100000)
  Sort Key: tarifa_noche DESC
  Sort Method: external merge  Disk: 48088kB
  Buffers: shared hit=377908 read=9644, temp read=7241 written=12040
```

`external merge Disk: 48088kB` indica que el ordenamiento no entró en `work_mem` (4 MB) y usó archivos temporales: 12.040 páginas escritas y 7.241 leídas de disco. Es la señal para revisar `work_mem` o el diseño de la consulta (Punto 43).

Opciones útiles

- `EXPLAIN (ANALYZE, BUFFERS)`: la combinación usada en todo el TP.
- `VERBOSE`: columnas de salida de cada nodo. `SETTINGS`: parámetros distintos de los por defecto que influyeron. `FORMAT JSON`: para herramientas de visualización.
- Cuidado: `EXPLAIN ANALYZE` **ejecuta** la sentencia. Sobre un `UPDATE` o `DELETE` hay que envolverlo en `BEGIN … ROLLBACK`.

30. SARGabilidad

Una condición es **SARGable** (*Search ARGument able*) cuando la columna aparece sola de un lado de la comparación, de modo que el motor puede usar el índice para **navegar** hasta los valores buscados. En las 8 comparaciones medidas, reescribir la condición sin funciones ni conversiones sobre la columna aceleró la consulta entre 30 y 1.100 veces con el mismo resultado en seis casos; en los otros dos (`LIKE` por prefijo y `OR`) la reescritura sola no alcanzó, por razones que se explican abajo.

Resultados

Patrón	No SARGable	Páginas / tiempo	SARGable	Páginas / tiempo
Función sobre la fecha	`extract(year FROM fecha_inicio) = 2024`	7.075 / 873 ms	`fecha_inicio >= '2024-01-01' AND < '2025-01-01'`	572 / 27,0 ms
Aritmética sobre la columna	`fecha_inicio + 30 < '2015-03-01'`	7.075 / 272 ms	`fecha_inicio < DATE '2015-03-01' - 30`	59 / 2,7 ms
Conversión de tipo	`huesped_id::text = '4242'`	92.002 (Seq Scan) / 815 ms	`huesped_id = 4242`	82 / 0,74 ms
Fecha como texto	`fecha_inicio::text LIKE '2024-07%'`	7.075 / 973 ms	rango de julio 2024	60 / 2,6 ms

Patrón	No SARGable	Páginas / tiempo	SARGable	Páginas / tiempo
Función sobre texto	`lower(apellido) = 'tolaba'` (sin índice funcional)	11.479 (Seq Scan) / 229 ms	`apellido = 'Tolaba'`	288 / 0,70 ms
`COALESCE` sobre la columna	`coalesce(fecha_inicio, '1900-01-01') = '2025-02-14'`	7.075 / 274 ms	`fecha_inicio = '2025-02-14'` (es `NOT NULL`)	6 / 0,51 ms
`LIKE` con comodín inicial	`apellido LIKE '%laba'`	893 / 54,7 ms	`apellido LIKE 'Tola%'`	893 / 45,5 ms (ver abajo)
`OR` entre columnas distintas	`huesped_id = 4242 OR fecha_inicio = '2015-02-03'`	1.260 / 6,5 ms	`UNION` de dos consultas	1.260 / 9,3 ms

Por qué pasa

El índice guarda los **valores de la columna** ordenados. Si la condición pide `extract(year FROM fecha_inicio) = 2024`, el índice no tiene ordenados los años sino las fechas, y el motor no puede saber dónde empieza 2024 sin calcular la función para cada entrada. Puede, como mucho, recorrer el índice completo (Index Only Scan de 7.075 páginas, caso de las fechas) o la tabla completa (Seq Scan, casos de `::text` y `lower`). Reescribiendo la condición como rango sobre la columna, navega directo.

Reglas aplicadas en las reescrituras:

1. Mover la aritmética al lado de la constante: `col + 30 < x` ⟶ `col < x - 30`.
2. Comparar con el tipo nativo de la columna; convertir la constante, no la columna.
3. Expresar años, meses y días como rangos semiabiertos.
4. No envolver en `COALESCE` columnas `NOT NULL`; si la columna admite nulos, escribir `(col = x OR col IS NULL)`.
5. Si la función es inevitable (búsqueda sin mayúsculas), crear un índice funcional con **la misma** expresión (Punto 16).

Casos particulares observados

- **`LIKE 'Tola%'` no navegó.** La base usa la collation `C.UTF-8`, y PostgreSQL sólo convierte un `LIKE` por prefijo en un rango de índice cuando el índice usa la collation `C` o la clase de operadores `text_pattern_ops`. Con `CREATE INDEX … (apellido`

`text_pattern_ops`) la misma consulta bajó a **15 páginas y 1,2 ms**. `LIKE '%laba'` (comodín inicial) no es SARGable en ningún caso: requiere un índice GIN de trigramas (Punto 13).

- **El `OR` no empeoró.** PostgreSQL combina dos índices con `BitmapOr` (una búsqueda por `huesped_id`, otra por fecha, y une los resultados), así que la reescritura con `UNION` no aportó y agregó el costo de eliminar duplicados. En motores sin combinación de índices (MySQL antes de *index merge*) un `OR` así sí obliga a recorrer la tabla.
- **Conversiones implícitas.** Si la aplicación envía un número como texto y la columna es entera, PostgreSQL convierte la constante y el índice se usa; el problema aparece cuando el `CAST` se escribe sobre la columna, como en el caso medido.

31. Estadísticas del optimizador

El optimizador no cuenta filas: las estima con estadísticas guardadas en `pg_statistic` (visibles en `pg_stats`), calculadas por `ANALYZE` sobre una muestra. Cuando las estadísticas están viejas el error puede ser enorme: con 600 mil filas nuevas sin analizar, el optimizador estimó 3.351 filas, eligió Nested Loop y la consulta tardó **2.982 ms**; tras `ANALYZE` eligió Hash Join y tardó **897 ms**.

Qué guarda por cada columna

Estadística	Para qué la usa	Ejemplo en `reserva`
`null_frac`	Selectividad de `IS NULL`	`tarifa_noche`: 0,088 (reservas pendientes y canceladas sin confirmar)
`n_distinct`	Selectividad de `=` sobre valores no frecuentes; cantidad de grupos en `GROUP BY`	`estado` 5; `fecha_inicio` 4.559; negativo = proporción de filas (`reserva_id` −1: todos distintos)
`most_common_vals` / `_freqs`	Selectividad exacta de los valores frecuentes	`estado`: F 0,81, X 0,10 …
`histogram_bounds`	Selectividad de rangos (`<`, `BETWEEN`)	101 límites que dividen `fecha_inicio` en 100 grupos de igual cantidad de filas
`correlation`	Costo de un Index Scan (¿lecturas en orden o salteadas?)	`fecha_inicio` 0,9996, `habitacion_id` 0,007

`ANALYZE` toma una muestra de 300 × `default_statistics_target` filas (100 por defecto: 30.000 filas) y se ejecuta automáticamente con el autovacuum cuando cambió más del 10 % de la tabla.

Experimento 1: estadísticas desactualizadas

Se copiaron las reservas de 2015 (603.241) a una tabla con autovacuum desactivado, se analizó, y se insertaron otras 603.241 con fechas de 2030 **sin** volver a analizar.

```
SELECT hu.apellido, count(*) FROM r_stats r JOIN huesped hu ON hu.huesped_id
= r.huesped_id
 WHERE r.fecha_inicio >= '2030-01-01' GROUP BY hu.apellido;
```

	Filas estimadas	Filas reales	Plan	Páginas	Tiempo
Antes de `ANALYZE`	3.351	603.241	Nested Loop: 603.241 búsquedas en `huesped_pkey` + Sort	2.774.299	2.982 ms
Después de `ANALYZE`	~600.000	603.241	Hash Join con Seq Scan de `huesped`	372.814	897 ms

El histograma decía que no había fechas posteriores a 2016; el optimizador supuso unas pocas filas y eligió el método que conviene para pocas filas. El error de estimación de **180 veces** produjo un plan 3,3 veces más lento.

Experimento 2: columnas correlacionadas

El optimizador supone que las condiciones son **independientes** y multiplica sus selectividades. En `huesped`, el tipo de documento y la nacionalidad no lo son: todos los argentinos tienen DNI y todos los extranjeros pasaporte.

```
SELECT count(*) FROM huesped WHERE tipo_documento = 'PASAPORTE' AND
nacionalidad_id = 1;
```

Estadísticas	Filas estimadas	Filas reales
Sólo por columna	129.058 (0,15 × 0,85 × 1 M)	0
`CREATE STATISTICS st_doc_nac (dependencies, mcv) ON tipo_documento, nacionalidad_id`	1	0

La estadística multicolumna midió la dependencia funcional `nacionalidad ⟹ tipo_documento` con grado 1,000 y guardó las combinaciones que realmente existen. En una consulta más compleja, la estimación de 129 mil filas habría llevado a elegir Hash Join o recorridos completos para lo que en realidad es un resultado vacío.

También se probó una estadística sobre la expresión `(fecha_confirmacion IS NULL)` combinada con `estado`: la estimación de `estado = 'P' AND fecha_confirmacion IS NULL` siguió en ~14 mil filas contra 148.730 reales, porque las estadísticas extendidas sólo se aplican cuando la condición coincide con la forma que el motor sabe combinar.

Experimento 3: tamaño de la muestra

`n_distinct` de `huesped_id` (valor real: 989.826 huéspedes con reservas):

`STATISTICS` de la columna	Filas muestreadas	Tiempo de `ANALYZE`	`n_distinct` estimado	Error
10	3.000	17 ms	170.948	−83 %
100 (por defecto)	30.000	169 ms	302.527	−69 %
1.000	300.000	1.196 ms	561.142	−43 %

Estimar cuántos valores distintos hay a partir de una muestra es difícil cuando la distribución está muy sesgada (Punto 21): la muestra ve muchas veces a los huéspedes frecuentes y casi nunca a los que tienen una sola reserva. Subir el objetivo mejora la estimación a cambio de un `ANALYZE` 7 veces más lento.

Casos donde el plan fue distinto del esperado

1. **Estadísticas viejas** (Experimento 1): Nested Loop en lugar de Hash Join.
2. **Correlación física engañosa** (Punto 32): el Index Scan elegido para el 48 % de la tabla fue 2,7 veces más lento que un Seq Scan.
3. **Índice nuevo no elegido** (Punto 37, O4): el índice de cobertura no se usó hasta que el mapa de visibilidad estuvo al día.

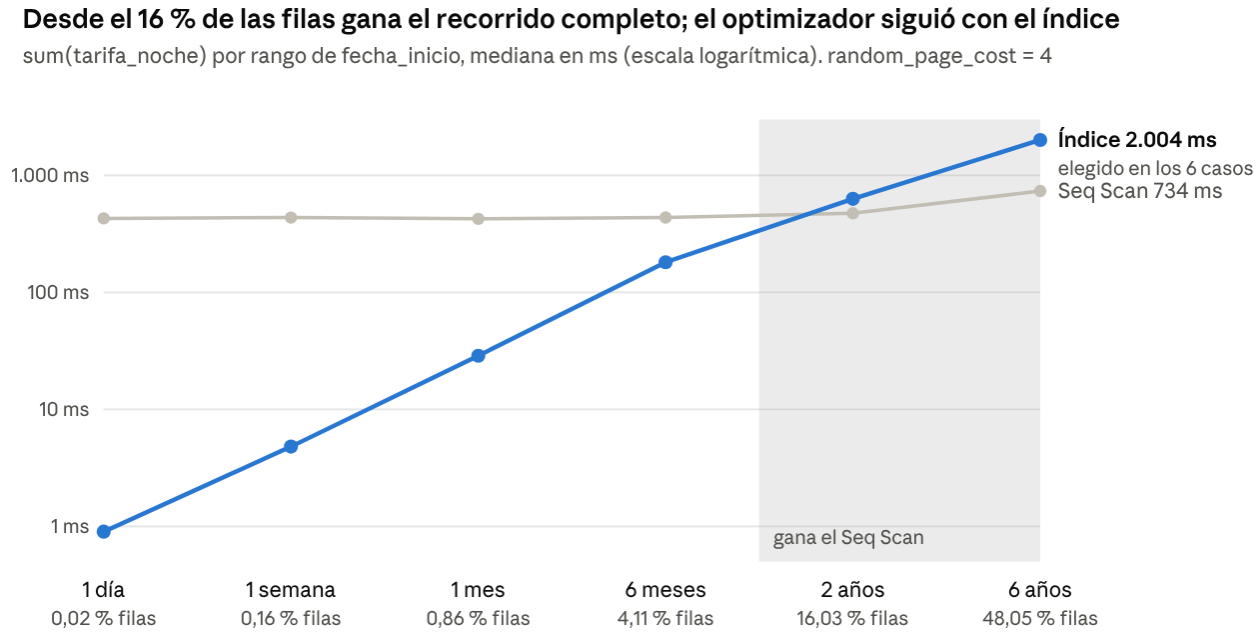
Buenas prácticas

- Ejecutar `ANALYZE` después de toda carga masiva y no depender sólo del autovacuum.
- Comparar `rows=` estimado y real en `EXPLAIN ANALYZE`: una diferencia de 10× o más suele explicar un mal plan.

- Usar `CREATE STATISTICS` para columnas que se filtran juntas y dependen entre sí.
- Subir `ALTER TABLE … ALTER COLUMN … SET STATISTICS` sólo en columnas con distribuciones sesgadas que lo justifiquen.

32. Costo y selección del plan

El optimizador elige el plan de menor **costo estimado**, y el costo depende de cuántas filas cree que devolverá la condición (selectividad) y de cuántas páginas cree que tendrá que leer. Al medir una suma de tarifas sobre rangos de fecha cada vez más anchos, el índice ganó hasta el 4 % de las filas, el recorrido completo ganó desde el 16 %, y el optimizador eligió el índice en los seis casos, incluido uno en que fue 2,7 veces más lento.



40_optimizador.py · Punto 32, 6 rangos de fecha_inicio sobre 7,5 M reservas

Costos estimados frente a tiempos reales

Rango	Filas	Costo índice	Costo Seq Scan	Elegido	Tiempo índice	Tiempo Seq Scan
2 años	16,03 %	45.966	208.038	Índice	630 ms	473 ms
6 años	48,05 %	138.935	214.120	Índice	2.004 ms	734 ms

Por qué el optimizador se equivoca aquí

El costo de un Index Scan depende mucho de la **correlación** entre el orden del índice y el orden físico de la tabla. `pg_stats` informa para `fecha_inicio` una correlación de **0,9996**: casi perfecta, porque las reservas se insertan en orden de creación y se crean poco antes

de la llegada. Con ese dato el optimizador supone que recorrer el índice equivale casi a leer la tabla en orden, y calcula un costo menor que el del recorrido completo incluso para el 48 % de las filas.

La realidad es más desordenada a pequeña escala: las reservas de un mismo día de llegada se crearon a lo largo de hasta 120 días y están repartidas en muchas páginas distintas. El Index Scan visita cada página varias veces: en el Punto 29 hizo **381.186 accesos a página para 518.250 filas**, cuando la tabla entera tiene 92.002 páginas. La correlación es una sola cifra para toda la columna y no captura ese desorden local.

Efecto de `random_page_cost`

`random_page_cost` (por defecto 4) es cuánto más cara que una lectura secuencial supone el optimizador que es una lectura salteada. Se repitió el experimento con 1,1 (valor típico para SSD):

Rango	Costo índice con 4	Costo índice con 1,1	Elegido con 1,1
1 mes	2.571	2.304	Índice
6 años	138.935	129.479	Índice

Con 1,1 el índice parece todavía más barato, así que el optimizador lo sigue eligiendo. En este entorno el parámetro no cambia la decisión porque, con la correlación en 0,9996, el modelo de costos ya trata casi todas las lecturas como secuenciales.

Por qué un recorrido completo puede ser mejor

1. **Lee cada página una sola vez y en orden**, aprovechando la lectura anticipada del sistema operativo (1,8 GB/s secuencial contra 195 MB/s salteado en este disco, Punto 10).
2. **El índice agrega trabajo por fila**: leer la entrada del índice, saltar a la tabla y verificar visibilidad.
3. A partir de cierta fracción, ese trabajo por fila supera el de leer todo. En este entorno el punto de equilibrio está entre el 4 % y el 16 % de las filas.

Qué se puede hacer cuando el plan es malo

- Verificar primero las estadísticas (`ANALYZE`, Punto 31) y que las filas estimadas coincidan con las reales.
- Para reportes de grandes períodos, agregar el filtro que hace al índice innecesario (una vista materializada, Punto 52, o una tabla particionada por año, Punto 50, donde el recorrido completo de una partición es lo natural).

- Forzar un plan con `SET enable_indexscan = off` sólo como diagnóstico, nunca en producción: lo que hoy es mejor puede dejar de serlo cuando cambian los datos.

33. Optimización de JOIN

Para la facturación por provincia de un trimestre (6 tablas), el optimizador eligió Hash Join en todas las uniones y fue la mejor opción: forzar Nested Loop la hizo 1,8 veces más lenta y Merge Join 1,4 veces. La reescritura que agrupa las reservas por habitación antes de unir redujo el trabajo de los JOIN de 156 mil filas a unas 7 mil, pero el tiempo total bajó apenas un 8 %, porque el costo dominante es leer las reservas, no unirlas.

Consulta

```
SELECT p.nombre AS provincia, sum(r.tarifa_noche * (r.fecha_fin -
r.fecha_inicio)) AS facturado
  FROM reserva r
  JOIN habitacion hb ON hb.habitacion_id = r.habitacion_id
  JOIN hotel h       ON h.hotel_id = hb.hotel_id
  JOIN domicilio d   ON d.domicilio_id = h.domicilio_id
  JOIN localidad l   ON l.localidad_id = d.localidad_id
  JOIN provincia p   ON p.provincia_id = l.provincia_id
 WHERE r.estado = 'F' AND r.fecha_inicio >= '2025-01-01' AND r.fecha_inicio <
'2025-04-01'
 GROUP BY p.nombre;
```

Cómo se ejecutan los JOIN

1. `reserva` se lee por `ix_reserva_fecha_inicio` (rango del trimestre) y se filtra `estado = 'F'`: 156.783 filas.
2. Las tablas del otro lado de cada unión (`habitacion` 7.663 filas, `hotel` 123, y la cadena domicilio–localidad–provincia) se cargan en tablas hash en memoria.
3. Cada reserva busca su habitación, hotel, domicilio, localidad y provincia en esas tablas hash: costo constante por fila.
4. `HashAggregate` acumula por provincia: 15 filas.

Métodos de JOIN comparados

Variante	Método	Páginas	Tiempo
Plan del optimizador	Hash Join (×5)	112.814	**194 ms**

Variante	Método	Páginas	Tiempo
`SET enable_hashjoin = off; enable_mergejoin = off`	Nested Loop (×5)	136.555	356 ms
`SET enable_hashjoin = off; enable_nestloop = off`	Merge Join (×5) con 5 Sort	112.836 + archivos temporales	264 ms
Reescritura: agrupar por habitación antes de unir	HashAggregate + Hash Join	112.814	**179 ms**
Reescritura + índice de cobertura `(fecha_inicio) INCLUDE (estado, habitacion_id, fecha_fin, tarifa_noche)`	El optimizador **no** lo eligió	112.814	187 ms

Método	Cómo funciona	Conviene cuando
Nested Loop	Por cada fila de la tabla externa busca coincidencias en la interna (idealmente por índice)	La externa tiene pocas filas y la interna tiene índice en la columna de unión (consultas 25.2, 25.6)
Hash Join	Arma una tabla hash con la tabla chica y recorre la grande consultándola	Igualdad, una tabla chica que entra en `work_mem`, muchas filas del otro lado (este caso)
Merge Join	Ordena ambas entradas por la columna de unión y las recorre en paralelo	Ambas entradas ya vienen ordenadas (por un índice) o son muy grandes para un hash

Modificaciones y resultados

- **Pre-agregación.** Agrupar `reserva` por `habitacion_id` antes de unir reduce las filas que atraviesan cinco Hash Join de 156.783 a como máximo 7.663 (una por habitación). Ganancia: 194 ⟶ 179 ms (8 %). Es poco porque el Index Scan sobre reserva ya representaba la mayor parte del tiempo.
- **Índice de cobertura.** Debería permitir un Index Only Scan, pero el optimizador siguió prefiriendo `ix_reserva_fecha_inicio`: con la correlación casi perfecta estima que el Index Scan común es igual de barato (el error del Punto 32). Se lo descartó.
- **Eliminar JOIN: desnormalización controlada.** La mejora real llega al quitar uniones del camino: con `hotel_id` copiado en `reserva` (Punto 5), las consultas por hotel no necesitan pasar por `habitacion`. Se mide en la Parte VII (optimizaciones 5 y 8).

- **Orden de los JOIN.** Con 6 tablas PostgreSQL evalúa todos los órdenes posibles (por debajo de `join_collapse_limit = 8`); a partir de 12 tablas pasa a un algoritmo genético. Por eso el orden en que se escriben los JOIN en el SQL no determina el orden de ejecución.

Parte VII — Optimización de Consultas

34. Selección de consultas a optimizar

Se eligieron 8 consultas que el sistema ejecuta con frecuencia y que en la Parte V o VI mostraron un problema identificable en su plan. Las 8 mejoraron, entre 3 y 9.000 veces; una de ellas necesitó un segundo intento porque la primera hipótesis no se confirmó.

#	Consulta	Problema detectado en el plan	Técnica
O1	Facturación de un mes (`extract(year)`, `extract(month)`)	Seq Scan: función sobre la columna	SARGabilidad
O2	Últimas 10 reservas de un huésped frecuente	Recorre el índice de fecha hacia atrás y descarta 4.276 filas	Índice compuesto
O3	Búsqueda de huésped sin distinguir mayúsculas	Seq Scan: `lower()` sobre la columna	Índice funcional
O4	Facturación diaria de un semestre	196.967 visitas a la tabla para leer 3 columnas	Índice de cobertura
O5	Reservas de un hotel en un mes	JOIN con `habitacion` sólo para obtener `hotel_id`	Desnormalización controlada + índice
O6	Paginación profunda del listado de reservas	`OFFSET 5.000.000` lee y descarta 5 millones de filas	Paginación por clave (*keyset*)
O7	Huéspedes sin reservas desde 2025	`NOT IN` con subconsulta materializada, recorrida por cada huésped	Reestructuración de subconsulta
O8	Ranking de hoteles por reservas	Subconsulta correlacionada ejecutada 123 veces	Reescritura de JOIN + desnormalización

35. Proceso de optimización

Cada consulta siguió el ciclo de la metodología (Punto 22): ejecutar la versión original y tomar la mediana, analizar el plan, formular una hipótesis, aplicar **un solo** cambio, volver a medir y comparar. Scripts: `50_optimizacion.py`, `52_optimizacion_resto.py` y `51_seguimiento_cobertura.py`.

#	Original	Páginas	Hipótesis y cambio	Optimizada	Páginas	Mejora
O1	972 ms	92.208	Reescribir como rango permite navegar `ix_reserva_fecha_inicio`	33,4 ms	38.319	**29×**
O2	1,92 ms	2.264	Un índice `(huesped_id, fecha_inicio DESC)` entrega las filas del huésped ya ordenadas	0,35 ms	13	**5,4×**
O3	222 ms	11.479	Índice sobre `(lower(apellido), lower(nombre))`	1,13 ms	340	**196×**
O4	244 ms	196.967	`INCLUDE (estado, fecha_fin, tarifa_noche)` permite no visitar la tabla	77,9 ms	1.817	**3,1×**
O5	29,5 ms	40.891	Con `hotel_id` en `reserva` y un índice `(hotel_id, fecha_inicio)` desaparece el JOIN	0,65 ms	250	**45×**
O6	2.404 ms	6.847.263	`WHERE reserva_id > último_visto` navega directo a la página pedida	0,26 ms	4	**9.166×**
O7	> 300 s (cancelada)	—	`NOT EXISTS` permite un Hash Anti Join	582 ms	896.392	**> 515×**
O8	9.347 ms	1.505.583	Un JOIN agrupado reemplaza 123 ejecuciones de la subconsulta	51,8 ms	25.097	**180×**

Detalle de cada caso

O1. El plan original hacía un Seq Scan de 92.208 páginas porque `extract()` impide usar el índice. Con el rango de fechas el plan pasa a Index Scan sobre el mes; las 38.319 páginas restantes son las visitas a la tabla para leer `tarifa_noche`.

O2. Sin el índice compuesto, el optimizador recorría `ix_reserva_fecha_inicio` desde la fecha más reciente hacia atrás, filtrando las reservas que no eran del huésped hasta juntar 10. Con `(huesped_id, fecha_inicio DESC)` lee exactamente las 10 primeras entradas del huésped. El nuevo índice también sirve para la FK `huesped_id` por prefijo izquierdo, así que reemplaza a `ix_reserva_huesped` (se eliminó).

O3. Igual que el Punto 16, aplicado como optimización.

O4. Hipótesis no confirmada en el primer intento. Con el índice de cobertura recién creado el optimizador siguió eligiendo el índice simple: 244 ms en ambos casos. Tras el mantenimiento de la tabla (Punto 48: `VACUUM FULL` y `VACUUM`, con el 100 % de las páginas marcadas como visibles) el plan pasó a **Index Only Scan** con `Heap Fetches: 0`: de 196.967 a 1.817 páginas y de 244 a 78 ms. La explicación más probable es que el mapa de visibilidad no estaba al día cuando se midió: si el optimizador estima que tendrá que ir a la tabla de todos modos, el índice de cobertura no le ofrece ventaja. Para comparar, sin índices la misma consulta tarda 595 ms (Seq Scan).

O5. Es la desnormalización propuesta en el Punto 5, implementada con `05_desnormalizacion.sql`. El plan original unía cada reserva del mes con `habitacion` para quedarse con las del hotel 13; el nuevo va directo a las reservas de ese hotel y ese mes con `ix_reserva_hotel_fecha`.

O6. Con `OFFSET 5000000` el motor recorre el índice de la PK y visita 5 millones de filas sólo para descartarlas; el costo crece con el número de página. Con *keyset* (`WHERE reserva_id > :ultimo ORDER BY reserva_id LIMIT 50`) toda página cuesta lo mismo. El Punto 28 muestra el crecimiento página por página.

O7. El plan estimado del `NOT IN` fue `Filter: NOT (SubPlan 1)` con una subconsulta materializada de 1,5 millones de filas que se recorre por cada uno del millón de huéspedes: costo estimado de 151 mil millones de unidades. Se canceló a los 300 s. `NOT IN` no puede convertirse en anti-join porque su semántica con `NULL` es distinta (si la subconsulta devolviera un `NULL`, el resultado sería vacío); `NOT EXISTS` no tiene esa restricción. Con `work_mem = 64MB` el `NOT IN` se resuelve como *hashed SubPlan* en 1.558 ms, pero `NOT EXISTS` sigue siendo 2,7 veces más rápido y no depende de la memoria disponible.

O8. La subconsulta en el `SELECT` se ejecutaba una vez por cada uno de los 123 hoteles, recorriendo cada vez las reservas del año. Reescrita como un único `JOIN` con `GROUP BY` sobre la columna desnormalizada `hotel_id`, recorre las reservas una sola vez. Se verificó que ambas devuelven el mismo resultado.

36. Técnicas de optimización utilizadas

Técnica	Optimizaciones	Requiere cambiar la base	Riesgo
Índice simple / compuesto	O2	Sí: un índice	Costo de escritura y espacio
Índice de cobertura	O4	Sí: un índice grande	Depende de `VACUUM` para rendir

Técnica	Optimizaciones	Requiere cambiar la base	Riesgo
Índice funcional	O3	Sí	La consulta debe usar la misma expresión
SARGabilidad	O1	No: sólo el SQL	Ninguno
Reestructuración de subconsultas	O7, O8	No: sólo el SQL	Verificar que el resultado sea idéntico (`NULL`)
Reescritura de JOIN	O8	No	Ninguno
Paginación por clave	O6	No: cambia la API de la aplicación	No permite saltar a una página arbitraria
Modificación justificada del diseño	O5, O8	Sí: columna nueva + FK compuesta	Redundancia controlada por la FK

Tres de las ocho mejoras (O1, O6, O7) no requirieron tocar la base: sólo escribir mejor el SQL. Son las primeras a aplicar porque no tienen costo de escritura ni de espacio.

37. Validación de la optimización

Todas las mejoras se confirmaron con la mediana de varias ejecuciones, con el plan y con la verificación de que el resultado no cambió. Los efectos secundarios se midieron explícitamente.

Efectos secundarios medidos

Aspecto	Antes de la Parte VII	Después	Variación
`INSERT` de 20.000 reservas (luego revertido)	1.561 ms	1.856 ms	**+19 %**
Tamaño de la tabla `reserva`	719 MB	774 MB	+55 MB (columna `hotel_id`)
Índices de `reserva`	714 MB (4 índices)	1.262 MB (6 índices)	+548 MB

Aspecto	Antes de la Parte VII	Después	Variación
Índice por huésped	`ix_reserva_huesped` 72 MB	`ix_reserva_huesped_fecha` 226 MB	El compuesto no aprovecha la deduplicación
Índice de cobertura	—	291 MB	El más caro de los agregados

Consultas que podrían haber empeorado

- **Búsquedas por la FK `huesped_id`** (al borrar un huésped o listar sus reservas): siguen usando el índice nuevo por prefijo izquierdo, ahora como Index Only Scan, 0,32 ms.
- **Reservas de un mes** (que antes usaban `ix_reserva_fecha_inicio`): el optimizador prefiere ahora el índice de cobertura, Index Only Scan, 8,6 ms. No empeoró.
- **El `UPDATE` que agregó `hotel_id`** duplicó transitoriamente la tabla (1.494 MB) y todos los índices (2.246 MB) por MVCC; recuperar el espacio requirió `VACUUM FULL` (2 min 18 s con la tabla bloqueada). En producción, una migración así se hace por lotes.

Balance

Las lecturas mejoraron entre 3 y 9.000 veces a cambio de un 19 % más de costo por inserción y 600 MB más de almacenamiento. Para un sistema de reservas, donde por cada reserva creada hay decenas de consultas de disponibilidad, historial y reportes, el balance es favorable. Si el volumen de escritura creciera mucho, el primer candidato a eliminar sería el índice de cobertura (291 MB para mejorar 3 veces un solo reporte); los cambios que sólo reescriben SQL (O1, O6, O7) no tienen contrapartida.

Parte VIII — Transacciones y Concurrencia

38. Bloqueos

PostgreSQL bloquea en dos niveles: **tabla** (8 modos, de `ACCESS SHARE` a `ACCESS EXCLUSIVE`) y **fila** (4 modos, de `FOR KEY SHARE` a `FOR UPDATE`). Gracias a MVCC, una lectura común nunca espera a una escritura ni la bloquea: en todas las pruebas, un `SELECT` simple de la conexión B respondió al instante aunque A tuviera la fila bloqueada para actualizar. Sólo `ACCESS EXCLUSIVE` (que toman `DROP`, `TRUNCATE`, `VACUUM FULL` y parte de los `ALTER TABLE`) detiene también a los lectores.

Los experimentos usan dos conexiones (A y B) y una tercera que observa `pg_locks` y `pg_stat_activity` mientras B espera (`60_concurrencia.py`). Se usaron las reservas 1000 y 2000; todos los cambios se revirtieron al final.

Bloqueos de tabla

Prueba	A tiene	B intenta	Resultado	Lo que muestra `pg_locks`
38.1	`LOCK TABLE reserva IN ACCESS EXCLUSIVE MODE`	`SELECT` de una fila	**Espera** 1,5 s, hasta el `COMMIT` de A	A: `AccessExclusiveLock` concedido; B: `AccessShareLock` **no** concedido
38.2	`LOCK TABLE reserva IN SHARE MODE`	`SELECT`	Responde al instante	—
38.2	idem	`UPDATE`	**Espera** 1,5 s	A: `ShareLock`; B: `RowExclusiveLock` no concedido

Bloqueos de fila: compartido y exclusivo

Prueba	A tiene	B intenta	Resultado
38.3	`SELECT … FOR SHARE` (compartido)	otro `FOR SHARE`	Lo obtiene al instante: dos bloqueos compartidos conviven
38.3	idem	`UPDATE` de la misma fila	**Espera** hasta el `ROLLBACK` de A
38.4	`SELECT … FOR UPDATE` (exclusivo)	`SELECT` simple	Responde al instante con el valor confirmado (MVCC)
38.4	idem	`FOR UPDATE NOWAIT`	Error inmediato: *could not obtain lock on row in relation "reserva"*
38.4	idem	`FOR UPDATE SKIP LOCKED` sobre las reservas 1000 y 2000	Devuelve sólo la 2000: saltea la bloqueada

Matriz de compatibilidad de los modos usados

A \ B	`ACCESS SHARE` (SELECT)	`ROW EXCLUSIVE` (INSERT, UPDATE, DELETE)	`SHARE`	`ACCESS EXCLUSIVE`
`ACCESS SHARE`	Compatible	Compatible	Compatible	Conflicto
`ROW EXCLUSIVE`	Compatible	Compatible	Conflicto	Conflicto
`SHARE`	Compatible	Conflicto	Compatible	Conflicto
`ACCESS EXCLUSIVE`	Conflicto	Conflicto	Conflicto	Conflicto

Fila: A \ B	`FOR SHARE`	`FOR UPDATE / UPDATE / DELETE`
`FOR SHARE`	Compatible	Conflicto
`FOR UPDATE`	Conflicto	Conflicto

Cómo se guardan los bloqueos de fila

PostgreSQL no guarda un bloqueo por fila en memoria (no hay escalamiento de bloqueos como en SQL Server): marca la propia tupla con el id de la transacción que la bloquea (`xmax`). Cuando B encuentra la fila marcada, se pone a esperar **a la transacción** de A. Por eso, en `pg_locks` , B aparece esperando un `ShareLock` de tipo `transactionid` (no concedido) y un bloqueo de tipo `tuple` que marca su lugar en la fila.

Comparación con InnoDB: InnoDB también bloquea a nivel fila, pero sobre los registros del índice, y en `REPEATABLE READ` agrega *gap locks* (bloquea los huecos entre claves) para impedir inserciones fantasma. PostgreSQL no usa *gap locks*: resuelve los fantasmas con instantáneas MVCC (Punto 40).

39. Concurrencia con dos o más conexiones

Cuando dos conexiones modifican la misma fila, la segunda espera a que la primera termine y luego aplica su cambio **sobre el valor ya confirmado**: no hay actualizaciones perdidas. Y cuando dos clientes intentan reservar la misma habitación para fechas superpuestas al mismo tiempo, la restricción `EXCLUDE` hace esperar al segundo y lo rechaza si el primero confirma.

39.1 Modificación concurrente de la misma reserva

Tarifa original de la reserva 1000: 422,68.

t (s)	Conexión A	Conexión B
4,55	`UPDATE … SET tarifa_noche = tarifa_noche + 100` (sin `COMMIT`)	
4,55		`SELECT tarifa_noche` ⟶ **422,68** al instante (ve la versión confirmada; no espera)
4,55		`UPDATE … SET tarifa_noche = tarifa_noche + 1` ⟶ **espera**
		`pg_stat_activity`: `wait_event_type = Lock`, `wait_event = transactionid`, bloqueado por el proceso de A
6,56	`COMMIT`	
6,56		Continúa y devuelve **523,68**

El resultado final es 422,68 + 100 + 1. En `READ COMMITTED`, cuando B obtiene el bloqueo, vuelve a leer la fila ya modificada por A y aplica su `+1` sobre esa versión.

39.2 Dos reservas superpuestas de la misma habitación al mismo tiempo

t (s)	Conexión A	Conexión B
6,59	`INSERT` habitación 617, del 10 al 15/03/2029 (sin `COMMIT`)	
6,59		`INSERT` de la misma habitación y fechas ⟶ **espera** a la transacción de A
8,60	`COMMIT`	
8,60		**ERROR**: *conflicting key value violates exclusion constraint "ex_reserva_superpuesta"*

Repitiendo con `ROLLBACK` en A en lugar de `COMMIT`, el `INSERT` de B se completó sin error: la habitación había quedado libre.

Esta es la demostración de por qué la regla RN1 debe vivir en la base (Punto 4). Si la aplicación consultara disponibilidad y luego insertara, ambas conexiones verían la habitación libre e insertarían. El índice GiST de la restricción detecta la entrada aún no confirmada de A, y B debe esperar a saber si esa reserva se confirma antes de decidir.

Qué espera y qué no

A tiene sin confirmar	B hace	¿Espera?
`UPDATE` de una fila	`SELECT` de esa fila	No: lee la versión anterior
`UPDATE` de una fila	`UPDATE` / `DELETE` / `FOR UPDATE` de esa fila	Sí, hasta `COMMIT` o `ROLLBACK` de A
`UPDATE` de una fila	`UPDATE` de **otra** fila	No
`INSERT` de una reserva	`INSERT` superpuesto en la misma habitación	Sí (por la restricción `EXCLUDE`)
`INSERT` de una reserva	`INSERT` en otra habitación	No

40. Niveles de aislamiento

PostgreSQL ofrece los cuatro niveles del estándar, pero `READ UNCOMMITTED` se comporta como `READ COMMITTED`: las lecturas sucias son imposibles. En los experimentos, `READ COMMITTED` permitió lecturas no repetibles y fantasmas; `REPEATABLE READ` evitó ambas pero dejó pasar una violación de regla de negocio por *write skew*; sólo `SERIALIZABLE` la detuvo.

Resultados

Fenómeno	Prueba	`READ UNCOMMITTED`	`READ COMMITTED` (por defecto)	`REPEATABLE READ`	`SERIALIZABLE`
Dirty read	A pone tarifa = 1 sin confirmar; B lee	No ocurre: B leyó 422,68	No ocurre	No ocurre	No ocurre
Non-repeatable read	B lee, A suma 50 y confirma, B relee	—	**Ocurre**: 422,68 ⟶ 472,68	No ocurre: 422,68 ⟶ 422,68	No ocurre

Fenómeno	Prueba	`READ UNCOMMITTED`	`READ COMMITTED` (por defecto)	`REPEATABLE READ`	`SERIALIZABLE`
Phantom read	B cuenta reservas 2029 de una habitación, A inserta una y confirma, B recuenta	—	**Ocurre**: 0 ⟶ 1	No ocurre: 0 ⟶ 0	No ocurre
Actualización concurrente	B lee; A actualiza y confirma; B actualiza la misma fila	—	B aplica su cambio sobre el valor nuevo	B recibe *could not serialize access due to concurrent update*	Igual que `REPEATABLE READ`
Write skew	Ver abajo	—	**Ocurre**	**Ocurre**: regla violada	No ocurre: B recibe *could not serialize access due to read/write dependencies among transactions*

Write skew: una regla que ninguna restricción garantiza

Regla de ejemplo: *un huésped no puede tener dos reservas pendientes que empiecen el mismo día*. Cada transacción verifica que el huésped 999999 no tenga ninguna y luego inserta una, en habitaciones distintas (la restricción `EXCLUDE` no interviene).

Paso	A	B
1	`SELECT count(*)` ⟶ 0	
2	`INSERT`	
3		`SELECT count(*)` ⟶ 0 (no ve el `INSERT` de A, sin confirmar)
4		`INSERT`
5	`COMMIT` ok	
6		`COMMIT`: **ok** en `REPEATABLE READ` (quedan 2 reservas) / **error** en `SERIALIZABLE` (queda 1)

Ninguna de las dos transacciones modificó una fila que la otra modificara, por eso `REPEATABLE READ` no ve conflicto. `SERIALIZABLE` registra qué leyó cada transacción (*predicate locks*, técnica SSI) y detecta que no existe un orden en serie que produzca el mismo resultado.

Qué nivel usar en este sistema

- **`READ COMMITTED`** para la operación general: la regla crítica (no superponer reservas) la garantiza la restricción `EXCLUDE` en cualquier nivel (Punto 39).
- **`REPEATABLE READ`** para reportes de varias consultas que deben ver una foto consistente (por ejemplo, totales y detalle de facturación del mismo mes).
- **`SERIALIZABLE`** para reglas que dependen de leer varias filas y que no pueden expresarse como restricción, como la del ejemplo. Exige que la aplicación reintente la transacción cuando recibe el error de serialización.

Diferencias con MySQL/InnoDB

En InnoDB `READ UNCOMMITTED` sí permite lecturas sucias, el nivel por defecto es `REPEATABLE READ` y los fantasmas se evitan con *gap locks* (bloqueando) en lugar de instantáneas. `SERIALIZABLE` en InnoDB convierte cada `SELECT` en `SELECT … FOR SHARE`, con lo que las transacciones se bloquean en lugar de fallar al confirmar.

41. Deadlocks

Se provocó un deadlock con dos transacciones que actualizan las reservas 1000 y 2000 en orden inverso. PostgreSQL lo detectó **1 segundo** después de que se cerró el ciclo (el valor de `deadlock_timeout`), abortó una de las transacciones y dejó continuar a la otra. Bloqueando las filas siempre en el mismo orden, el mismo escenario terminó sin deadlock.

Escenario

t (s)	Transacción A	Transacción B
10,16	`UPDATE reserva … WHERE reserva_id = 1000` ⟶ bloquea la 1000	
10,16		`UPDATE reserva … WHERE reserva_id = 2000` ⟶ bloquea la 2000
10,16	`UPDATE … WHERE reserva_id = 2000` **⟶ espera a B**	

t (s)	Transacción A	Transacción B
10,66		`UPDATE … WHERE reserva_id = 1000` ⟶ **espera a A**: ciclo cerrado
11,16	**ERROR: deadlock detected** (transacción abortada)	Obtiene la 1000 y continúa

Transacciones y recursos involucrados (log del servidor)

```
ERROR:  deadlock detected
DETAIL:  Process 1156 waits for ShareLock on transaction 11623; blocked by
process 1157.
         Process 1157 waits for ShareLock on transaction 11622; blocked by
process 1156.
         Process 1156: UPDATE reserva SET observaciones = 'A' WHERE
reserva_id = 2000
         Process 1157: UPDATE reserva SET observaciones = 'B' WHERE
reserva_id = 1000
CONTEXT:  while locking tuple (13547,47) in relation "reserva"
```

Cada proceso espera la **transacción** del otro (recordar que los bloqueos de fila se esperan esperando a la transacción dueña, Punto 38). El recurso concreto es la tupla en la página 13.547, posición 47, es decir la reserva 2000. El contador `pg_stat_database.deadlocks` pasó de 0 a 1.

Cómo se detecta

Cuando una transacción lleva esperando un bloqueo más que `deadlock_timeout` (1 s por defecto), el servidor construye el grafo de esperas (quién espera a quién) y busca un ciclo. Si lo hay, aborta a la transacción que hizo la comprobación, que recibe el error y pierde todo su trabajo; la otra continúa. Revisar sólo después de un segundo evita el costo de buscar ciclos en cada espera corta, que es lo más frecuente.

Estrategias para reducir la probabilidad

1. **Orden consistente.** Si todas las transacciones bloquean las filas en el mismo orden, no puede formarse un ciclo. Probado: con `SELECT … WHERE reserva_id IN (1000, 2000) ORDER BY reserva_id FOR UPDATE` en ambas, B esperó 1 s a que A confirmara y luego obtuvo las dos filas, sin error.
2. **Transacciones cortas**: menos tiempo con bloqueos tomados, menos posibilidades de cruzarse. No esperar al usuario (ni a una API de pagos) con una transacción abierta.

3. **Bloquear todo lo necesario al principio** con `SELECT … FOR UPDATE` en lugar de ir bloqueando a medida que se actualiza.
4. **`NOWAIT`** o **`lock_timeout`** para fallar rápido en lugar de esperar, cuando la operación puede reintentarse.
5. **Reintentar** automáticamente la transacción abortada: el deadlock no es un error de datos sino de sincronización, y el reintento casi siempre funciona.
6. **Índices en las FK**: sin índice, verificar una FK puede bloquear más filas de las necesarias y aumentar los cruces.

42. Identificación y diagnóstico de bloqueos

PostgreSQL expone bloqueos, esperas y transacciones activas en vistas del sistema que se consultan con SQL común. Durante todas las pruebas de esta parte, una tercera conexión ejecutó las dos consultas siguientes mientras B estaba esperando, y así se obtuvieron los diagnósticos que aparecen en los Puntos 38 a 41.

Quién espera y quién bloquea

```
SELECT a.pid, a.application_name, a.state, a.wait_event_type, a.wait_event,
       pg_blocking_pids(a.pid) AS bloqueado_por,
       now() - a.xact_start AS duracion_transaccion,
       left(a.query, 60) AS consulta
  FROM pg_stat_activity a
 WHERE a.datname = current_database() AND a.state <> 'idle';
```

Resultado real durante el Punto 39.1:

application_name	wait_event_type	wait_event	bloqueado_por	consulta
A	Client	ClientRead	{}	`UPDATE … tarifa_noche + 100 …` (esperando que el cliente envíe `COMMIT`)
B	Lock	transactionid	{1156}	`UPDATE … tarifa_noche + 1 …`

Detalle de los bloqueos

```
SELECT a.application_name, l.locktype, l.relation::regclass, l.mode,
l.granted
  FROM pg_locks l JOIN pg_stat_activity a ON a.pid = l.pid
 WHERE a.datname = current_database()
 ORDER BY a.application_name, l.granted DESC;
```

`granted = false` identifica el bloqueo que se está esperando. En 39.1, B tenía `RowExclusiveLock` sobre la tabla y `ExclusiveLock` de tipo `tuple` (concedidos) y esperaba un `ShareLock` de tipo `transactionid` (no concedido): está esperando que termine la transacción de A.

Herramientas disponibles

Herramienta	Qué muestra	Uso
`pg_stat_activity`	Cada conexión: estado, consulta actual, inicio de la transacción, evento de espera	Encontrar transacciones largas o "idle in transaction" que retienen bloqueos
`pg_blocking_pids(pid)`	Los procesos que bloquean a uno dado	Armar la cadena de bloqueos
`pg_locks`	Cada bloqueo: tipo, objeto, modo, concedido o no	Detalle de qué se está esperando
`pg_stat_database.deadlocks`	Cantidad acumulada de deadlocks por base	Monitoreo (pasó de 0 a 1 en el Punto 41)
Log del servidor	Mensaje `deadlock detected` con procesos, consultas y tupla	Análisis posterior del deadlock
`log_lock_waits = on`	Registra toda espera de bloqueo que supere `deadlock_timeout`	Detectar contención sin estar mirando en el momento (se activó en el Punto 47)

Herramienta	Qué muestra	Uso
`lock_timeout`	Cancela la sentencia que espera más de lo indicado	Evitar esperas indefinidas (se usó 15 s en las pruebas)
`idle_in_transaction_session_timeout`	Cierra sesiones que abren una transacción y no la terminan	Prevención
`pg_cancel_backend(pid)` / `pg_terminate_backend(pid)`	Cancela la consulta o cierra la conexión que bloquea	Resolver una emergencia

Equivalentes en MySQL/InnoDB: `SHOW ENGINE INNODB STATUS` (sección *LATEST DETECTED DEADLOCK*), `performance_schema.data_locks` y `data_lock_waits`, `information_schema.innodb_trx` y la variable `innodb_print_all_deadlocks` para enviar todos los deadlocks al log de errores.

Parte IX — Memoria, Servidor y Rendimiento

43. Configuración del servidor

La configuración de PostgreSQL está en `postgresql.conf` (en este entorno `/var/lib/pgtp/data/postgresql.conf`); los cambios hechos con `ALTER SYSTEM` se guardan aparte en `postgresql.auto.conf`, que tiene prioridad. Cada parámetro tiene un *contexto* que indica cómo se aplica: `postmaster` exige reiniciar, `sighup` se aplica con `SELECT pg_reload_conf()`, `user` se puede cambiar por sesión con `SET`. Aumentar `work_mem` de 4 a 64 MB hizo que un ordenamiento dejara de usar disco y bajara de 940 a 709 ms.

Parámetros relevantes (valores del entorno)

Área	Parámetro	Valor	Contexto	Función
Memoria / caché	`shared_buffers`	128 MB	reinicio	Caché de páginas propia de PostgreSQL (Punto 44)
	`effective_cache_size`	4 GB	sesión	No reserva memoria: le dice al optimizador cuánta caché total (propia + SO) hay; favorece índices cuando es grande
	`huge_pages`	try	reinicio	Páginas de memoria de 2 MB para `shared_buffers` grandes

Área	Parámetro	Valor	Contexto	Función
Ordenamiento, JOIN, tablas temporales	`work_mem`	4 MB	sesión	Memoria por operación de Sort o Hash antes de usar disco; una consulta puede usar varias veces este valor
	`hash_mem_multiplier`	2	sesión	Las tablas hash pueden usar 2 × `work_mem`
	`temp_buffers`	8 MB	sesión	Caché para tablas temporales de la sesión
	`temp_file_limit`	sin límite	superusuario	Tope de archivos temporales por proceso
Mantenimiento	`maintenance_work_mem`	64 MB	sesión	Memoria para `CREATE INDEX`, `VACUUM`, `ALTER TABLE ADD FOREIGN KEY`
	`autovacuum` / `autovacuum_vacuum_scale_factor`	on / 0,2	recarga	Limpieza automática cuando cambió el 20 % de una tabla
	`default_statistics_target`	100	sesión	Tamaño de la muestra de `ANALYZE` (Punto 31)
Conexiones	`max_connections`	100	reinicio	Cada conexión es un proceso del sistema operativo
	`superuser_reserved_connections`	3	reinicio	Reservadas para administrar aunque el resto esté ocupado
Almacenamiento y WAL	`wal_buffers`	4 MB	reinicio	Buffer del WAL antes de escribirlo
	`max_wal_size`	1 GB	recarga	WAL acumulado antes de forzar un checkpoint (Punto 20)
	`checkpoint_timeout` / `checkpoint_completion_target`	5 min / 0,9	recarga	Frecuencia y suavizado de los checkpoints
	`synchronous_commit`, `fsync`	on, on	sesión / recarga	Durabilidad: esperar al disco en cada `COMMIT`
Optimizador	`random_page_cost` / `seq_page_cost`	4 / 1	sesión	Costo relativo de lecturas salteadas y secuenciales (Punto 32)
	`effective_io_concurrency`	1	sesión	Lecturas anticipadas en Bitmap Heap Scan; en SSD conviene 200
	`join_collapse_limit` / `from_collapse_limit`	8 / 8	sesión	Hasta cuántas tablas reordena el optimizador (Punto 33)
	`max_parallel_workers_per_gather`	2 (0 en las mediciones)	sesión	Procesos paralelos por consulta
Diagnóstico	`track_io_timing`	on (activado)	superusuario	Tiempos de I/O en `EXPLAIN` (Punto 23)
	`log_min_duration_statement`	−1 ⟶ 500 ms en el Punto 47	superusuario	Registro de consultas lentas

Área	Parámetro	Valor	Contexto	Función
	`deadlock_timeout`, `lock_timeout`, `statement_timeout`	1 s, 0, 0	—	Esperas y cancelaciones (Parte VIII)

Efecto medido de los parámetros de memoria

Parámetro	Operación	Valor bajo	Valor alto
`work_mem`	Ordenar las ~600 mil reservas de 2024 por tarifa	4 MB: *external merge* con 14,5 MB en disco, **940 ms**	64 MB: *quicksort* en 47,9 MB de memoria, **709 ms**
`work_mem`	Contar reservas por huésped (7,5 M filas, ~1 M grupos)	4 MB: GroupAggregate sobre el índice, **2.509 ms**	256 MB: HashAggregate de 115 MB en memoria, **5.792 ms**
`maintenance_work_mem`	`CREATE INDEX` sobre `fecha_fin`	16 MB: **4,4 s**	512 MB: **2,8 s**

El segundo caso muestra que más memoria no siempre es mejor: con 256 MB el optimizador cambió a una agregación por hash que entra en memoria, pero construir una tabla hash de un millón de grupos fue más lento que recorrer el índice ya ordenado. Como `work_mem` se multiplica por operación y por conexión (100 conexiones × varias operaciones), en producción se deja un valor moderado global y se sube sólo en sesiones de reportes (`SET work_mem = '256MB'`).

Configuración recomendada para este servidor (7,8 GB de RAM)

Parámetro	Por defecto	Recomendado	Motivo
`shared_buffers`	128 MB	2 GB (≈25 % de la RAM)	Mantener en caché índices y reservas recientes
`effective_cache_size`	4 GB	6 GB	Reflejar la caché real del SO
`work_mem`	4 MB	16 MB global, más en reportes	Evitar ordenamientos en disco sin riesgo de agotar memoria

Parámetro	Por defecto	Recomendado	Motivo
`maintenance_work_mem`	64 MB	512 MB	Índices y `VACUUM` más rápidos
`max_wal_size`	1 GB	4 GB	Menos checkpoints en cargas masivas
`random_page_cost`	4	1,1 si el disco es SSD	Modelo de costos realista

44. InnoDB Buffer Pool y su equivalente en PostgreSQL

El equivalente del *Buffer Pool* de InnoDB en PostgreSQL es **`shared_buffers`**, complementado por la caché de páginas del sistema operativo. Al pasarlo de 128 MB a 2 GB, 60.000 lecturas dispersas por PK bajaron de **343 ms a 155 ms** en caliente, porque todas las páginas necesarias quedaron en memoria. En cambio, un recorrido completo de la tabla no mejoró: PostgreSQL protege la caché de los recorridos masivos.

Cómo funciona `shared_buffers`

- Es un área de memoria compartida dividida en páginas de 8 KB (`shared_buffers = 128MB` son 16.384 páginas). Toda lectura y escritura de tablas e índices pasa por ahí.
- Cada página en caché tiene un contador de uso (`usagecount`, de 0 a 5). Para hacer lugar, el algoritmo *clock-sweep* recorre las páginas, baja el contador y desaloja la primera que llega a 0: las páginas usadas seguido permanecen.
- Las páginas modificadas (*dirty*) se escriben al disco en segundo plano (*background writer* y *checkpointer*), no en el momento del `COMMIT` (en ese momento sólo se asegura el WAL).
- Debajo está la caché del sistema operativo: una página que no está en `shared_buffers` puede estar en la RAM del SO, por eso `shared read` no siempre significa disco (Punto 23). De ahí la recomendación habitual de ~25 % de la RAM para `shared_buffers`, frente al 50–75 % del Buffer Pool de InnoDB, que no depende de la caché del SO (usa `O_DIRECT`).

Experimento (`71_servidor_resto.py`): 128 MB frente a 2 GB

En cada caso se reinició el servidor, se vació la caché del SO y se ejecutó cada consulta 3 veces.

Consulta	`shared_buffers`	1ª (fría)	2ª	3ª	Páginas leídas en la 3ª
Recorrido completo (`sum` de 7,5 M filas)	128 MB	1.857 ms	1.448 ms	1.453 ms	99.048 de 99.112
	2 GB	1.970 ms	1.683 ms	1.470 ms	99.048 de 99.112
60.000 reservas dispersas por PK	128 MB	480 ms	343 ms	364 ms	**77.526**
	2 GB	678 ms	190 ms	**156 ms**	**0**

Contenido de la caché después del experimento (`pg_buffercache`)

`shared_buffers`	`reserva`	`reserva_pkey`	Páginas libres
128 MB	74 MB (58 % de la caché)	53 MB (42 %)	0
2 GB	445 MB (22 %)	161 MB, el índice completo (8 %)	184.332

Interpretación

- **Con 128 MB la caché no alcanza** para las páginas que tocan 60.000 lecturas dispersas (unos 600 MB entre tabla e índice): cada repetición desaloja páginas que la siguiente vuelve a necesitar, y siempre hay 77.000 lecturas. Con 2 GB todo entra y la tercera ejecución no lee nada: **2,3 veces más rápida**.
- **El recorrido completo nunca se quedó en caché**, ni con 2 GB: todas las repeticiones leyeron 99.048 páginas. Es deliberado: para recorridos de tablas más grandes que un cuarto de `shared_buffers`, PostgreSQL usa un **anillo de buffers** de 256 kB que reutiliza siempre las mismas páginas. Así, un reporte que recorre toda la tabla no expulsa de la caché los datos que usa la operación diaria. Esas lecturas se sirven desde la caché del SO.
- La tasa de aciertos global (`blks_hit / (blks_hit + blks_read)`) fue 95,9 % en ambos casos: es un promedio que oculta el comportamiento por consulta.

Parámetros relacionados

PostgreSQL	MySQL / InnoDB	Función
`shared_buffers`	`innodb_buffer_pool_size`	Tamaño de la caché
(una sola área)	`innodb_buffer_pool_instances`	Divisiones para reducir contención
`effective_cache_size`	—	Estimación de caché total para el optimizador
`bgwriter_*`, `checkpoint_*`	`innodb_io_capacity`, `innodb_max_dirty_pages_pct`	Escritura de páginas modificadas
Anillo de buffers para recorridos	Inserción en el punto medio de la lista LRU (`innodb_old_blocks_pct`)	Proteger la caché de recorridos masivos
`pg_buffercache`, `pg_stat_database.blks_hit`	`SHOW ENGINE INNODB STATUS`, `Innodb_buffer_pool_read_requests`	Observación

45. CPU, disco y red

Una consulta está limitada por el recurso en el que pasa la mayor parte de su tiempo. Se diseñaron tres experimentos, uno por recurso, y se identificó el cuello de botella con métricas concretas: tiempo de I/O de `EXPLAIN` (con `track_io_timing`), CPU consumida por el proceso servidor (de `/proc/<pid>/stat`) y diferencia entre el tiempo en el servidor y el tiempo hasta que el cliente recibe todo.

Disco

`SELECT count(*) FROM reserva WHERE observaciones IS NOT NULL` obliga a leer las 99.112 páginas de la tabla.

Estado	Tiempo	Páginas leídas	Tiempo esperando I/O
Fría (reinicio + caché del SO vaciada)	930 ms	99.112	**261 ms (28 %)**

Estado	Tiempo	Páginas leídas	Tiempo esperando I/O
Segunda ejecución	536 ms	99.080	143 ms (27 %)

Una parte importante del tiempo es espera de disco. Que la segunda ejecución también "lea" casi todo se explica en el Punto 44: los recorridos completos de tablas grandes usan un anillo de buffers pequeño y no se quedan en `shared_buffers`; esas lecturas llegan desde la caché del SO, más rápidas que el disco pero no gratis.

CPU

Mediana de las tarifas y cantidad de combinaciones distintas `md5(huésped + fecha)` de las reservas de 2024, con todo en caché:

Métrica	Valor
Tiempo total	1,33 s
CPU usada por el proceso servidor	**1,32 s = 99 % de un núcleo**
Páginas	11.153, todas en caché (`shared hit`)
I/O de tablas	ninguna; sólo archivos temporales del ordenamiento (42 ms)

El proceso estuvo ocupado el 99 % del tiempo: la consulta está limitada por CPU (calcular 600 mil hashes MD5 y ordenar). Más memoria o un disco más rápido no la acelerarían; sí lo haría una CPU más rápida, el paralelismo, o no calcular `md5` sobre la columna.

Red y transporte

Mismo filtro (reservas de 2019 y 2020, 1.207.044 filas), distinto volumen enviado al cliente:

Qué se pide	Tiempo en el servidor	Tiempo hasta que el cliente tiene todo	Datos transferidos (aprox.)
`count(*)` (1 fila)	156 ms	48 ms	0 MB
2 columnas	343 ms	646 ms	20,5 MB
`SELECT *`	228 ms	**3.842 ms**	119,7 MB

Con `SELECT *` el servidor termina en 228 ms, pero el cliente recibe los datos 17 veces más tarde: el tiempo se va en serializar, enviar y convertir 120 MB. Es una consulta limitada por la transferencia, aun sin red real (cliente y servidor en la misma máquina). Con red de por medio la diferencia sería mayor.

Viajes de ida y vuelta: pedir 1.000 reservas con 1.000 consultas tardó 290 ms; con una sola consulta `WHERE reserva_id = ANY(...)`, 9 ms. Cada consulta paga la latencia del viaje completo, que en una red real es de 0,5 a 50 ms.

Cómo identificar el recurso limitante

Indicador	Disco	CPU	Red / transporte
`I/O Timings` en `EXPLAIN`	Alto respecto del total	Bajo o nulo	Bajo
`shared read`	Alto	Bajo (`hit`)	Indiferente
CPU del proceso servidor	Baja (espera)	Cercana al 100 %	Baja
Tiempo cliente vs `Execution Time`	Similares	Similares	Cliente mucho mayor
Remedio	Más RAM, índices, menos páginas	Menos cálculo por fila, paralelismo	Menos columnas y filas, paginación, menos viajes

46. Connection Pool

Abrir una conexión a PostgreSQL cuesta unos **2,8 ms**, 25 veces más que la consulta típica del sistema (una reserva por número, 0,11 ms). Con 8 clientes concurrentes, reutilizar conexiones permitió **26.682 consultas por segundo** contra **317** abriendo una conexión por consulta: 84 veces más.

Por qué conectarse es caro

PostgreSQL crea un **proceso del sistema operativo por conexión**. Abrir una implica: conexión de red o socket, autenticación, `fork()` de un proceso nuevo, inicialización de su memoria y carga de la caché del catálogo. Al cerrarla, el proceso termina y esa caché se pierde.

Medición	Mediana
Abrir y cerrar conexión por socket Unix	2,78 ms
Abrir y cerrar conexión por TCP (localhost)	3,20 ms
Consulta de una reserva por PK con conexión abierta	0,11 ms

Experimento con `pgbench` (10 s por prueba)

Script: `SELECT * FROM hotel.reserva WHERE reserva_id = :id` con un id aleatorio. La opción `-C` abre una conexión nueva para cada transacción.

Clientes	Conexión persistente: TPS	Latencia	Conexión por transacción (`-C`): TPS	Latencia
1	9.171	0,11 ms	241	4,1 ms
8	26.682	0,30 ms	317	25,2 ms
32	22.693	1,41 ms	328	97,6 ms

Pool en la aplicación (`psycopg_pool`)

2.000 pedidos servidos por 8 hilos de una aplicación Python:

Estrategia	Tiempo total	Pedidos por segundo
Una conexión nueva por pedido	6,76 s	296
Pool de 8 conexiones reutilizadas	0,63 s	3.155

Análisis bajo concurrencia

- **Sin pool, el costo de conexión domina**: el rendimiento queda en ~300 TPS sin importar cuántos clientes haya, porque el servidor pasa casi todo el tiempo creando y destruyendo procesos.
- **Con conexiones persistentes, más clientes ayudan hasta saturar la CPU**: de 1 a 8 clientes el rendimiento casi se triplicó; de 8 a 32 bajó (de 26.682 a 22.693) y la latencia se multiplicó por 4,7. Con 2 núcleos, 32 procesos activos compiten por la CPU y se suman cambios de contexto.

- **Tamaño del pool**: conviene un pool chico respecto de los usuarios (del orden de 2 a 4 conexiones por núcleo) y que los pedidos esperen su turno en el pool. Más conexiones no aumentan el rendimiento y cada una consume memoria (`work_mem` por operación, más su caché).
- **`max_connections = 100`** es un límite duro: una aplicación sin pool con 200 usuarios simultáneos recibiría errores *too many clients*.

Opciones

Dónde	Herramienta	Características
En la aplicación	`psycopg_pool`, HikariCP (Java), el pool de `node-postgres`, Prisma	Cada instancia de la aplicación mantiene sus conexiones
Entre aplicación y base	**PgBouncer**, Pgpool-II	Un pool compartido por todas las instancias; modo *transaction* permite miles de clientes con decenas de conexiones reales

47. Slow Query Log y diagnóstico

Se activaron tres mecanismos complementarios y se ejecutó una carga mixta de 515 consultas. El log registró las **6 ejecuciones que superaron 500 ms**, `auto_explain` guardó el plan de cada una, y `pg_stat_statements` mostró que dos consultas (`GROUP BY huesped_id` y `extract(year …)`) concentraban el 94 % del tiempo total.

Configuración

```
ALTER SYSTEM SET shared_preload_libraries = pg_stat_statements, auto_explain;
-- requiere reinicio
-- después del reinicio:
ALTER SYSTEM SET log_min_duration_statement = '500ms';   -- registrar
consultas lentas
ALTER SYSTEM SET auto_explain.log_min_duration = '500ms'; -- y su plan
ALTER SYSTEM SET auto_explain.log_analyze = on;           -- con filas y
tiempos reales
ALTER SYSTEM SET log_lock_waits = on;                     -- esperas de
bloqueo > deadlock_timeout
SELECT pg_reload_conf();
CREATE EXTENSION pg_stat_statements;
```

Dificultad encontrada: el primer intento escribió `shared_preload_libraries = 'pg_stat_statements,auto_explain'` entre comillas, que PostgreSQL interpreta como **una** librería con ese nombre, y el servidor no volvió a arrancar (*could not access file "pg_stat_statements,auto_explain"*). Se corrigió editando `postgresql.auto.conf`. Además, los parámetros `auto_explain.*` no existen hasta que la librería está cargada, por eso se configuran después del reinicio.

Lo que registró el log

```
LOG:  duration: 793.314 ms  statement: SELECT count(*) FROM reserva WHERE
extract(year FROM fecha_inicio) = 2023
LOG:  duration: 1737.456 ms  statement: SELECT huesped_id, count(*) FROM
reserva GROUP BY huesped_id ORDER BY 2 DESC LIMIT 10
```

Y `auto_explain` agregó el plan real de la primera:

```
LOG:  duration: 768.470 ms  plan:
  Query Text: SELECT count(*) FROM reserva WHERE extract(year FROM
fecha_inicio) = 2023
  Aggregate  (actual time=768.445..768.447 rows=1 loops=1)
    ->  Index Only Scan using ix_reserva_fecha_inicio on reserva (rows=37669)
(actual rows=603475)
          Filter: (EXTRACT(year FROM fecha_inicio) = '2023'::numeric)
          Rows Removed by Filter: 6932105
```

`pg_stat_statements`: dónde se va el tiempo

Agrupa las consultas normalizadas (las constantes se reemplazan por `$1`, `$2`) y acumula estadísticas.

Consulta normalizada	Llamadas	Tiempo total	Media	Páginas en caché / leídas
`SELECT huesped_id, count(*) FROM reserva GROUP BY huesped_id ORDER BY 2 DESC LIMIT 10`	3	4.650 ms	1.550 ms	7.573.135 / 28.778
`SELECT count(*) FROM reserva WHERE extract($1 FROM fecha_inicio) = $2`	3	2.407 ms	802 ms	33.930 / 6.606

Consulta normalizada	Llamadas	Tiempo total	Media	Páginas en caché / leídas
`SELECT count(*) FROM huesped_email WHERE email LIKE $1`	3	475 ms	158 ms	20.034 / 10.017
`SELECT * FROM reserva WHERE reserva_id = $1`	496	14 ms	0,03 ms	945 / 1.039
`SELECT count(*) FROM reserva WHERE fecha_inicio >= $1 AND fecha_inicio < $2`	3	4 ms	1,3 ms	83 / 1

La consulta por PK se ejecutó 496 veces pero suma sólo 14 ms: no hay que optimizarla. El log de consultas lentas no la muestra nunca (cada ejecución es rápida) y `pg_stat_statements` confirma que su costo acumulado es irrelevante. Si una consulta de 5 ms se ejecutara un millón de veces, sólo `pg_stat_statements` la haría visible.

Análisis y mejoras propuestas

Consulta lenta	Diagnóstico con `EXPLAIN`	Mejora
`extract(year …) = 2023`	El filtro con función recorre todo el índice y descarta 6,9 M de filas; estimó 37.669 filas y fueron 603.475	Reescribir como rango (Punto 30, optimización O1): 802 ⟶ ~25 ms
`GROUP BY huesped_id … LIMIT 10`	Agrupa 7,5 M de filas en ~1 M de grupos para quedarse con 10	Si es un reporte frecuente: vista materializada de reservas por huésped (Punto 52) o contador mantenido por la aplicación
`email LIKE '%rojas%'`	Comodín inicial: no SARGable	Índice GIN con `pg_trgm` (Punto 13)

Flujo de diagnóstico

1. `pg_stat_statements` ordenado por tiempo total para saber **qué** optimizar.
2. Log de consultas lentas y `auto_explain` para obtener los parámetros y el plan **real** de las ejecuciones lentas.
3. `EXPLAIN (ANALYZE, BUFFERS)` manual para probar hipótesis (Parte VII).
4. Repetir la medición con `pg_stat_statements_reset()` para validar.

48. Mantenimiento

Por MVCC, cada `UPDATE` y `DELETE` deja versiones muertas en tablas e índices, y el espacio no se recupera solo. En este TP el `UPDATE` que agregó `hotel_id` a las 7,5 millones de reservas duplicó la tabla (719 ⟶ 1.494 MB) e infló sus índices hasta 2.246 MB; después de `VACUUM FULL` quedaron en 774 MB de tabla y 1.262 MB de índices. A mayor volumen, el mantenimiento preventivo deja de ser opcional.

Experimento controlado (1 millón de reservas, `pgstattuple`)

Momento	Tabla	Tuplas muertas	Espacio libre	Índice sobre `tarifa_noche`	Densidad de hojas del índice
Recién creada	108,5 MB	0	0,5 %	21 MB	90,2 %
Tras `UPDATE` de 1/3 de las filas (5,2 s)	**144,5 MB**	**333.333 (22,9 %)**	0,4 %	**42 MB**	61,9 %
Tras `VACUUM` (0,4 s)	144,5 MB	0	**24,2 %**	42 MB	46,6 %
Tras `REINDEX` del índice (0,7 s)	144,5 MB	0	24,2 %	**21 MB**	90,2 %
Tras `VACUUM FULL` (2,6 s)	**108,1 MB**	0	0,5 %	21 MB	90,2 %

Lectura:

- **El `UPDATE` no modifica en el lugar**: escribe versiones nuevas. La tabla creció 33 % y el índice se duplicó, aunque la cantidad de filas vivas no cambió.
- **`VACUUM` no achica la tabla**: marca el espacio de las tuplas muertas como reutilizable (24,2 % libre) para futuros `INSERT` y `UPDATE`. No bloquea lecturas ni escrituras.
- **El índice no se compacta con `VACUUM`**: quita las entradas muertas pero las páginas quedan medio vacías (densidad 46,6 %). `REINDEX` lo reconstruye compacto.
- **`VACUUM FULL` devuelve el espacio al sistema** reescribiendo tabla e índices, pero toma un bloqueo `ACCESS EXCLUSIVE`: nadie puede leer la tabla mientras dura. Sobre la tabla real de reservas tardó **2 min 18 s** (Parte VII).

Tareas preventivas

Tarea	Para qué	Cuándo en este sistema
Autovacuum (`VACUUM` + `ANALYZE` automáticos)	Reutilizar espacio, actualizar estadísticas y el mapa de visibilidad	Siempre activo. Con los valores por defecto se dispara en `reserva` recién al acumular 50 + 20 % × 7,5 M = **1.506.817 tuplas muertas**: conviene bajarle el umbral a la tabla, por ejemplo `ALTER TABLE reserva SET (autovacuum_vacuum_scale_factor = 0.02)`
`ANALYZE` manual	Estadísticas al día después de cargas masivas	Después de cada importación (Punto 31)
`VACUUM` **manual**	Mantener al día el mapa de visibilidad para Index Only Scan	Después de cargas y migraciones (sin él, el índice de cobertura no se usó: Punto 37)
`REINDEX CONCURRENTLY`	Compactar índices inflados sin bloquear	Cuando `pgstatindex` muestre densidad baja, por ejemplo tras actualizaciones masivas
`VACUUM FULL` o **`pg_repack`**	Recuperar espacio de una tabla inflada	Sólo tras operaciones excepcionales, en una ventana de mantenimiento
Revisar índices sin uso	Quitar costo de escritura	Periódicamente con `pg_stat_user_indexes.idx_scan = 0` (Punto 18)
Particiones	Archivar o borrar años viejos sin `DELETE` masivo	Anualmente (Punto 50)
Vigilar el *wraparound*	Los ids de transacción son de 32 bits: `VACUUM` debe "congelar" las filas viejas antes de 2.000 millones de transacciones	`age(datfrozenxid)` era 11.016: lejos del límite
Migraciones por lotes	Evitar duplicar la tabla con un `UPDATE` gigante	Actualizar de a 50.000 filas con `VACUUM` entre lotes (lección del `UPDATE` de `hotel_id`)

Estado de `reserva` al final del TP

`pg_stat_user_tables` muestra que el autovacuum procesó `reserva` una vez y la analizó una vez, después del `UPDATE` masivo (7,5 M de tuplas muertas), y que al final no quedaban tuplas muertas.

Parte X — Escalabilidad, Particionamiento y Estructuras Avanzadas

49. Escalabilidad vertical y horizontal

Para esta cadena de hoteles la escalabilidad vertical alcanza con holgura: 7,5 millones de reservas ocupan 2,2 GB y entran enteras en la memoria de un servidor modesto. La horizontal sólo se justificaría por disponibilidad o por un volumen de consultas que un servidor no pueda atender.

Aspecto	Vertical (*scale up*)	Horizontal (*scale out*)
Qué es	Un servidor más grande: más RAM, más núcleos, discos más rápidos	Más servidores que se reparten la carga o los datos
Qué problema resuelve	Consultas que leen de disco (más RAM = más datos en caché), ordenamientos y JOIN que no entran en memoria, más conexiones simultáneas	Más lecturas de las que un servidor atiende (réplicas); volumen de datos o de escrituras mayor al de un servidor (*sharding*); tolerancia a fallas
Cambios en la aplicación	Ninguno	Réplicas: enviar lecturas a otro servidor y tolerar datos con segundos de atraso. *Sharding*: elegir una clave de distribución y evitar consultas que crucen fragmentos
Integridad y transacciones	Intactas: un solo motor garantiza todo	Las réplicas son de sólo lectura. Con *sharding* las FK, los `UNIQUE` y el `EXCLUDE` entre fragmentos dejan de ser garantizables por el motor, y una transacción que toca dos fragmentos requiere *commit* en dos fases

Aspecto	Vertical (*scale up*)	Horizontal (*scale out*)
Límite	El servidor más grande disponible y su costo, que crece más rápido que la capacidad; sigue siendo un único punto de falla	Complejidad operativa: replicación, balanceo, rebalanceo de fragmentos, consistencia eventual

Opciones en PostgreSQL

- **Réplicas de lectura** con replicación física por *streaming* del WAL: un primario recibe todas las escrituras y una o más réplicas atienden reportes y consultas de disponibilidad. Sirven para separar los reportes históricos pesados (Parte V) de la operación diaria.
- **Particionamiento** (Punto 50): no escala horizontalmente por sí mismo (todo sigue en un servidor), pero divide la tabla en partes que se mantienen y se recorren por separado. Es el paso previo natural al *sharding*.
- ***Sharding*** con extensiones como Citus o con *foreign data wrappers*: cada fragmento vive en un servidor distinto.

Qué convendría a este sistema

La clave natural de distribución sería `hotel_id`: casi todas las operaciones (reservar, consultar disponibilidad, ocupación) ocurren dentro de un hotel, y la regla de no superposición es por habitación, que pertenece a un único hotel. Con `hotel_id` en `reserva` (desnormalización del Punto 5), el `EXCLUDE` seguiría siendo local a cada fragmento. Lo que se complicaría son los reportes globales (rankings de la cadena, historial de un huésped que visitó varios hoteles), que pasarían a consultar todos los fragmentos.

Con los volúmenes medidos, el orden razonable es: primero índices y consultas correctas (Partes III y VII), después más memoria, después réplicas de lectura para reportes, y *sharding* sólo si la cadena creciera en órdenes de magnitud.

50. Particionamiento

Se particionó `reserva` por año de `fecha_inicio` (13 particiones + una por defecto). El beneficio claro está en el **mantenimiento**: eliminar un año completo tardó 14 ms con `DROP` de la partición contra 0,7 s con `DELETE` (que además deja 603 mil tuplas muertas para el

`VACUUM`). En consultas, la tabla particionada ganó en los reportes de uno o más años y perdió en los de un mes. Y apareció un costo de diseño importante: **PostgreSQL 16 no admite la restricción `EXCLUDE` en tablas particionadas**.

Estrategias de particionamiento

Estrategia	Cómo reparte	Uso en este dominio
Por rango	Intervalos de una columna (fechas, números)	Reservas por año o mes de `fecha_inicio`: los reportes y el archivado son por período
Por lista	Valores explícitos	Por provincia o por estado (activas / históricas)
Por hash	`hash(columna) mod N`	Por `hotel_id` o `huesped_id` para repartir uniformemente; no ayuda a consultas por rango
Compuesta	Subparticiones (rango + hash)	Por año y, dentro de cada año, por hotel

Implementación (`80_particiones.py`)

```
CREATE TABLE reserva_part (LIKE reserva INCLUDING DEFAULTS) PARTITION BY
RANGE (fecha_inicio);
CREATE TABLE reserva_p2015 PARTITION OF reserva_part FOR VALUES FROM ('2015-
01-01') TO ('2016-01-01');
-- … una por año hasta 2027
CREATE TABLE reserva_pdef PARTITION OF reserva_part DEFAULT;
ALTER TABLE reserva_part ADD PRIMARY KEY (reserva_id, fecha_inicio);
INSERT INTO reserva_part SELECT * FROM reserva;          -- 7.535.580 filas
en 19 s
CREATE INDEX ON reserva_part (fecha_inicio);              -- se crea en cada
partición
CREATE INDEX ON reserva_part (huesped_id);
```

Limitaciones encontradas al crearla:

Intento	Resultado
`EXCLUDE USING gist (habitacion_id WITH =, daterange(...) WITH &&)`	**ERROR**: *exclusion constraints are not supported on partitioned tables*
`PRIMARY KEY (reserva_id)`	**ERROR**: la clave única debe incluir la columna de partición

Intento	Resultado
`PRIMARY KEY (reserva_id, fecha_inicio)`	Aceptada

La regla RN1 sólo podría garantizarse con un `EXCLUDE` **por partición**, y eso deja un hueco: una reserva del 30/12/2025 al 3/1/2026 (partición 2025) y otra del 1/1/2026 (partición 2026) en la misma habitación no se verifican entre sí. Habría que agregar un trigger o elegir otra clave (por ejemplo, partición por hash de `habitacion_id`, donde toda la historia de una habitación queda en la misma partición). La unicidad de `reserva_id` tampoco está garantizada globalmente: depende de la secuencia.

Tamaño

Cada partición anual ocupa 62 MB de datos y entre 34 y 47 MB de índices; la de 2027 (medio año), 29 MB. La tabla particionada completa ocupa 1.262 MB; la original, 2.037 MB, pero tiene más índices (cobertura, GiST, `hotel_id`), así que la diferencia no es mérito del particionamiento.

Consultas: particionada frente a sin particionar

Consulta	Sin particionar	Particionada	Plan en la particionada
Un mes, columnas no indexadas	**17,3 ms**	47,2 ms	Index Scan en la partición 2024
Un trimestre	**39,1 ms**	93,2 ms	Index Scan en la partición 2024
Un año completo	103,5 ms	**87,6 ms**	Seq Scan de **una** partición
Dos años	197,3 ms	**154,3 ms**	Seq Scan de dos particiones
Por huésped (no usa la clave de partición)	0,6 ms	0,5 ms	Index Scan en las 14 particiones
Por `reserva_id` (no usa la clave)	**0,6 ms**	1,0 ms	14 búsquedas de índice
Toda la tabla	340 ms	346 ms	Seq Scan de todas

Lectura de los resultados:

- **Cuando la consulta pide una partición entera** (un año), recorrerla secuencialmente es más barato que ir por índice sobre la tabla grande: gana la particionada.
- **Cuando pide una porción chica** (un mes), las dos usan índice y gana la tabla sin particionar: el optimizador eligió para la partición un Index Scan común que visitó 39.343 páginas, frente a un Bitmap Heap Scan de 3.112 en la tabla grande. Particionar no reemplaza a un buen índice.
- **Si la consulta no filtra por la clave de partición**, se consultan todas las particiones: es igual o un poco más lenta.

Mantenimiento

Operación: eliminar las reservas de 2015 (603.241)	Tiempo	Efecto
`DELETE FROM reserva WHERE fecha_inicio < '2016-01-01'`	0,7 s	Deja 603 mil tuplas muertas; los índices conservan las entradas hasta el `VACUUM`
`ALTER TABLE reserva_part DETACH PARTITION reserva_p2015; DROP TABLE reserva_p2015`	**14 ms**	Libera el espacio al instante, sin tuplas muertas

Además, `VACUUM`, `ANALYZE` y `REINDEX` se pueden hacer partición por partición, y las particiones históricas, que no cambian, casi no requieren mantenimiento. Para una cadena de hoteles que guarda años de historial, este es el argumento principal a favor de particionar.

51. Partition pruning

Cuando la condición compara la clave de partición con valores conocidos, el optimizador descarta las particiones que no pueden contener filas: un rango de un mes visitó **1 de las 14 particiones**. Si la clave queda dentro de una función (`extract(year FROM fecha_inicio)`) o se filtra por otra columna, se visitan las 14.

Casos probados sobre `reserva_part`

Condición	Particiones visitadas	Tiempo	¿Pruning?
`fecha_inicio >= '2024-07-01' AND fecha_inicio < '2024-08-01'`	1 (`reserva_p2024`)	6,8 ms	Sí, al planificar

Condición	Particiones visitadas	Tiempo	¿Pruning?
`fecha_inicio = '2026-01-15'`	1 (`reserva_p2026`)	0,3 ms	Sí, al planificar
`fecha_inicio >= CURRENT_DATE - 30`	3 en el plan; 11 subplanes descartados al ejecutar	90,5 ms	Sí, **al ejecutar** (`CURRENT_DATE` no es constante al planificar)
Sentencia preparada `fecha_inicio >= $1 AND fecha_inicio < $2` con plan genérico	1 (`Subplans Removed: 13`)	13,3 ms	Sí, **al ejecutar**
`extract(year FROM fecha_inicio) = 2024`	14	854 ms	**No**: función sobre la clave
`fecha_fin >= '2024-07-01' AND fecha_fin < '2024-08-01'`	14	589 ms	**No**: no es la clave
Rango de julio con `enable_partition_pruning = off`	14	15,9 ms (contra 9,8 ms con pruning)	Desactivado

Plan con pruning

```
Aggregate (actual time=7.809..7.811 rows=1)
  ->  Index Only Scan using reserva_p2024_fecha_inicio_idx on reserva_p2024
reserva_part
        Index Cond: ((fecha_inicio >= '2024-07-01') AND (fecha_inicio <
'2024-08-01'))
Execution Time: 7.840 ms
```

El plan nombra una sola partición: las otras 13 ni siquiera aparecen. Con una sentencia preparada, el plan las incluye, pero al ejecutarse con los parámetros concretos informa `Subplans Removed: 13`.

Dos momentos de pruning

1. **Al planificar**: cuando los valores son constantes en el texto de la consulta. Es el más eficiente porque el plan ni siquiera contempla las otras particiones.

2. **Al ejecutar**: cuando los valores llegan como parámetros (sentencias preparadas, `CURRENT_DATE`, valores que vienen de otra tabla en un JOIN). El plan incluye todas las particiones y descarta las que no corresponden al iniciar la ejecución.

Relación con la SARGabilidad

Las reglas son las mismas que para usar un índice (Punto 30): la clave tiene que aparecer sola de un lado de la comparación. `extract(year FROM fecha_inicio) = 2024` no permite pruning y recorre los 7,5 millones de filas (854 ms); el rango equivalente visita una sola partición. El particionamiento sólo rinde si las consultas de la aplicación filtran por la clave de partición en forma SARGable.

52. Vistas y materialización

Una vista simplifica la escritura de consultas pero no las acelera: el motor la reemplaza por su definición en cada ejecución. Una **vista materializada** guarda el resultado: el ranking de hoteles por ingresos de 2025 bajó de **389 ms a 2,3 ms** (170 veces) leyendo una tabla de 18.450 filas en lugar de 600 mil reservas, a cambio de quedar desactualizado hasta el próximo `REFRESH` (6 segundos).

Vista común: `v_reserva_detalle`

```
CREATE VIEW v_reserva_detalle AS
SELECT r.reserva_id, r.fecha_inicio, r.fecha_fin, r.fecha_fin -
r.fecha_inicio AS noches, r.estado,
       r.tarifa_noche, r.tarifa_noche * (r.fecha_fin - r.fecha_inicio) AS
importe,
       hb.habitacion_id, hb.numero, th.nombre AS tipo, h.hotel_id, h.nombre
AS hotel, h.categoria,
       l.nombre AS localidad, p.nombre AS provincia, hu.huesped_id,
hu.apellido, hu.nombre
  FROM reserva r
  JOIN habitacion hb      ON hb.habitacion_id = r.habitacion_id
  JOIN tipo_habitacion th ON th.tipo_habitacion_id = hb.tipo_habitacion_id
  JOIN hotel h            ON h.hotel_id = hb.hotel_id
  JOIN domicilio d        ON d.domicilio_id = h.domicilio_id
  JOIN localidad l        ON l.localidad_id = d.localidad_id
  JOIN provincia p        ON p.provincia_id = l.provincia_id
  JOIN huesped hu         ON hu.huesped_id = r.huesped_id;
```

Con la vista, el historial de una reserva con hotel, provincia y huésped se escribe `SELECT hotel, apellido, fecha_inicio FROM v_reserva_detalle WHERE reserva_id = 4000000`: 1,5 ms, 23 páginas. El optimizador combina la vista con la consulta y aplica el

filtro por `reserva_id` antes de los JOIN. Pero ejecuta las 8 uniones aunque la consulta sólo use 3 columnas: PostgreSQL no elimina `INNER JOIN` que no aportan columnas (sólo elimina `LEFT JOIN` a tablas con clave única). Para consultas que agregan muchas filas, una vista "ancha" puede agregar trabajo innecesario.

Usos de una vista común: ocultar la complejidad de los JOIN, dar una interfaz estable a la aplicación aunque cambie el esquema, y limitar qué columnas ve cada usuario (permisos sobre la vista y no sobre las tablas).

Vista materializada: `mv_ocupacion_mensual`

```
CREATE MATERIALIZED VIEW mv_ocupacion_mensual AS
SELECT hb.hotel_id, date_trunc('month', r.fecha_inicio)::date AS mes,
       count(*) AS reservas,
       count(*) FILTER (WHERE r.estado = 'X') AS canceladas,
       sum(r.fecha_fin - r.fecha_inicio) FILTER (WHERE r.estado IN ('C','F'))
AS noches,
       sum(r.tarifa_noche * (r.fecha_fin - r.fecha_inicio)) FILTER (WHERE
r.estado IN ('C','F')) AS ingresos
  FROM reserva r JOIN habitacion hb ON hb.habitacion_id = r.habitacion_id
 GROUP BY 1, 2;
CREATE UNIQUE INDEX ON mv_ocupacion_mensual (hotel_id, mes);   -- necesario
para REFRESH CONCURRENTLY
```

Creación: 6,3 s; 18.450 filas (123 hoteles × 150 meses); 1,5 MB.

Costos de lectura y de actualización

Operación	Sobre las tablas	Sobre la vista materializada
Top 10 hoteles por ingresos 2025	389 ms, 11.198 páginas	**2,3 ms, 195 páginas**
¿Mismo resultado?	—	Sí (verificado)
Reflejar una reserva nueva	Inmediato	**No la ve** hasta el `REFRESH`
`REFRESH MATERIALIZED VIEW`	—	6,2 s; bloquea las lecturas de la vista mientras corre
`REFRESH MATERIALIZED VIEW CONCURRENTLY`	—	6,0 s; no bloquea lecturas (calcula la nueva versión y aplica sólo las diferencias); requiere índice `UNIQUE`

Se comprobó la desactualización insertando una reserva en marzo de 2029 para el hotel 13: la vista siguió sin fila para ese mes hasta el `REFRESH`, después del cual mostró 1 reserva.

Cuándo usar cada una

	Vista común	Vista materializada
Datos	Siempre actuales	Foto al momento del último `REFRESH`
Lectura	Tan cara como la consulta subyacente	Lectura de una tabla chica e indexable
Escritura	Sin costo	`REFRESH` completo (6 s aquí), periódico
En este sistema	Simplificar consultas de detalle y permisos	Tableros de gestión, ocupación e ingresos históricos, que toleran datos del día anterior

En MySQL/MariaDB no existen vistas materializadas: el equivalente es una tabla de resumen que se recalcula con un evento programado (`CREATE EVENT`) o se mantiene con triggers. En PostgreSQL, si se necesitara actualización incremental (sin recalcular todo), también habría que recurrir a triggers o a extensiones como `pg_ivm`.

Esta es la segunda forma de desnormalización controlada propuesta en el Punto 5: no compromete la integridad porque la fuente de verdad sigue siendo `reserva`.

Conclusiones

La conclusión central del trabajo es que el rendimiento de una base de datos se explica casi siempre por **cuántas páginas lee cada consulta y de dónde las lee**, y que esas páginas dependen más de cómo está escrito el SQL y de qué índices existen que del hardware. Las mejoras más grandes medidas (OFFSET a keyset: 9.166×; `NOT IN` a `NOT EXISTS`: más de 515×; conversión de tipo eliminada: 1.100×) no requirieron cambiar el servidor.

Hallazgos principales

1. **La integridad se garantiza en la base.** La restricción `EXCLUDE` con rangos de fechas impide reservas superpuestas incluso entre transacciones simultáneas (Punto 39), algo que la aplicación sola no puede lograr. Cuesta: la carga de 500 mil reservas pasó de 5,3 s a 25,7 s, y el particionamiento en PostgreSQL 16 no la admite.

2. **Un índice sirve si la consulta es SARGable y selectiva.** Funciones sobre columnas, conversiones de tipo y comodines iniciales anulan los índices (Punto 30); columnas de baja cardinalidad como `estado` sólo justifican índices parciales (Punto 17).
3. **Indexar todo empeora la escritura**: 13 índices hicieron 4,6 veces más lentos los `INSERT` sin acelerar las consultas que ya estaban cubiertas (Punto 18).
4. **El optimizador se equivoca cuando sus estadísticas no reflejan los datos**: estadísticas viejas (Nested Loop 3,3 veces más lento), columnas correlacionadas (129 mil filas estimadas para un resultado vacío) y correlación física engañosa (Index Scan 2,7 veces más lento que un Seq Scan) (Puntos 31 y 32).
5. **Medir requiere método.** La caché cambia los tiempos hasta 24 veces (Punto 23), y un detalle del cliente (sentencias preparadas automáticas) invalidó una primera ronda de mediciones forzadas, que hubo que repetir. Comparar páginas leídas además de tiempos evitó conclusiones falsas.
6. **MVCC tiene un costo de mantenimiento**: un `UPDATE` masivo duplicó la tabla y los índices; `VACUUM`, `ANALYZE` y, excepcionalmente, `VACUUM FULL` son parte del diseño, no tareas opcionales (Punto 48).
7. **Concurrencia**: `READ COMMITTED` alcanza para la operación diaria porque las reglas críticas están en restricciones; `SERIALIZABLE` es necesario para reglas que dependen de leer varias filas (Punto 40).
8. **Particionar no acelera todo**: ayuda en reportes de períodos completos y sobre todo en el mantenimiento (borrar un año: 14 ms contra 0,7 s), pero no reemplaza a un buen índice para consultas chicas (Punto 50).

Recomendaciones para el sistema de reservas

Prioridad	Acción	Puntos
1	Mantener las restricciones `EXCLUDE`, `CHECK` y FK; no moverlas a la aplicación	4, 8, 39
2	Escribir filtros de fechas como rangos y paginar con *keyset*	27, 28, 30
3	Índices: los de la estrategia del Punto 11 más `(huesped_id, fecha_inicio)` y `(hotel_id, fecha_inicio)`; eliminar los redundantes	11, 34–37
4	Pool de conexiones (PgBouncer o el de la aplicación) dimensionado a 2–4 conexiones por núcleo	46

Prioridad	Acción	Puntos
5	`shared_buffers` de 2 GB, `work_mem` moderado y `ANALYZE` tras cada carga	31, 43, 44
6	Vistas materializadas para los tableros de gestión, refrescadas cada noche	52
7	`pg_stat_statements` siempre activo y revisión periódica de consultas lentas e índices sin uso	18, 47
8	Evaluar particionamiento por año cuando el historial crezca, resolviendo antes cómo garantizar la no superposición entre particiones	50

Entregables

- Este informe, que incluye el script completo de creación (Punto 6), las consultas desarrolladas (Parte V), los planes de ejecución analizados y los resultados de cada experimento.

Las mediciones corresponden a la máquina descripta en el Punto 10. Repetidas en otro equipo, los tiempos absolutos cambiarán; las proporciones entre variantes y la cantidad de páginas leídas deberían mantenerse.